

TIA Linter Handbuch

TIA Linter — Benutzerhandbuch

Version dieses Handbuchs: 0.53 (Entwurf) **Stand:** 25.07.2026 **Programmversion:** 0.1.0

Hinweis zum Bearbeitungsstand: Dieses Handbuch ist inhaltlich vollständig — alle Kapitel 1–9 (grundsätzliche Funktion und Bedienung der Oberfläche) sowie Kapitel 10 mit allen 39 Prüfpunkten (siehe [Übersichtstabelle am Anfang von Kapitel 10](#)) sind ausgearbeitet. Es trägt weiterhin die Kennzeichnung “Entwurf”, bis eine erste vollständige Durchsicht stattgefunden hat.

Bild-Platzhalter: An über 20 Stellen steht bereits ein Bild-Platzhalter (`![Beschreibung]` (`images/dateiname.png`)), der auf einen noch zu erstellenden Screenshot bzw. eine Grafik verweist. Details zum Ergänzen der Bilder stehen in [Anhang B](#).

Inhaltsverzeichnis

1. Über dieses Handbuch
2. Was ist der TIA Linter?
3. Voraussetzungen
4. Grundprinzip der Prüfung
5. Installation und Einrichtung
6. Bedienung der Oberfläche
 1. Programmstart
 2. Die Eingabeseite
 3. Während der Prüfung
 4. Die Ergebnisseite
 5. Gemerkte Einstellungen
7. Der PDF-Report
8. Das Pfad-Format der Befunde
9. Die Protokolldatei (Log-Datei)
10. Die Prüfpunkte im Detail

Anhang

- A. Glossar
 - B. Aus diesem Handbuch ein PDF erstellen
 - C. Änderungshistorie dieses Handbuchs
-

1. Über dieses Handbuch

Dieses Handbuch richtet sich an alle, die mit dem **TIA Linter** ein TIA-Portal-Projekt prüfen möchten — unabhängig davon, ob sie das Programm selbst bedienen oder nur die Ergebnisse (den PDF-Report) lesen und verstehen wollen. Es setzt keine Programmierkenntnisse voraus.

Der Aufbau folgt bewusst vom Allgemeinen zum Speziellen:

- Die **Kapitel 1–9** erklären, was das Programm tut, wie es aufgebaut ist und wie man es bedient — unabhängig davon, welche einzelnen Prüfpunkte gerade aktiv sind.
- **Kapitel 10** enthält für jeden einzelnen der 39 Prüfpunkte eine eigene, ausführliche Beschreibung (was genau geprüft wird, warum das wichtig ist, und wie ein Befund behoben wird).

Wer dieses Dokument am Bildschirm liest (z. B. in Obsidian oder auf GitHub), kann über die Links im Inhaltsverzeichnis direkt zum jeweiligen Kapitel springen. Hinweise zur PDF-Erstellung stehen in [Anhang B](#).

2. Was ist der TIA Linter?

Der **TIA Linter** ist ein Prüfwerkzeug für **TIA-Portal-Projekte** von Siemens. Er untersucht ein Projekt automatisiert auf Qualität und Konventionen — zum Beispiel auf Kommentierung, Namensgebung, Programmstruktur, Hardware-Konfiguration, Projektmetadaten und die Einhaltung des Siemens-Styleguides — und fasst die Ergebnisse in einem übersichtlichen **PDF-Report** zusammen.

Man kann sich den TIA Linter wie eine **Rechtschreibprüfung für SPS-Programme** vorstellen: Er liest das Projekt, vergleicht es gegen einen Satz konfigurierbarer Regeln und meldet Abweichungen — er greift dabei selbst nicht in das Projekt ein.

Wofür ist das Programm gedacht?

- **Qualitätssicherung** vor der Inbetriebnahme oder Übergabe eines Projekts
- **Wiederkehrende Prüfung** nach einem einheitlichen, dokumentierten Maßstab (statt manueller Sichtprüfung durch verschiedene Personen)
- **Nachvollziehbare Berichte** (PDF), die z. B. einer Abnahme oder internen Dokumentation beigelegt werden können

Was das Programm ausdrücklich *nicht* tut

- Es **verändert keine SPS-Konfiguration**, keinen Programmcode und keine Steuerung, und speichert das Projekt zu keinem Zeitpunkt. Eine Ausnahme: [Prüfpunkt 21](#) kompiliert das Projekt am Ende des Prüflaufs im Speicher. Das Projekt wird dabei nicht gespeichert — alle anderen Prüfpunkte laufen davor und sind davon nicht betroffen.
- Es ersetzt **keine funktionale Sicherheitsprüfung**. Bei sicherheitsrelevanten Systemen (funktionale Sicherheit, SIL, Performance Level) ist weiterhin eine unabhängige Prüfung durch eine qualifizierte Fachkraft erforderlich.

Was am Ende dabei herauskommt

Jeder Prüflauf erzeugt zwei Ergebnisse:

1. Eine **Log-Datei** mit dem vollständigen, zeitlich geordneten Ablauf der Prüfung (siehe [Kapitel 9](#)).
2. Auf Wunsch einen **PDF-Report** mit Deckblatt, Zusammenfassung und allen Einzelbefunden (siehe [Kapitel 7](#)).

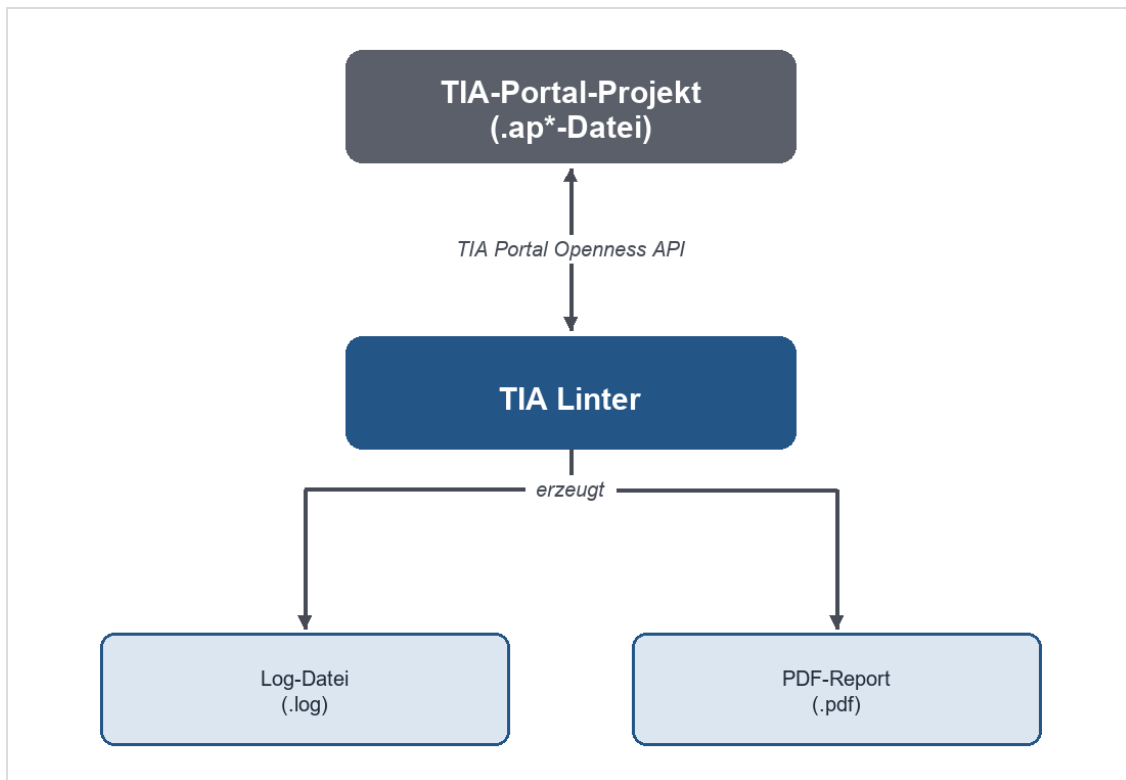


Schaubild: Der TIA Linter verbindet sich über die TIA Portal Openness API mit einem TIA-Portal-Projekt und erzeugt daraus eine Log-Datei sowie einen PDF-Report

3. Voraussetzungen

Damit der TIA Linter ein echtes Projekt prüfen kann, müssen folgende Voraussetzungen erfüllt sein:

Voraussetzung	Anmerkung
TIA Portal V21	Aktuell die einzige unterstützte Version. Weitere Versionen sind vorgesehen.
Windows	Die Schnittstelle zu TIA Portal (Openness API) ist eine Windows-Komponente.
Python 3.11 oder neuer	Wird für die Installation und den Betrieb des Programms benötigt.

Wichtig: TIA Portal muss während der Prüfung **nicht geöffnet** sein. Der Zugriff erfolgt im Hintergrund ("headless"). Näheres dazu in [Kapitel 4](#).

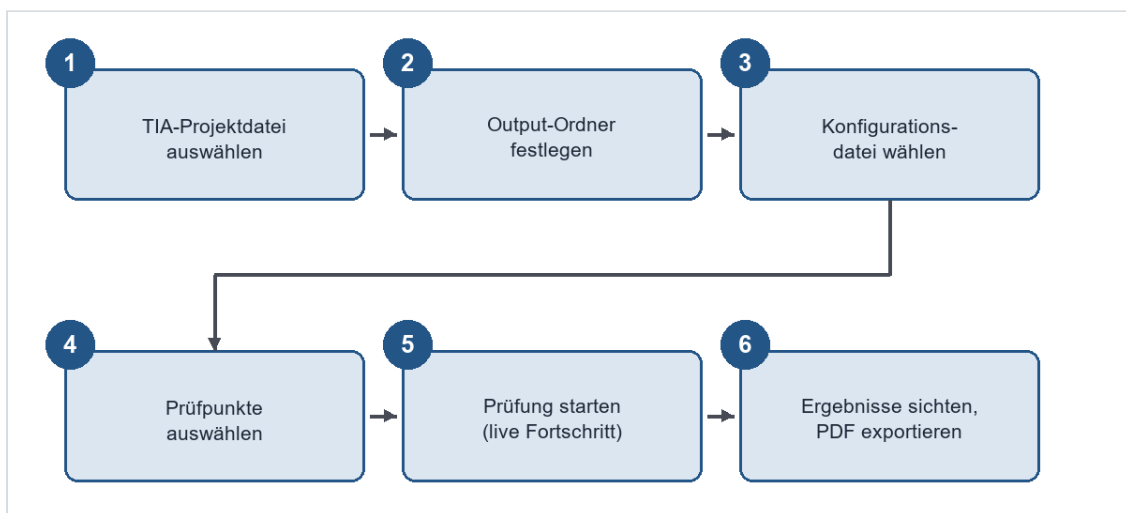
Für reine Testzwecke ohne echtes TIA-Portal-Projekt steht zusätzlich ein **Testmodus** zur Verfügung, der mit Beispielbefunden arbeitet (siehe [Abschnitt 6.2](#)). In diesem Modus entfallen die oben genannten Voraussetzungen.

4. Grundprinzip der Prüfung

4.1 Ablauf in Kurzform

Ein Prüflauf besteht immer aus denselben Schritten:

1. Eine **TIA-Projektdatei** (`.ap*`) wird ausgewählt.
2. Ein **Output-Ordner** wird festgelegt, in dem später die Log-Datei und der PDF-Report gespeichert werden.
3. Eine **Konfigurationsdatei** legt fest, welche Prüfpunkte es überhaupt gibt, welche davon standardmäßig aktiv sind und mit welchen Schwellenwerten sie arbeiten.
4. Die gewünschten **Prüfpunkte** werden ausgewählt (einzeln oder kategorienweise).
5. Die Prüfung wird gestartet — Fortschritt und Meldungen laufen live in der Oberfläche mit.
6. Auf der Ergebnisseite können die Befunde gesichtet, gefiltert und abschließend als PDF-Report exportiert werden.



Ablaufschema der sechs Schritte eines Prüflaufs, von der Auswahl der Projektdatei bis zum fertigen PDF-Report

4.2 Wie die Verbindung zu TIA Portal funktioniert

Der TIA Linter verbindet sich über die **TIA Portal Openness API** mit dem Projekt. Das ist eine offizielle Programmierschnittstelle von Siemens, über die externe Programme lesend (und grundsätzlich auch schreibend) auf TIA-Portal-Projekte zugreifen können. Der TIA Linter nutzt ausschließlich die **lesenden** Möglichkeiten dieser Schnittstelle.

Die Verbindung wird **ohne sichtbare TIA-Portal-Oberfläche** aufgebaut. Es ist also nicht nötig (und nicht vorgesehen), TIA Portal vorher manuell zu öffnen.

Bei größeren Projekten baut das Programm die Verbindung zwischendurch **automatisch neu auf** (Reconnect). Das ist ein reiner Stabilitätsmechanismus im Hintergrund: TIA Portal begrenzt die Anzahl gleichzeitig offener Objekte je Sitzung, und ältere TIA-Portal-Versionen haben sich bei sehr langen Sitzungen

gelegentlich unerwartet getrennt. Ein Reconnect unterbricht die Prüfung nicht — bereits abgeschlossene Prüfpunkte werden nicht wiederholt, und der Vorgang wird im Log sichtbar protokolliert.

Ein weiterer Performance-Mechanismus im Hintergrund: Exportiert ein Prüfpunkt den XML-Inhalt eines Bausteins oder die Projektttexte des Projekts, wird das Ergebnis für den Rest des Prüflaufs zwischengespeichert — andere Prüfpunkte, die denselben Inhalt brauchen, exportieren ihn nicht erneut. Bei einem Reconnect wird dieser Zwischenspeicher automatisch geleert und bei Bedarf neu aufgebaut, da danach ohnehin neue Objekte von TIA Portal geholt werden.

4.3 Wie die Prüfpunkte organisiert sind

Damit die Oberfläche bei einer größeren Anzahl an Prüfpunkten übersichtlich bleibt, sind alle Prüfpunkte **Kategorien** zugeordnet. Diese Kategorien bilden die Gruppen, in denen die Prüfpunkte auf der Eingabeseite als Kontrollkästchen angezeigt werden (siehe [Abschnitt 6.2](#)):

Kategorie
Kommentare & Beschreibungen
Namenskonventionen
Programmstruktur
Hardware & Konfiguration
Projektmetadaten
Bibliotheken & Typen
Siemens Styleguide

Was innerhalb dieser Kategorien im Einzelnen geprüft wird, ist **nicht** Teil dieses Kapitels. Jeder einzelne Prüfpunkt wird ausführlich in [Kapitel 10](#) beschrieben, sobald dieser Teil des Handbuchs ergänzt ist.

4.4 Wie ein Befund bewertet wird

Jeder Befund im Report hat genau einen von drei möglichen Status:

Status	Bedeutung
OK	Der Prüfpunkt wurde vollständig eingehalten. Kein Handlungsbedarf.
Warnung	Eine Abweichung von der Konvention wurde festgestellt, die den Betrieb nicht unmittelbar gefährdet.
Fehler	Eine schwerwiegende Abweichung wurde festgestellt, die dringend behoben werden sollte.

Ob eine konkrete Abweichung als Warnung oder als Fehler gilt, ist für jeden Prüfpunkt einzeln in der Konfigurationsdatei festgelegt (siehe [Kapitel 5](#)) und kann bei Bedarf angepasst werden.

Wie “OK” zustande kommt: Ein Prüfpunkt meldet grundsätzlich nur tatsächliche Verstöße einzeln (ein Fehler- oder Warnungs-Befund pro betroffenem Objekt, z. B. pro unkommentierter Variable) — für Objekte ohne Verstoß entsteht **kein** eigener Befund. Läuft ein Prüfpunkt über das gesamte Projekt hinweg vollständig ohne einen einzigen Fehler oder eine einzige Warnung durch, erzeugt er stattdessen **genau einen** zusammenfassenden OK-Befund für sich selbst (nicht einen OK-Befund pro geprüftem Objekt) — ohne diesen zusammenfassenden Befund wäre ein vollständig sauberer Prüfpunkt im Report nirgends sichtbar, weder als Fehler/Warnung noch als OK. Die “OK”-Zeile in der Gesamtübersicht der GUI (siehe [Abschnitt 6.4](#)) zählt also nicht Objekte, sondern **vollständig fehlerfreie Prüfpunkte**.

5. Installation und Einrichtung

5.1 Installation

Das Programm wird als Python-Paket installiert:

```
pip install -e .
```

Danach steht der Befehl `tia-linter` zur Verfügung, mit dem sich die Oberfläche starten lässt (siehe [Kapitel 6](#)).

5.2 Die Konfigurationsdatei

Alle grundsätzlichen Einstellungen des Programms — welche TIA-Portal-Version verwendet wird, welche Prüfpunkte es gibt, welche davon standardmäßig aktiv sind und mit welchen Schwellenwerten sie arbeiten — stehen in einer **YAML-Konfigurationsdatei**. Mitgeliefert wird eine Standardkonfiguration unter `config/default.yaml`.

Es können beliebig viele eigene Konfigurationsdateien angelegt werden, zum Beispiel eine je Kunde oder Projektstandard. Auf der Eingabeseite der Oberfläche lässt sich die verwendete Konfigurationsdatei jederzeit umschalten (siehe [Abschnitt 6.2](#)) — die Liste der Prüfpunkt-Kontrollkästchen passt sich dann automatisch an.

Die Konfigurationsdatei ist auch der Ort, an dem der **DLL-Pfad** der installierten TIA-Portal-Version hinterlegt ist. Dieser muss zur tatsächlichen Installation auf dem verwendeten Rechner passen.

```
! default.yaml X
config > ! default.yaml
1 # TIA Linter – Standard-Konfiguration
2 #
3 # Enthält alle 39 Prüfpunkte aus Pruefpunkte.md (einige Prüfpunkte sind hier
4 # als mehrere separat schaltbare Einträge abgebildet, z. B. Prüfpunkt 6 als
5 # plc_tag_eingaenge + plc_tag_ausgaenge).
6 #
7 # enabled – Standardzustand den Checkbox in der GUI
8 # severity – Standard-Schweregrad bei Verstoß: warning | error
9 # weitere Schlüssel je Prüfpunkt: Regex, Schwellenwerte, Listen, ...
10
11 report:
12   pruefer: ""
13   firma: ""
14   logo_pfad: ""
15
16 logging:
17   level: "INFO"
18   file: null
19   console: true
20
21 tia_versionen:
22   verfuegbar:
23     - name: "TIA Portal V21"
24       version: 21
25       dll_pfad: "C:\\Program Files\\Siemens\\Automation\\Portal V21\\PublicAPI\\V21\\net48\\Siemens.Engineering.Base.dll"
26   standard: "TIA Portal V21"
27
28 # Maximale Anzahl automatischer Reconnect-Versuche, falls die TIA-Portal-
29 # Session während der Prüfung unerwartet stirbt (bekannt von TIA V19, siehe
30 # README). Bereits abgeschlossene Prüfpunkte werden bei einem Reconnect nicht
31 # wiederholt.
32 max_reconnect_attempts: 3
33
34 # Nach so vielen Prüfpunkten wird die TIA-Portal-Verbindung vorsorglich neu
35 # aufgebaut (unabhängig von echten Verbindungsabbrüchen). Grund: TIA Portal
36 # Openness zählt offene Objekt-Instanzen pro Session und wirft ab einem
37 # harten Limit (beobachtet: 500.000) eine EngineeringOutOfMemoryException,
38 # lange bevor ein größeres Projekt vollständig geprüft ist – siehe README.
39 reconnect_every_n_checks: 10
40
41 # Innerhalb der Prüfpunkte 11a/11b/12a/12b/13 (CrossReferenceService pro Tag/
42 # Baustein, siehe structure.py) wird alle gc_interval verarbeiteten Objekte
43 # zusätzlich eine .NET-Garbage-Collection erzwungen (ohne Reconnect) – bei
44 # sehr großen Projekten (mehrere Tausend Tags/Bausteine) kann sonst bereits
45 # ein einzelner dieser Checks das oben genannte Handle-Limit reißen, bevor
46 # der Check zu Ende ist und reconnect_every_n_checks überhaupt greift.
47 gc_interval: 200
48
49 # Maximale Anzahl gleichzeitig gecachter Baustein-XML-Exporte (LRU-
50 # Verdrängung des am längsten unbenutzten Eintrags, siehe _tia_helpers.py:
51 # export_block_xml). Der Cache bleibt über Reconnects hinweg erhalten (siehe
52 # runner.py::reset_run_caches) und wird nur zu Lauf-Beginn geleert – ohne
53 # Begrenzung würde er bei sehr großen Projekten (mehrere Tausend Bausteine)
54 # unbeschränkt wachsen (mehrere GB RAM möglich).
55 xml_cache_max_size: 500
56
57 # Maximale Anzahl gleichzeitig gecachter CrossReference-Abfragen (LRU, siehe
58 # _tia_helpers.py::cached_cross_reference_locations). Wie der XML-Cache bleibt
59 # er über Reconnects hinweg erhalten und wird nur zu Lauf-Beginn geleert –
60 # deckt echte Mehrfachabfragen ab, z. B. fragt Prüfpunkt 11a jedes Tag mit
61 # demselben Filter ab wie anschließend 12a/12b/13 für Ein-/Ausgänge.
62 xref_cache_max_size: 1000
63
```

Ausschnitt der Datei config/default.yaml in einem Text-Editor, mit den Abschnitten tia_versionen und checks

Ordner von der Prüfung ausnehmen

Über den Schlüssel `ausgeschlossene_ordner` lässt sich eine Liste von Ordernamen hinterlegen, die komplett von der Prüfung ausgenommen werden sollen — zum Beispiel ein Bibliotheksordner, der bereits mehrfach geprüft wurde und sich nicht mehr ändert:

```
ausgeschlossene_ordner:
- "Standardbibliothek"
- "Bereits geprüft"
```

Ein ausgeschlossener Ordner nimmt automatisch auch **alle seine Unterordner** von der Prüfung aus. Der Vergleich erfolgt rein anhand des Ordernamens (ohne Groß-/Kleinschreibung) — unabhängig davon, an

welcher Stelle im Projekt ein Ordner mit diesem Namen liegt. Die Einstellung gilt gleichermaßen für Programmbausteine-, Datenbaustein- und Variablentabellen-Ordner.

Einzelne Bausteine von der Prüfung ausnehmen

Über den Schlüssel `ausgeschlossene_bausteine` lässt sich zusätzlich eine Liste einzelner Bausteinennamen hinterlegen (FB, FC, OB oder DB) — zum Beispiel ein einzelner Altsystem-Baustein, der bewusst nicht mehr an aktuelle Konventionen angepasst werden soll:

```
ausgeschlossene_bausteine:  
- "FB_Altsystem"  
- "DB_Legacy_Rezepte"
```

Ein hier eingetragener Baustein wird **komplett von jedem Prüfpunkt** ausgenommen — sowohl von Namensprüfungen (z. B. [Prüfpunkt 5](#) oder [7](#)) als auch von Inhaltsprüfungen (z. B. Kommentare, unbenutzte Variablen, Netzwerkkomplexität) — unabhängig davon, in welchem Ordner der Baustein liegt. Der Vergleich erfolgt wie bei `ausgeschlossene_ordner` ohne Berücksichtigung von Groß-/Kleinschreibung.

Anders als `ausgeschlossene_ordner` gilt diese Einstellung ausschließlich für Programmbausteine und Datenbausteine — nicht für Variablentabellen oder einzelne PLC-Tags. Wer einzelne Variablen von der Kommentarprüfung ausnehmen möchte, nutzt stattdessen `ausnahme_prefixe` bei [Prüfpunkt 1a](#).

5.3 Angaben für den Report

In der Konfigurationsdatei lassen sich außerdem allgemeine Angaben für das Deckblatt des PDF-Reports hinterlegen, etwa der Name des Prüfers oder der Firma (siehe [Kapitel 7](#)).

6. Bedienung der Oberfläche

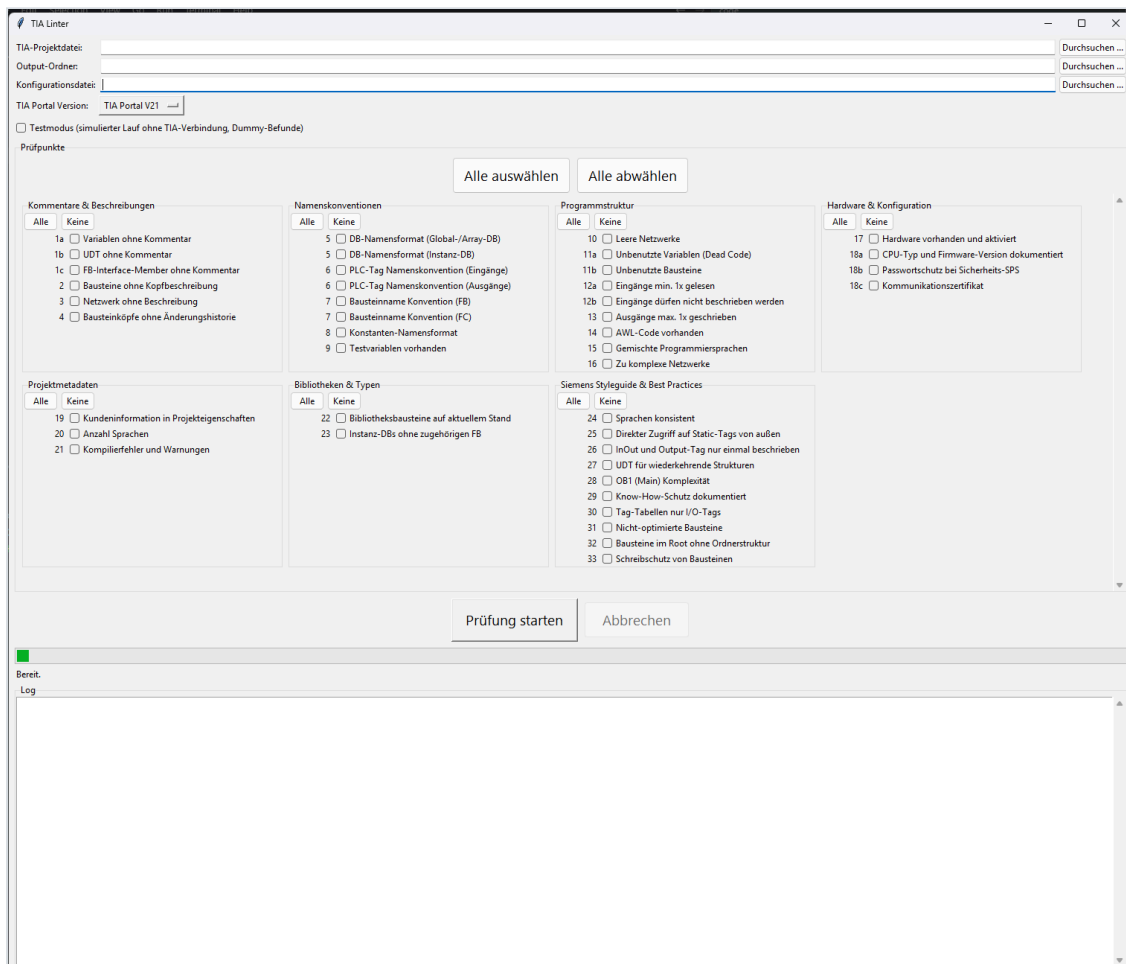
Die Oberfläche des TIA Linters besteht aus zwei Seiten, zwischen denen das Programm automatisch wechselt: der **Eingabeseite** (vor und während der Prüfung) und der **Ergebnisseite** (nach Abschluss der Prüfung).

6.1 Programmstart

Das Programm wird über den Befehl

```
tia-linter
```

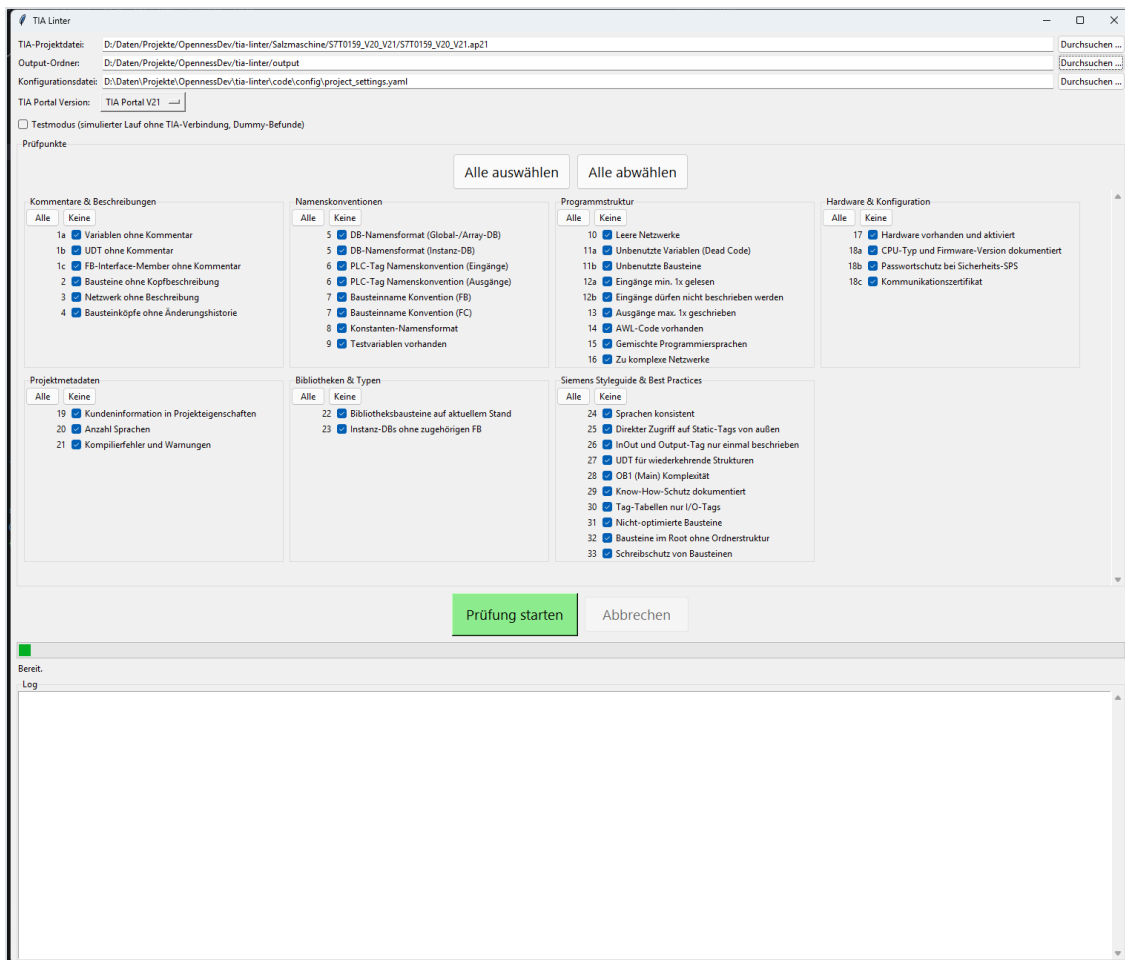
gestartet. Es öffnet sich ein einzelnes Fenster mit der Eingabeseite. Fenstergröße und zuletzt verwendete Einstellungen werden automatisch wiederhergestellt (siehe [Abschnitt 6.5](#)).



Das TIA-Linter-Fenster direkt nach dem Start mit der leeren Eingabeseite

6.2 Die Eingabeseite

Die Eingabeseite gliedert sich von oben nach unten in folgende Bereiche:



Die vollständige Eingabeseite mit ausgefüllten Feldern für Projektdatei, Output-Ordner, Konfigurationsdatei und TIA-Portal-Version

1. Eingabefelder

Feld	Beschreibung
TIA-Projektdatei	Über “Durchsuchen ...” wird die zu prüfende Projektdatei (.ap*) ausgewählt.
Output-Ordner	Der Ordner, in dem Log-Datei und PDF-Report abgelegt werden.
Konfigurationsdatei	Die YAML-Datei mit den Prüfpunkt-Definitionen (siehe Abschnitt 5.2). Beim Wechsel wird die Liste der Prüfpunkte unten automatisch neu aufgebaut.
TIA Portal Version	Auswahl der zu verwendenden TIA-Portal-Version aus den in der Konfiguration hinterlegten Versionen.

2. Testmodus

Über die Kontrollkästchen “Testmodus” lässt sich ein simulierter Prüflauf **ohne echte TIA-Portal-Verbindung** durchführen, der mit Beispielbefunden arbeitet. Das ist nützlich, um die Oberfläche, die Filterfunktionen oder den PDF-Report kennenzulernen, ohne ein echtes Projekt zur Hand zu haben. Der Testmodus ist standardmäßig **ausgeschaltet** — im Normalbetrieb wird also echt geprüft.

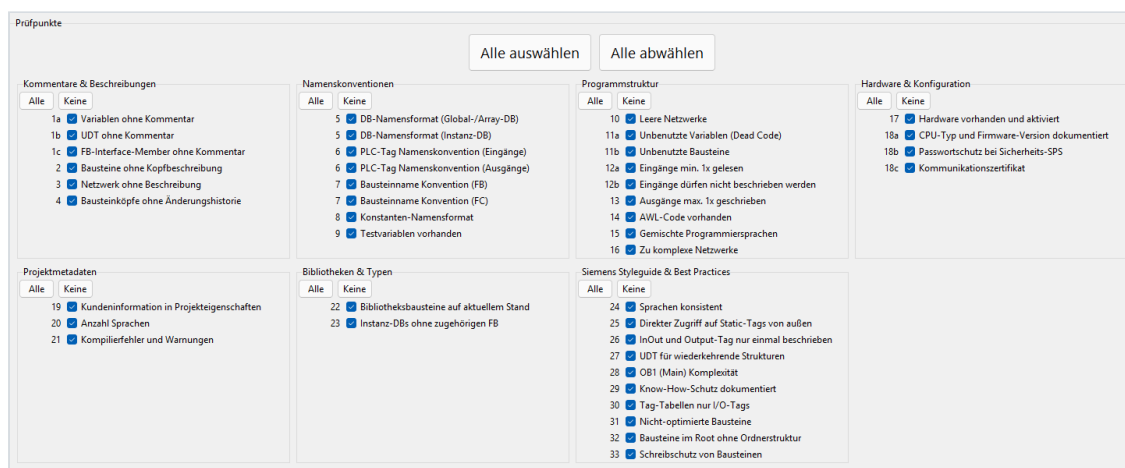
3. Prüfpunkte

In diesem Bereich werden alle verfügbaren Prüfpunkte als Kontrollkästchen angezeigt, gruppiert nach Kategorie (siehe [Abschnitt 4.3](#)). Links neben jedem Kontrollkästchen steht die zugehörige Prüfpunkt-Nummer aus diesem Handbuch (z. B. “17” oder “18c”) — damit lässt sich ein Kontrollkästchen direkt einem Abschnitt in [Kapitel 10](#) zuordnen, ohne den Namen abgleichen zu müssen. Zur schnellen Auswahl stehen zur Verfügung:

- “Alle auswählen” / “Alle abwählen” — wirkt auf sämtliche Prüfpunkte. Beide Buttons sind vergrößert und mittig über dem Prüfpunkte-Bereich angeordnet.
- “Alle” / “Keine” je Kategorie — wirkt nur auf die Prüfpunkte der jeweiligen Kategorie.
- Einzelne Kontrollkästchen für jeden Prüfpunkt.

Welche Prüfpunkte hier standardmäßig angehakt sind, ist in der aktiven Konfigurationsdatei festgelegt.

Die Kategorien stehen zu je drei nebeneinander (erste Zeile z. B. “Kommentare & Beschreibungen”, “Namenskonventionen”, “Programmstruktur”), statt strikt untereinander — das nutzt die verfügbare Fensterbreite besser aus. Innerhalb des gesamten Prüfpunkte-Bereichs scrollt das Mausrad, egal über welchem Kontrollkästchen oder welcher Kategorie sich der Mauszeiger gerade befindet — nicht nur direkt über der Bildlaufleiste.



Der Prüfpunkte-Bereich der Eingabeseite mit den nach Kategorie gruppierten Kontrollkästchen sowie den Buttons “Alle auswählen”/“Alle abwählen” und “Alle”/“Keine” je Kategorie

4. Start, Fortschritt und Log

- “**Prüfung starten**” beginnt den Prüflauf mit den aktuell ausgewählten Einstellungen. Der Button ist deaktiviert, solange keine Projektdatei, kein Output-Ordner oder kein Prüfpunkt ausgewählt ist. Er färbt sich hellgrün, sobald mindestens ein Prüfpunkt angehakt ist — als visueller Hinweis, dass der Lauf startbereit ist.
- “**Abbrechen**” ist nur während eines laufenden Prüflaufs aktiv und bricht diesen kontrolliert ab.
- Beide Buttons sind vergrößert und mittig unter dem Prüfpunkte-Bereich angeordnet.
- Eine **Fortschrittsanzeige** und eine **Statuszeile** zeigen, dass eine Prüfung läuft bzw. welche Meldung zuletzt eingegangen ist.
- Ein **Log-Fenster** zeigt alle Meldungen des laufenden Prüflaufs live und fortlaufend an.

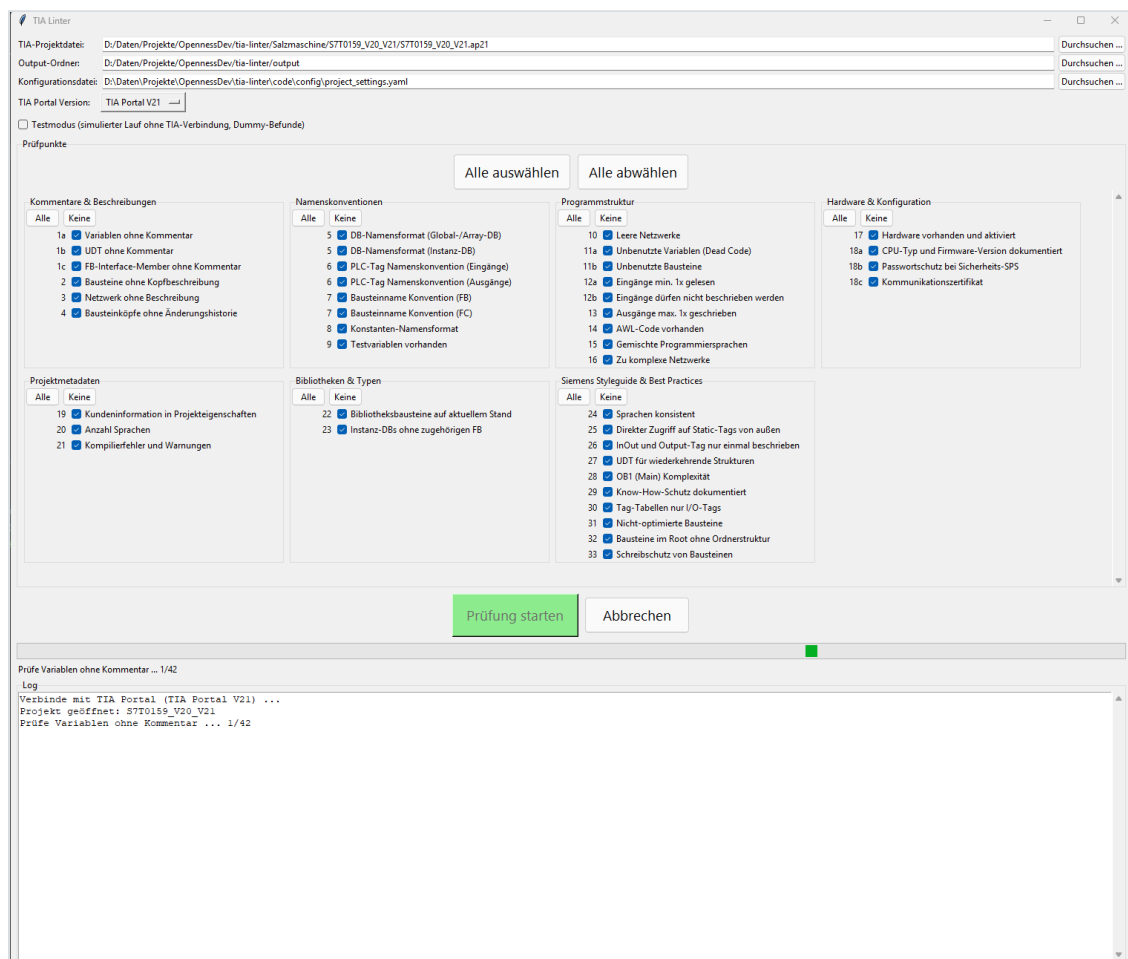
6.3 Während der Prüfung

Während die Prüfung läuft, bleibt die Oberfläche bedienbar. Im Log-Fenster erscheinen laufend Meldungen zum Fortschritt, unter anderem:

- welcher Prüfpunkt gerade bearbeitet wird,
- falls nötig, Hinweise zu einem automatischen Neuverbinden mit TIA Portal (siehe [Abschnitt 4.2](#)),
- eine Meldung, sobald die Prüfung abgeschlossen ist.

Über den Button **“Abbrechen”** kann die Prüfung jederzeit vorzeitig beendet werden. Ein Abbruch wird angefordert und beim nächstmöglichen Zeitpunkt ausgeführt — bereits ermittelte Befunde gehen dabei nicht verloren, es werden lediglich keine weiteren Prüfpunkte mehr bearbeitet.

Sobald die Prüfung abgeschlossen ist, wechselt das Programm automatisch zur Ergebnisseite.



Die Eingabeseite während eines laufenden Prüflaufs: aktive Fortschrittsanzeige, live einlaufende Meldungen im Log-Fenster und aktivierter Abbrechen-Button

6.4 Die Ergebnisseite

Die Ergebnisseite zeigt die Befunde des zuletzt abgeschlossenen Prüflaufs.

TIA Linter				
18 Fehler 312 Warnungen 0 OK				
Status: (Alle Status) Kategorie: (Alle Kategorien) Pfad: Pfad / Baustein filtern...				
Status	Prüfpunkt	Pfad	Beschreibung	
Warnung	Variablen ohne Kommentar	pn4805-15a1 > Variablen tabellen > Default tag table > TEST_Input	Variable 'TEST_Input' hat keinen Kommentar.	
Warnung	Variablen ohne Kommentar	pn4805-15a1 > Variablen tabellen > Outputs > NoOutput3	Variable 'NoOutput3' hat keinen Kommentar.	
Warnung	Variablen ohne Kommentar	pn4805-15a1 > Variablen tabellen > Outputs > o0ol0(1)	Variable 'o0ol0(1)' hat keinen Kommentar.	
Warnung	Variablen ohne Kommentar	pn4805-15a1 > Variablen tabellen > Inputs > Sensor2	Variable 'Sensor2' hat keinen Kommentar.	
Warnung	Variablen ohne Kommentar	pn4805-15a1 > Variablen tabellen > Inputs > Lueckenschalter	Variable 'Lueckenschalter' hat keinen Kommentar.	
Warnung	Variablen ohne Kommentar	pn4805-15a1 > Variablen tabellen > Inputs > o0ol0	Variable 'o0ol0' hat keinen Kommentar.	
Warnung	Variablen ohne Kommentar	pn4805-15a1 > Datenbaustein > Visu > V405Customer > Member > InPotFree	DB-Variable 'InPotFree' hat keinen Kommentar.	
Warnung	Variablen ohne Kommentar	pn4805-15a1 > Datenbaustein > 01 [4805] DrySaltingMachine > A03 > Member > '4740	DB-Variable '4740_30M1'.CycleCounter.CV hat keinen Kommentar.	
Warnung	Variablen ohne Kommentar	pn4805-15a1 > Datenbaustein > 01 [4805] DrySaltingMachine > A03 > Member > '4740	DB-Variable '4740_30M1'.CycleCounter.CD hat keinen Kommentar.	
Warnung	Variablen ohne Kommentar	pn4805-15a1 > Datenbaustein > 01 [4805] DrySaltingMachine > A03 > Member > '4740	DB-Variable '4740_30M1'.CycleCounter.R hat keinen Kommentar.	
Warnung	Variablen ohne Kommentar	pn4805-15a1 > Datenbaustein > 01 [4805] DrySaltingMachine > A03 > Member > '4740	DB-Variable '4740_30M1'.CycleCounter.LD hat keinen Kommentar.	
Warnung	Variablen ohne Kommentar	pn4805-15a1 > Datenbaustein > 01 [4805] DrySaltingMachine > A03 > Member > '4740	DB-Variable '4740_30M1'.CycleCounter.QD hat keinen Kommentar.	
Warnung	Variablen ohne Kommentar	pn4805-15a1 > Datenbaustein > 01 [4805] DrySaltingMachine > A03 > Member > '4740	DB-Variable '4740_30M1'.CycleCounter.QV hat keinen Kommentar.	
Warnung	Variablen ohne Kommentar	pn4805-15a1 > Datenbaustein > 01 [4805] DrySaltingMachine > A03 > Member > '4740	DB-Variable '4740_30M1'.CycleCounter.CV hat keinen Kommentar.	
Warnung	Variablen ohne Kommentar	pn4805-15a1 > Datenbaustein > 01 [4805] DrySaltingMachine > A03 > Member > '4730	DB-Variable '4730_30S1' hat keinen Kommentar.	
Warnung	Variablen ohne Kommentar	pn4805-15a1 > Datenbaustein > 01 [4805] DrySaltingMachine > A03 > Member > '4730	DB-Variable '4730_30S1'.Ref hat keinen Kommentar.	
Warnung	UDT ohne Kommentar	pn4805-15a1 > PLC-Datentypen > TypeProject > U_EdnCustomer	PLC-Datentyp 'U_EdnCustomer' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 40 [4805] General > 40Alm > Member > It_Ton4	FB-Variable 'It_Ton4805_15A15' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 40 [4805] General > 40Alm > Member > It_Ton6	FB-Variable 'It_Ton6512' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 40 [4805] General > 40Alm > Member > It_Ton4	FB-Variable 'It_Ton4A21' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 40 [4805] General > 40Alm > Member > It_xc	FB-Variable 'It_xc' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > 01Vis > Member	FB-Variable 'test' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > 01OrigPrg > Mem	FB-Variable 'It_RcpReq' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > 01OrigPrg > Mem	FB-Variable 'It_xc' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable '14805_33S1' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'Id_27M41BlowOffTime' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'Id_DistanceSensorToFoller' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_DistanceSensorToValve' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_Length1Rack' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_Length2Racks' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_SpTemperatureHighLimit' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_SpTemperatureLowLimit' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_46A11TemperatureAlm' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_Tof27M11On' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_Tof27M21On' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_Tof27M31On' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_Tof27M41On' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_Tof33M1On' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_TofBlowOffRampDelay' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_TofCustomerReady' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_TofS1Mute' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_Ton33M1On' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_Ton46A1A3Alm' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_Ton46A1A3On' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_27M11' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_27M21' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_27M31' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_27M41' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_27Z51' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_30M1' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_33M1M2' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_46A11' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_46A1A3' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_27M41BlowOff' hat keinen Kommentar.	
Warnung	FB-Interface-Member ohne Kommentar	pn4805-15a1 > Programmabusteine > 01 [4805] DrySaltingMachine > [4805] DrySalting	FB-Variable 'It_20M11' hat keinen Kommentar.	

Die vollständige Ergebnissseite mit Zusammenfassungszeile, Filterleiste und der farblich markierten Befundtabelle

1. Zusammenfassung

Am oberen Rand steht eine kurze Zusammenfassung mit der Gesamtzahl an Fehlern, Warnungen und OK-Befunden (siehe [Abschnitt 4.4](#)).

2. Filter

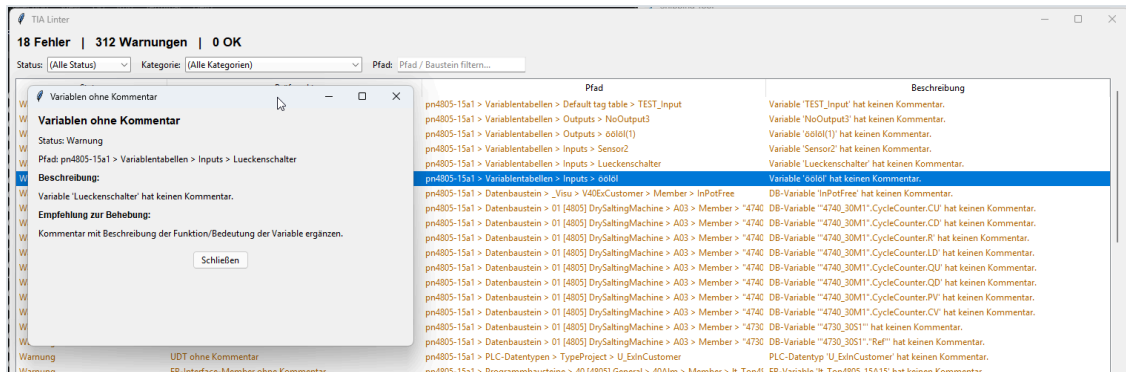
Die Befundtabelle lässt sich über drei Filter eingrenzen, die sich beliebig kombinieren lassen:

Filter	Wirkung
Status	Zeigt nur Befunde mit einem bestimmten Status (Fehler, Warnung, OK) oder alle.
Kategorie	Zeigt nur Befunde einer bestimmten Kategorie oder alle.
Pfad	Freitextsuche innerhalb des Pfads jedes Befunds (siehe Kapitel 8), z. B. um alle Befunde zu einem bestimmten Baustein zu finden.

3. Befundtabelle

Jede Zeile der Tabelle zeigt einen einzelnen Befund mit den Spalten **Status**, **Prüfpunkt**, **Pfad** und **Beschreibung**. Der Status ist farblich hervorgehoben: **Fehler** in Rot, **Warnung** in Orange, **OK** in Grün.

Ein **Doppelklick** auf eine Zeile öffnet ein Detailfenster mit dem vollständigen Pfad, der vollständigen Beschreibung des Befunds sowie einer **Empfehlung zur Behebung**.



Das Detailfenster eines einzelnen Befunds nach Doppelklick auf eine Zeile der Befundtabelle, mit Status, Pfad, Beschreibung und Empfehlung zur Behebung

4. Aktionen

- **“PDF-Report erstellen”** erzeugt den vollständigen PDF-Report im gewählten Output-Ordner (siehe Kapitel 7).
- **“Neue Prüfung”** kehrt zur Eingabeseite zurück, ohne die zuletzt verwendeten Einstellungen zu verwerfen.

6.5 Gemarkte Einstellungen

Das Programm merkt sich zwischen zwei Programmstarts automatisch:

- die Fenstergröße,
- die zuletzt verwendete Projektdatei und den zuletzt verwendeten Output-Ordner,
- die zuletzt verwendete Konfigurationsdatei,
- die zuletzt gewählte TIA-Portal-Version,
- den Zustand des Testmodus-Kontrollkästchens.

Diese Angaben werden lokal gespeichert und beim nächsten Start automatisch wieder vorausgefüllt.

7. Der PDF-Report

Der PDF-Report ist das zentrale Ergebnisdokument einer Prüfung und wird über den Button “PDF-Report erstellen” auf der Ergebnisseite erzeugt (siehe [Abschnitt 6.4](#)). Er ist im DIN-A4-Format aufgebaut und gliedert sich in fünf Teile:

1. Deckblatt

Enthält den Projektnamen, die verwendete TIA-Portal-Version, das Prüfdatum sowie — sofern in der Konfiguration hinterlegt — den Namen des Prüfers und der Firma (siehe [Abschnitt 5.3](#)).

TIA Linter — Prüfbericht

S7T0159_V20_V21
TIA-Portal-Version: TIA Portal V21
Prüfdatum: 18.07.2026 14:57

Beispiel-Deckblatt eines PDF-Reports mit Projektname, TIA-Portal-Version, Prüfdatum, Prüfer und Firma

2. Prüfpunkte-Übersicht

Eine Tabelle mit **allen** im TIA Linter vorhandenen Prüfpunkten (nicht nur den in diesem Lauf aktivierten) — je Zeile: ob der Prüfpunkt durchgeführt wurde, ob er dabei technisch erfolgreich durchgelaufen ist (im Unterschied zu einem inhaltlichen Verstoß, den ein Prüfpunkt meldet), sowie die Anzahl Fehler und

Warnungen. Eine abschließende Gesamtzeile fasst diese Spalten zusammen. So lässt sich auf einen Blick erkennen, welche Prüfpunkte überhaupt gelaufen sind, ohne erst die Detailseiten durchsuchen zu müssen.

3. Zusammenfassung

Eine Übersichtsseite mit der Gesamtzahl an Fehlern, Warnungen und OK-Befunden sowie einer Aufschlüsselung dieser Zahlen je Kategorie.

Zusammenfassung

Fehler	Warnungen	OK
18	312	0

Nach Kategorie

Kategorie	Fehler	Warnungen	OK
Kommentare & Beschreibungen	0	66	0
Namenskonventionen	18	2	0
Programmstruktur	0	244	0

4. Details

Für jede Kategorie eine eigene Tabelle mit allen zugehörigen Befunden (Status, Pfad, Beschreibung, Empfehlung zur Behebung) — farblich hervorgehoben analog zur Befundtabelle in der Oberfläche.

Details

Kommentare & Beschreibungen

Status	Pfad	Beschreibung	Empfehlung
Warnung	pn4805-15a1 > Variablentabellen > Default tag table > TEST_Input	Variable 'TEST_Input' hat keinen Kommentar.	Kommentar mit Beschreibung der Funktion/Bedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Variablentabellen > Outputs > NoOutput3	Variable 'NoOutput3' hat keinen Kommentar.	Kommentar mit Beschreibung der Funktion/Bedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Variablentabellen > Outputs > öölöl(1)	Variable 'öölöl(1)' hat keinen Kommentar.	Kommentar mit Beschreibung der Funktion/Bedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Variablentabellen > Inputs > Sensor2	Variable 'Sensor2' hat keinen Kommentar.	Kommentar mit Beschreibung der Funktion/Bedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Variablentabellen > Inputs > Lueckenschalter	Variable 'Lueckenschalter' hat keinen Kommentar.	Kommentar mit Beschreibung der Funktion/Bedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Variablentabellen > Inputs > öölöl	Variable 'öölöl' hat keinen Kommentar.	Kommentar mit Beschreibung der Funktion/Bedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Datenbaustein > _Visu > V40ExCustomer > Member > InPotFree	DB-Variable 'InPotFree' hat keinen Kommentar.	Kommentar mit Beschreibung der Funktion/Bedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Datenbaustein > 01 [4805] DrySaltingMachine > A03 > Member > "4740_30M1".CycleCounter.CU	DB-Variable "'4740_30M1".CycleCounter.CU' hat keinen Kommentar.	Kommentar mit Beschreibung der Funktion/Bedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Datenbaustein > 01 [4805] DrySaltingMachine > A03 > Member > "4740_30M1".CycleCounter.CD	DB-Variable "'4740_30M1".CycleCounter.CD' hat keinen Kommentar.	Kommentar mit Beschreibung der Funktion/Bedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Datenbaustein > 01 [4805] DrySaltingMachine > A03 > Member > "4740_30M1".CycleCounter.R	DB-Variable "'4740_30M1".CycleCounter.R' hat keinen Kommentar.	Kommentar mit Beschreibung der Funktion/Bedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Datenbaustein > 01 [4805] DrySaltingMachine > A03 > Member > "4740_30M1".CycleCounter.LD	DB-Variable "'4740_30M1".CycleCounter.LD' hat keinen Kommentar.	Kommentar mit Beschreibung der Funktion/Bedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Datenbaustein > 01 [4805] DrySaltingMachine > A03 > Member > "4740_30M1".CycleCounter.QU	DB-Variable "'4740_30M1".CycleCounter.QU' hat keinen Kommentar.	Kommentar mit Beschreibung der Funktion/Bedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Datenbaustein > 01 [4805] DrySaltingMachine > A03 > Member > "4740_30M1".CycleCounter.QD	DB-Variable "'4740_30M1".CycleCounter.QD' hat keinen Kommentar.	Kommentar mit Beschreibung der Funktion/Bedeutung der Variable ergänzen.

5. Anhang A — Verwendete Konfiguration

Der vollständige Parameterinhalt der YAML-Konfigurationsdatei, mit der diese Prüfung durchgeführt wurde (ohne die erläuternden Kommentare der Originaldatei, bis auf einen Verweis auf die jeweilige Prüfpunkt-Nummer über jedem Prüfpunkt-Eintrag) — so bleibt dauerhaft nachvollziehbar, unter welchen Einstellungen (Schwellenwerte, Regex-Muster, Ausnahmelisten usw.) ein bestimmter Report entstanden ist, auch wenn sich die Konfigurationsdatei später ändert.

Der Dateiname wird automatisch nach folgendem Schema vergeben und im gewählten Output-Ordner gespeichert:

```
Lintreport_{Projektname}_{Datum}_{Uhrzeit}.pdf
```

8. Das Pfad-Format der Befunde

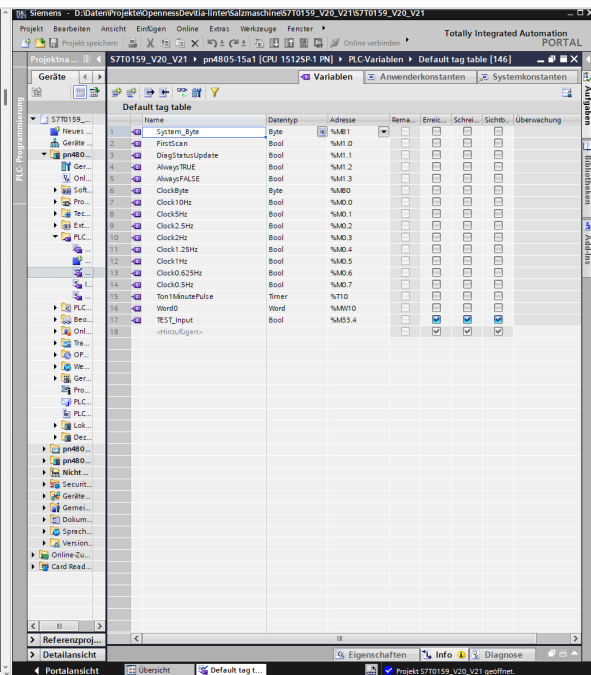
Jeder Befund — sowohl in der Oberfläche als auch im PDF-Report — zeigt den vollständigen Pfad zum betroffenen Objekt im Projekt an. Die Ebenen der Projekthierarchie werden dabei einheitlich durch > getrennt, vom Allgemeinen zum Speziellen gelesen. Beispiele:

```
PLC_1 > Programmbausteine > FB_Motor > Netzwerk 3
PLC_1 > Variablen Tabellen > Tags_Eingaenge > I_Sensor_01
PLC_1 > Datenbaustein > DB_Rezept > Member > Solltemperatur
Projekt > Eigenschaften > Autor
```

Details

Kommentare & Beschreibungen

Status	Pfad	Beschreibung	Empfehlung
Warnung	pn4805-15a1 > Variablen Tabellen > Default tag table > TEST_input	Variable "TEST_input" hat keinen Kommentar.	Kommentar mit Beschreibung der Funktionsbedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Variablen Tabellen > Outputs > NCOutput3	Variable "NCOutput3" hat keinen Kommentar.	Kommentar mit Beschreibung der Funktionsbedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Variablen Tabellen > Outputs > 000K11	Variable "000K11" hat keinen Kommentar.	Kommentar mit Beschreibung der Funktionsbedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Variablen Tabellen > Inputs > Sensor2	Variable "Sensor2" hat keinen Kommentar.	Kommentar mit Beschreibung der Funktionsbedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Variablen Tabellen > Inputs > Luckenschalter	Variable "Luckenschalter" hat keinen Kommentar.	Kommentar mit Beschreibung der Funktionsbedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Variablen Tabellen > Inputs > 0001	Variable "0001" hat keinen Kommentar.	Kommentar mit Beschreibung der Funktionsbedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Datenbaustein > _Vau > VAOCustomer > Member > InPolFree	DB-Variable "InPolFree" hat keinen Kommentar.	Kommentar mit Beschreibung der Funktionsbedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Datenbaustein > 01 14805 DrySaltingMachine > A03 > Member > "4740_30M1" CycleCounter CU	DB-Variable "4740_30M1" CycleCounter CU hat keinen Kommentar.	Kommentar mit Beschreibung der Funktionsbedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Datenbaustein > 01 14805 DrySaltingMachine > A03 > Member > "4740_30M1" CycleCounter CD	DB-Variable "4740_30M1" CycleCounter CD hat keinen Kommentar.	Kommentar mit Beschreibung der Funktionsbedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Datenbaustein > 01 14805 DrySaltingMachine > A03 > Member > "4740_30M1" CycleCounter R	DB-Variable "4740_30M1" CycleCounter R hat keinen Kommentar.	Kommentar mit Beschreibung der Funktionsbedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Datenbaustein > 01 14805 DrySaltingMachine > A03 > Member > "4740_30M1" CycleCounter LD	DB-Variable "4740_30M1" CycleCounter LD hat keinen Kommentar.	Kommentar mit Beschreibung der Funktionsbedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Datenbaustein > 01 14805 DrySaltingMachine > A03 > Member > "4740_30M1" CycleCounter OU	DB-Variable "4740_30M1" CycleCounter OU hat keinen Kommentar.	Kommentar mit Beschreibung der Funktionsbedeutung der Variable ergänzen.
Warnung	pn4805-15a1 > Datenbaustein > 01 14805 DrySaltingMachine > A03 > Member > "4740_30M1" CycleCounter QD	DB-Variable "4740_30M1" CycleCounter QD hat keinen Kommentar.	Kommentar mit Beschreibung der Funktionsbedeutung der Variable ergänzen.



Der TIA-Portal-Projektnavigator neben dem daraus abgeleiteten Pfad, wie er in der Befundtabelle und im PDF-Report erscheint

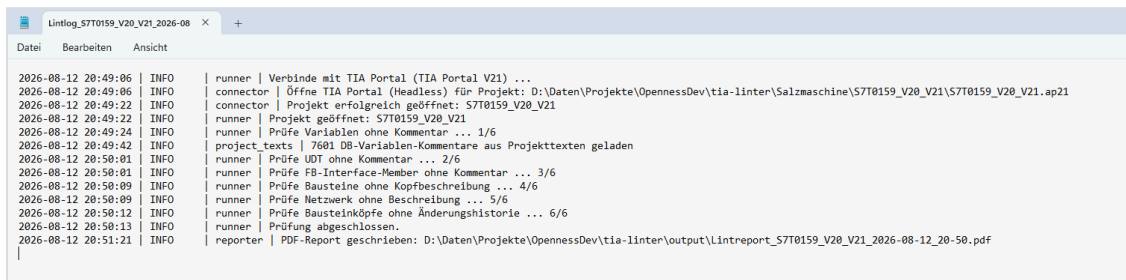
Dieses einheitliche Format erlaubt es, einen Befund im TIA-Portal-Projekt gezielt wiederzufinden, und lässt sich über die Pfad-Filterung auf der Ergebnisseite (siehe [Abschnitt 6.4](#)) auch gezielt durchsuchen.

9. Die Protokolldatei (Log-Datei)

Zusätzlich zum PDF-Report schreibt jeder Prüflauf eine **Log-Datei** in den gewählten Output-Ordner. Sie enthält denselben Ablauf, der während der Prüfung auch im Log-Fenster der Oberfläche zu sehen ist (siehe [Abschnitt 6.3](#)), jedoch dauerhaft gespeichert.

Der Dateiname wird automatisch nach folgendem Schema vergeben:

```
Lintlog_{Projektname}_{Datum}_{Uhrzeit}.log
```



Beispielhafter Ausschnitt einer Log-Datei, geöffnet in einem Texteditor, mit Zeitstempeln und Prüfpunkt-Meldungen

Die Log-Datei ist vor allem dann hilfreich, wenn ein Prüflauf unerwartet abgebrochen wurde oder länger gedauert hat als erwartet — sie zeigt in diesem Fall genau, bei welchem Schritt das Programm zuletzt war.

10. Die Prüfpunkte im Detail

Dieses Kapitel beschreibt jeden einzelnen Prüfpunkt im Detail: was genau geprüft wird, warum das relevant ist, welche Einstellungen dazu in der Konfigurationsdatei zur Verfügung stehen, und wie ein Befund behoben wird. Die Prüfpunkte sind — wie schon auf der Eingabeseite der Oberfläche (siehe [Abschnitt 6.2](#)) — nach Kategorie gruppiert.

Jeder Prüfpunkt wird einheitlich nach folgendem Schema beschrieben:

- **Was wird geprüft?** — die eigentliche Prüflogik in einfachen Worten.
- **Warum ist das wichtig?** — der praktische Nutzen, wenn der Prüfpunkt eingehalten wird.
- **Parameter** — die zugehörigen Einstellungen in der Konfigurationsdatei (Schwellenwerte, Muster, Ausnahmen), sofern vorhanden.
- **Beispiel** — ein typischer Befund, wie er in der Befundtabelle oder im PDF-Report erscheinen würde.
- **Besonderheiten** — Sonderfälle und Ausnahmen, die der Prüfpunkt automatisch berücksichtigt.
- **Empfehlung zur Behebung** — wie ein gemeldeter Verstoß typischerweise aufgelöst wird.

Dieses Kapitel wird schrittweise ausgebaut. Der aktuelle Bearbeitungsstand:

Unterkapitel	Kategorie	Prüfpunkte	Status
10.1	Kommentare & Beschreibungen	1–4	ausgearbeitet
10.2	Namenskonventionen	5–9	ausgearbeitet
10.3	Programmstruktur	10–16	ausgearbeitet
10.4	Hardware & Konfiguration	17–18c	ausgearbeitet
10.5	Projektmetadaten	19–21	ausgearbeitet
10.6	Bibliotheken & Typen	22–23	ausgearbeitet
10.7	Siemens Styleguide & Best Practices	24–33	ausgearbeitet

10.1 Kommentare & Beschreibungen (Prüfpunkte 1-4)

Diese Kategorie prüft, ob ein Projekt so dokumentiert ist, dass sich auch jemand, der es nicht selbst programmiert hat, darin zurechtfindet — eine Grundvoraussetzung für Wartung, Fehlersuche und die Übergabe eines Projekts an Kolleginnen und Kollegen oder an den Kunden.

S7T0159_V20_V21 ▶ pn4805-15a1 [CPU 1512SP-1 PN] ▶ PLC-Variablen ▶ Default tag table [146]									
Default tag table									
	Name	Datentyp	Adresse	Rema...	Erreic...	Schrei...	Sichtb...	Überwachung	Kommentar
1	System_Byte	Byte	%MB1						
2	FirstScan	Bool	%M1.0						System Bit erster Zyklus
3	DiagStatusUpdate	Bool	%M1.1						Diagnose Status Update
4	AlwaysTRUE	Bool	%M1.2						Definierte Eins
5	AlwaysFALSE	Bool	%M1.3						Definierte Null
6	ClockByte	Byte	%MB0						Takt Byte
7	Clock10Hz	Bool	%M0.0						Taktmerker 0,1 s
8	Clock5Hz	Bool	%M0.1						Taktmerker 0,2 s
9	Clock2.5Hz	Bool	%M0.2						Taktmerker 0,4 s
10	Clock2Hz	Bool	%M0.3						Taktmerker 0,5 s
11	Clock1.25Hz	Bool	%M0.4						Taktmerker 0,8 s
12	Clock1Hz	Bool	%M0.5						Taktmerker 1,0 Sec
13	Clock0.625Hz	Bool	%M0.6						Taktmerker 1,6 Sec
14	Clock0.5Hz	Bool	%M0.7						Taktmerker 2,0 Sec
15	Ton1MinutePulse	Timer	%T10						Zeit für Minutenimpuls
16	Word0	Word	%MW10						Definiertes Null-Merkerwort
17	TEST_Input	Bool	%M3.4						
18	<Hinzufügen>								

Beispiel aus TIA Portal: eine Variablentabelle mit teilweise fehlenden Kommentaren in der Spalte "Kommentar"

Prüfpunkt 1a — Variablen ohne Kommentar

Kategorie	Kommentare & Beschreibungen
Standard-Schweregrad	Warnung
Config-Schlüssel	checks.kommentare.variablen_kommentar

Was wird geprüft? Für jeden PLC-Tag (in den Variablentabellen) und jede Variable innerhalb eines Datenbausteins wird geprüft, ob ein Kommentar hinterlegt ist. Fehlt der Kommentar oder besteht er nur aus Leerzeichen, wird ein Befund erzeugt.

Warum ist das wichtig? Variablennamen allein erklären selten, wofür ein Signal tatsächlich steht (z. B. welcher physische Sensor dahintersteht oder welchen Grenzwert eine Zahl darstellt). Ohne Kommentar muss diese Information mühsam aus dem Programmcode oder aus dem Anlagenplan rekonstruiert werden.

Parameter

Parameter	Standardwert	Bedeutung
ausnahme_prefixe	[" _ "]	Variablen, deren Name mit einem dieser Präfixe beginnt, werden von der Prüfung ausgenommen. Ein einfacher Unterstrich eignet sich dafür in der Praxis oft nicht, da er häufig verwendet wird, um Variablennamen zu bilden, die nicht mit einer Ziffer beginnen dürfen/sollen — daher als Standard ein doppelter Unterstrich.
ausnahme_variables	[]	Einzelne Variablen, die unabhängig von <code>ausnahme_prefixe</code> von der

Parameter	Standardwert	Bedeutung
		Prüfung ausgenommen werden sollen — vollständiger Name, exakte Übereinstimmung (kein Präfix-/Teilstring-Abgleich). Gilt für PLC-Tags und DB-Member gleichermaßen; bei DB-Memberrn inkl. eines eventuellen Punktpfads (z. B. "Alm.Station_1").
ausnahme_udts	[]	Datentypnamen (UDTs), deren Items von dieser Prüfung ausgenommen werden sollen — wirkt wie ein manueller Zusatzeintrag zu den automatisch erkannten UDTs (siehe Prüfpunkt 1b). Gedacht in erster Linie für System-/Bibliotheksdentypen, die in TIA Portal nirgends als PLC-Datentyp sichtbar definiert sind (z. B. "TON" , "IEC_TIMER") — für solche Typen gibt es keine Möglichkeit, ihr Inneres zu prüfen. Nur das UDT-typisierte Member selbst bleibt geprüft, seine Items werden übersprungen.

Beispiel

```
PLC_1 > Variablentabellen > Tags_Eingaenge > I_Sensor_01
→ Variable 'I_Sensor_01' hat keinen Kommentar.
```

Besonderheiten

- Bei Datenbaustein-Variablen, die **Arrays** sind, genügt ein Kommentar auf dem Array selbst — die einzelnen Array-Elemente (z. B. `Rezepte[3]`) werden nicht zusätzlich einzeln bemängelt. Verschachtelte **Strukturen** (z. B. `Motor.Drehzahl`) gelten dagegen als eigenständige, einzeln zu kommentierende Variablen.
- Ist eine Datenbaustein-Variable vom Typ eines **PLC-Datentyps (UDT)**, gilt dieselbe Logik wie bei Arrays: Ein Kommentar auf der Variable selbst genügt, die einzelnen Items *innerhalb* des UDT werden hier nicht zusätzlich einzeln geprüft. Deren Kommentare (sowohl der des UDT selbst als auch die seiner Items) prüft stattdessen der separate [Prüfpunkt 1b](#). Diese UDT-Erkennung wirkt unabhängig davon, ob die Variable selbst über `ausnahme_prefixe` oder `ausnahme_variables` von der eigenen Kommentarprüfung ausgenommen ist — eine ausgenommene, aber UDT-typisierte Variable schützt ihre Items trotzdem vor Einzelprüfung. Sie wirkt ebenso unabhängig von `ausgeschlossene_ordner` / `ausgeschlossene_bausteine` : Auch ein UDT (oder FB, siehe nächster Punkt), der selbst in einem ausgeschlossenen Ordner liegt und deshalb nirgends im Projekt eigenständig geprüft wird, bleibt dem Linter als UDT/FB bekannt — nur so schützt er weiterhin die Items einer

Variable, die diesen Typ verwendet, vor Einzelprüfung. Ein ausgeschlossener Ordner steuert also nur, was selbst geprüft wird, nicht welche Datentypen der Linter kennt.

- Dieselbe Logik gilt auch für **Multi-Instanz-Aufrufe von Funktionsbausteinen (FBs)** — z. B. eine Variable vom Typ eines FB innerhalb eines Datenbausteins. Deren Interface-Member werden hier ebenfalls nicht einzeln geprüft, sondern vom separaten [Prüfpunkt 1c](#).
- **Instanz-Datenbausteine** (Instanzen eines FB) werden hier komplett übersprungen — nicht nur verschachtelte Multi-Instanzen innerhalb einer DB, sondern auch eine Instanz-DB als eigenständiger Baustein. Alle ihre Member gehören zur Interface-Definition der zugehörigen FB und werden ausschließlich von [Prüfpunkt 1c](#) geprüft — ein Kommentar “gehört” konzeptionell zur FB-Definition, nicht zur einzelnen Instanz, und soll dort genau einmal gepflegt werden statt redundant an jeder Verwendungsstelle. **Tradeoff:** TIA erlaubt es grundsätzlich, den von der FB geerbten Kommentar an einer einzelnen Instanz zu überschreiben — ein solcher instanzspezifischer, abweichender Kommentar wird durch diese Vereinfachung nicht mehr erkannt, nur der Kommentar an der FB-Definition zählt für die Prüfung.

Empfehlung zur Behebung Kommentar mit Beschreibung der Funktion bzw. Bedeutung der Variable ergänzen — z. B. welcher Anlagenteil betroffen ist oder was ein Grenzwert bedeutet.

Prüfpunkt 1b — UDT ohne Kommentar

Kategorie	Kommentare & Beschreibungen
Standard-Schweregrad	Warnung
Config-Schlüssel	<code>checks.kommentare.udt_kommentar</code>

Was wird geprüft? Für jeden PLC-Datentyp (UDT) im Projekt wird geprüft, ob er selbst einen Kommentar hat, und zusätzlich für jedes seiner Items (Interface-Member), ob dieses einen Kommentar hat. Fehlt einer der beiden, wird je ein eigener Befund erzeugt.

Warum ist das wichtig? Prüfpunkt 1a prüft Items *innerhalb* eines UDT-typisierten Datenbaustein-Members bewusst nicht mehr einzeln (siehe dortige Besonderheiten) — ein Kommentar auf der Variable selbst genügt dort, analog zu Array-Elementen. Ohne diesen eigenen Prüfpunkt blieben die Items eines UDT damit vollständig ungeprüft, egal wie oft und an wie vielen Stellen der UDT im Projekt verwendet wird. Da ein UDT häufig an vielen Stellen eingesetzt wird, lohnt sich die Kommentierung an der UDT-Definition selbst ohnehin mehr als an jeder einzelnen Verwendungsstelle.

Parameter

Parameter	Standardwert	Bedeutung
<code>ausnahme_prefixe</code>	<code>["__"]</code>	UDTs bzw. Items, deren Name mit einem dieser Präfixe beginnt, werden von der Prüfung ausgenommen.

Beispiel

```
PLC_1 > PLC-Datentypen > U_Motor  
→ PLC-Datentyp 'U_Motor' hat keinen Kommentar.
```

```
PLC_1 > PLC-Datentypen > U_Motor > Member > Drehzahl  
→ UDT-Variable 'Drehzahl' hat keinen Kommentar.
```

Besonderheiten

- War in der ursprünglichen Liste der 33 Prüfpunkte kein eigener Punkt — ergänzt Prüfpunkt 1a um eine Lücke, die erst durch dessen eigene UDT-Sonderbehandlung entstanden ist (siehe dort).
- Ist ein Item selbst wieder vom Typ eines (anderen oder desselben) UDT, wird ab dort **nicht** weiter in die Tiefe geprüft — dieses verschachtelte UDT wird eigenständig geprüft, sobald die Prüfung bei ihm ankommt. Jeder UDT wird also genau einmal an seiner Definition geprüft, unabhängig davon, wie oft und wo er im Projekt verwendet wird.
- Wie bei Prüfpunkt 1a genügt bei UDT-Items, die Arrays sind, ein Kommentar auf dem Array selbst.

Empfehlung zur Behebung Kommentar auf dem PLC-Datentyp bzw. dem betroffenen Item ergänzen — da ein UDT oft mehrfach verwendet wird, wirkt sich ein einmal ergänzter Kommentar an der Definition überall dort aus, wo der Typ eingesetzt wird.

Prüfpunkt 1c — FB-Interface-Member ohne Kommentar

Kategorie	Kommentare & Beschreibungen
Standard-Schweregrad	Warnung
Config-Schlüssel	<code>checks.kommentare.fb_member_kommentar</code>

Was wird geprüft? Für jeden Funktionsbaustein (FB) im Projekt wird für jedes seiner Interface-Member (Input, Output, InOut, Static) geprüft, ob ein Kommentar hinterlegt ist. Geprüft wird ausschließlich die FB-Definition selbst, nicht einzelne Verwendungsstellen — weder als Multi-Instanz innerhalb einer DB noch als eigenständige Instanz-DB. Dieser Prüfpunkt ist damit die **alleinige Quelle** für Kommentar-Befunde zu FB-Interface-Membern im gesamten Projekt. Der Kopfkomentar des FB als Ganzes wird hier **nicht** geprüft — das übernimmt bereits [Prüfpunkt 2](#).

Warum ist das wichtig? Prüfpunkt 1a prüft Items *innerhalb* einer Multi-Instanz-FB-Variable sowie sämtliche Member von Instanz-DBs bewusst nicht mehr einzeln (siehe dortige Besonderheiten) — ein Kommentar auf einem FB-Interface-Member “gehört” konzeptionell zur FB-Definition, nicht zur einzelnen Verwendungsstelle, und ein FB wird oft mehrfach instanziiert. Ohne diesen eigenen Prüfpunkt blieben die Interface-Member eines FB damit vollständig ungeprüft, egal wie oft und an wie vielen Stellen der FB verwendet wird — analog zur Begründung bei [Prüfpunkt 1b](#) für UDTs. Die Alternative (Kommentare an jeder einzelnen Instanz-DB prüfen) hätte bei mehrfach instanziierten FBs zu redundanten Befunden für dasselbe Grundproblem geführt.

Parameter

Parameter	Standardwert	Bedeutung
ausnahme_prefixe	["___"]	Interface-Member, deren Name mit einem dieser Präfixe beginnt, werden von der Prüfung ausgenommen.

Beispiel

```
PLC_1 > Programmbausteine > FB_Motor > Member > Drehzahl
→ FB-Variable 'Drehzahl' hat keinen Kommentar.
```

Besonderheiten

- War in der ursprünglichen Liste der 33 Prüfpunkte kein eigener Punkt — ergänzt Prüfpunkt 1a um eine Lücke, die erst durch dessen eigene Multi-Instanz-Sonderbehandlung entstanden ist (siehe dort).
- Technischer Sonderfall: Anders als bei Datenbausteinen und PLC-Datentypen liefert die Openness-API für Funktionsbausteine über die direkte Objektnavigation keine Interface-Member — dieser Prüfpunkt liest sie stattdessen aus demselben XML-Export, den auch die Netzwerk-bezogenen Prüfpunkte (z. B. Prüfpunkt 3) verwenden. Praktisch wirkt sich das nicht aus, außer dass verschachtelte Struct-/UDT-/Multi-Instanz-Felder innerhalb eines Interface-Members hier nicht gesondert aufgelöst werden (das jeweils zugehörige UDT bzw. der FB wird wie gewohnt eigenständig geprüft).
- Temp-Variablen werden bewusst nicht geprüft — sie persistieren zwischen Aufrufen nicht und werden in der Praxis nicht einzeln kommentiert.
- Deckt auch Instanz-DBs ab, die als eigenständige Bausteine im Projekt angelegt sind (nicht nur verschachtelte Multi-Instanzen innerhalb einer DB) — siehe die Besonderheiten und den dortigen Tradeoff bei [Prüfpunkt 1a](#).

Empfehlung zur Behebung Kommentar auf dem betroffenen FB-Interface-Member ergänzen — da ein FB oft mehrfach als Multi-Instanz verwendet wird, wirkt sich ein einmal ergänzter Kommentar an der Definition überall dort aus, wo der FB eingesetzt wird.

Prüfpunkt 2 — Bausteine ohne Kopfbeschreibung

Kategorie	Kommentare & Beschreibungen
Standard-Schweregrad	Warnung
Config-Schlüssel	checks.kommentare.baustein_beschreibung

Was wird geprüft? Für jeden Baustein (standardmäßig FB, FC, OB — siehe `check_db` für Datenbausteine) wird die Kopfbeschreibung geprüft: Sowohl das `Title` - als auch das `Comment` -Attribut der Bausteineigenschaften werden gelesen, das längere der beiden zählt. Fehlt eine ausreichend lange Kopfbeschreibung in beiden Feldern, wird ein Befund erzeugt.

Warum ist das wichtig? Die Kopfbeschreibung ist meist die erste Stelle, an der sich jemand über Zweck und Funktionsweise eines Bausteins informiert, bevor er sich durch die einzelnen Netzwerke arbeitet. Eine zu kurze oder fehlende Beschreibung (z. B. nur ein Wort) erfüllt diesen Zweck nicht.

Parameter

Parameter	Standardwert	Bedeutung
<code>min_laenge</code>	<code>20</code>	Mindestanzahl an Zeichen, die die Kopfbeschreibung haben muss, um als aussagekräftig zu gelten.
<code>check_db</code>	<code>false</code>	Ob auch Datenbausteine (Global-, Instanz- und Array-DB) auf eine Kopfbeschreibung geprüft werden sollen.

Beispiel

```
PLC_1 > Programmausteine > FB_Motor
→ Baustein 'FB_Motor' hat keine oder zu kurze Kopfbeschreibung
(mind. 20 Zeichen erwartet).
```

Besonderheiten

- Eine Kopfbeschreibung ist bei Datenbausteinen in der Praxis unüblich — anders als bei FB/FC/OB, wo sie den Zweck des Codes beschreibt, hat ein DB meist nur Daten ohne eigene “Funktion” zu beschreiben. Deshalb sind DBs standardmäßig (`check_db: false`) von dieser Prüfung ausgenommen; FB/FC/OB werden davon unabhängig immer geprüft. Mit `check_db: true` lässt sich die ursprüngliche Prüfung aller Bausteintypen wiederherstellen.
- `Title` und `Comment` sind zwei unabhängige, jeweils mehrsprachige Felder, die im Bausteinkopf des TIA-Editors beide sichtbar sind, aber getrennt gepflegt werden — in der Praxis wird die eigentliche Kurzbeschreibung häufig nur in eines der beiden Felder eingetragen (meist `Title`). Dieser Prüfpunkt zählt daher das jeweils längere der beiden Felder, statt nur `Comment` zu betrachten.

Empfehlung zur Behebung Kopfbeschreibung mit Zweck und Funktionsweise des Bausteins ergänzen — was steuert oder überwacht der Baustein, und was sollte jemand wissen, bevor er ihn ändert?

Prüfpunkt 3 — Netzwerk ohne Beschreibung

Kategorie	Kommentare & Beschreibungen
Standard-Schweregrad	Warnung
Config-Schlüssel	<code>checks.kommentare.netzwerk_beschreibung</code>

Was wird geprüft? Für jedes Netzwerk innerhalb eines Bausteins wird geprüft, ob ein Netzwerktitel vergeben ist. Fehlt der Titel, wird das gemeldet; ist er länger als die konfigurierte Obergrenze, wird das ebenfalls gemeldet — ein zu langer Titel ist auf Dauer meist kein prägnanter Titel mehr, sondern ein Fließtext.

Warum ist das wichtig? Ein kurzer, aussagekräftiger Netzwerktitel erlaubt es, einen Baustein zu überfliegen, ohne jedes einzelne Netzwerk im Detail lesen zu müssen — das beschleunigt sowohl die Einarbeitung als

auch die Fehlersuche im laufenden Betrieb erheblich.

Parameter

Parameter	Standardwert	Bedeutung
max_zeichen	80	Maximale Länge des Netzwerktitels in Zeichen.

Beispiel

```
PLC_1 > Programmbausteine > FB_Motor > Netzwerk 3  
→ Netzwerk hat keinen Titel.
```

Besonderheiten

- Bausteine, die vollständig in **SCL oder AWL/STL** programmiert sind, besitzen keine einzelnen Netzwerke mit Titeln (das Konzept “Netzwerk” gibt es nur in den grafischen Sprachen KOP/FUP/GRAPH) — sie werden von diesem Prüfpunkt automatisch ausgenommen.
- Der Netzwerktitel ist wie viele andere Text-Attribute in TIA mehrsprachig — gelesen wird gezielt der Text für die Referenzsprache des Projekts, nicht ein sprachunabhängiger Rohwert.
- Der Ausschluss vollständig SCL-/STL-programmierter Bausteine (siehe oben) war durch einen Vergleichsfehler bislang wirkungslos (der Baustein-Grundsprachen-Vergleich prüfte gegen ein .NET-Enum-Objekt statt gegen einen Text, siehe docs/Handbuch.md Anhang C, Version 0.41) — dadurch wurden reine SCL-Bausteine bislang fälschlich mitgeprüft. Live an Salzmaschine verifiziert: 194 → 46 Befunde nach dem Fix.

Empfehlung zur Behebung Kurzen, prägnanten Netzwerktitel ergänzen, der zusammenfasst, was das Netzwerk tut (z. B. “Freigabe Antrieb prüfen” statt keines Titels oder eines mehrzeiligen Fließtexts).

Prüfpunkt 4 — Bausteinköpfe ohne Änderungshistorie

Kategorie	Kommentare & Beschreibungen
Standard-Schweregrad	Warnung
Config-Schlüssel	checks.kommentare.aenderungshistorie

Was wird geprüft? Für jeden Baustein (standardmäßig FB, FC, OB und Global-/Array-DBs — siehe check_idb für Instanz-Datenbausteine) wird geprüft, ob im Bausteinkopf **sowohl** ein Autor **als auch** eine Versionsangabe hinterlegt sind. Ein Befund entsteht nur, wenn **beide** Angaben vollständig fehlen — ist mindestens eine der beiden Angaben vorhanden, gilt der Prüfpunkt für diesen Baustein als erfüllt.

Warum ist das wichtig? Autor und Version im Bausteinkopf machen nachvollziehbar, wer einen Baustein zuletzt bearbeitet hat und ob es sich um den aktuellen Stand handelt. Das ist besonders bei Projekten hilfreich, an denen mehrere Personen über einen längeren Zeitraum arbeiten.

Parameter

Parameter	Standardwert	Bedeutung
<code>check_idb</code>	<code>false</code>	Ob auch Instanz-Datenbausteine auf Autor/Version im Bausteinkopf geprüft werden sollen.

Beispiel

```
PLC_1 > Programmbausteine > FB_Motor
→ Baustein 'FB_Motor' hat weder Autor noch Version im
  Bausteinkopf hinterlegt.
```

Besonderheiten

- Eine nie gesetzte Version liest die Openness-API intern als `"0.0.0.0"` — dieser technische Standardwert wird hier wie eine leere Version behandelt, nicht wie eine echte Versionsangabe.
- Instanz-Datenbausteine pflegen in der Praxis keine eigene Änderungshistorie — Autor/Version “gehören” konzeptionell zur FB-Definition, nicht zur einzelnen Instanz. Deshalb sind Instanz-DBs standardmäßig (`check_idb: false`) von dieser Prüfung ausgenommen; Global-/Array-DBs sowie FB/FC/OB werden davon unabhängig immer geprüft. Mit `check_idb: true` lässt sich die Prüfung auch für Instanz-DBs wiederherstellen. Anders als bei Prüfpunkt 2s `check_db` betrifft dieser Parameter also gezielt nur Instanz-DBs, nicht alle Datenbausteine.

Empfehlung zur Behebung Änderungshistorie im Bausteinkopf pflegen — mindestens Autor und Versionsnummer, idealerweise ergänzt um Datum und eine Kurzbeschreibung der Änderung, gemäß dem Siemens Standardisierungsleitfaden.

10.2 Namenskonventionen (Prüfpunkte 5-9)

Diese Kategorie prüft, ob sich Bausteine, Variablen und Konstanten anhand ihres Namens auf den ersten Blick richtig einordnen lassen — ohne dass man erst nachschauen muss, ob z. B. “Sensor_Endlage” ein Eingang oder ein Ausgang ist, oder ob “Rezept_01” ein Datenbaustein oder etwas anderes ist. Einheitliche Namen erleichtern außerdem die Suche im Projekt und das Wiedererkennen von Mustern über mehrere Projekte hinweg.

Alle Prüfpunkte dieser Kategorie arbeiten mit **regulären Ausdrücken** (Regex): einem in der Konfigurationsdatei hinterlegten Muster, gegen das jeder betroffene Name geprüft wird. Wer mit regulären Ausdrücken nicht vertraut ist, kann sich an den mitgelieferten Standardmustern orientieren und sie bei Bedarf mit Unterstützung anpassen.

S7T0159_V20_V21 ▶ pn4805-15a1 [CPU 1512SP-1 PN] ▶ PLC-Variablen ▶ Inputs [23]									
Inputs									
	Name	Datentyp	Adresse	Rema...	Erreic...	Schrei...	Sichtb...	Überwachung	Kommentar
1	I4805_6531	Bool	%I0.0						1 = Betrieb 2 = Unterbrechung
2	Sensor1	Bool	%I0.1						1 = Betrieb 2 = Unterbrechung
3	I4805_6510	Bool	%I3.3						Druckwächter Zuleitung
4	I4805CustomerReady	Bool	%I2.0						Kunde bereit
5	I4805_21A1	Bool	%I0.5						Anlage Start Not-Halt
6	I4805_15A15	Bool	%I0.7						Router ok
7	I4805_27Q11	Bool	%I1.0						Motorschutzscharter Gebläse Salzförderu...
8	I4805_27Q21	Bool	%I1.1						Motorschutzscharter Gebläse Fluidisierung
9	I4805_27Q31	Bool	%I1.2						Motorschutzscharter Gebläse Abblaseinri...
10	I4805_27Q41	Bool	%I1.3						Motorschutzscharter Gebläse Düsen
11	I4805_30Q1	Bool	%I1.4						Motorschutzscharter Antrieb Salzrolle
12	I4805_33Q1	Bool	%I1.5						Motorschutzscharter Antriebe Transportb...
13	I4805_46Q1	Bool	%I1.6						Motorschutzscharter Heizung Fluidisierun...
14	I4805_46A3	Bool	%I1.7						Stromüberwachung Heizung Fluidisierung
15	I4805_33X1	Bool	%I2.4						Sensor Horde vorhanden
16	I4805_46A11	Int	%IW752						Sonde Temperatur Luft nach Heizung 4-2...
17	I4805_10F514	Bool	%I0.4						Sicherungsverteilung Versorgung 24VDC ...
18	I4805_24B1	Bool	%I2.3						Sonde Füllstand Minimum Salzbehälter u...
19	I4805_24B11	Bool	%I2.5						Sonde Füllstand Minimum Salzbehälter S...
20	I1234Test	Bool	%I2.6						Test var
21	Sensor2	Bool	%I0.2						
22	Lueckenschalter	Bool	%I0.3						
23	öööl	Bool	%M6.6						
24	<Hinzufügen>								

Beispiel aus TIA Portal: PLC-Tags mit uneinheitlicher Namensgebung, teils mit und teils ohne Präfix für Eingänge und Ausgänge

Prüfpunkt 5 — DB-Namensformat (Global-/Array-DB und Instanz-DB)

Kategorie	Namenskonventionen
Standard-Schweregrad	Fehler
Config-Schlüssel	<code>checks.namenskonventionen.db_format_global</code> und <code>checks.namenskonventionen.db_format_instance</code>

Dieser eine Prüfpunkt ist in der Oberfläche als **zwei getrennte Kontrollkästchen** abgebildet — eines für Global-/Array-Datenbausteine, eines für Instanz-Datenbausteine — damit sich beide Arten unabhängig voneinander aktivieren und mit einem eigenen Muster konfigurieren lassen. Grund: Der Name eines Instanz-DBs wird meist automatisch aus dem zugehörigen FB-Aufruf abgeleitet und folgt in der Praxis oft einer komplett anderen Namenskonvention als frei benannte Global- oder Array-Datenbausteine.

Was wird geprüft? Für jeden Datenbaustein wird der Name gegen das zur jeweiligen Art passende Muster geprüft. Entspricht der Name nicht dem Muster, wird ein Befund erzeugt.

Warum ist das wichtig? Ein einheitliches Namenspräfix für Datenbausteine macht auf den ersten Blick sichtbar, dass es sich um reine Datenablage handelt — im Unterschied zu Bausteinen mit Programmlogik (FB/FC). Weil Instanz-DBs technisch anders entstehen als Global-/Array-DBs, lohnt sich dafür meist ein eigenes Namensschema statt eines gemeinsamen.

Parameter

Parameter	Standardwert	Bedeutung
<code>regex</code> (Global-/Array-DB)	<code>^DB_[A-Za-z]</code>	Muster, dem Global- und Array-Datenbausteine entsprechen müssen.
<code>regex</code> (Instanz-DB)	<code>^DB_[A-Za-z]</code>	Muster, dem Instanz-Datenbausteine entsprechen müssen — unabhängig vom obigen Muster konfigurierbar.

Beispiel

```
PLC_1 > Datenbaustein > Rezept_01
→ DB-Name 'Rezept_01' entspricht nicht dem Muster '^DB_[A-Za-z]'.
```

Besonderheiten

- Das Standardmuster prüft nur den **Anfang** des Namens (es endet nicht mit einem Endanker `$`) — es genügt also, dass der Name mit `DB_` und einem Buchstaben beginnt, der Rest des Namens ist frei wählbar. Wer eine strengere, vollständige Prüfung möchte, kann in der Konfiguration ein eigenes Muster mit `$` am Ende hinterlegen.
- Array-Datenbausteine** werden zusammen mit Global-DBs geprüft (über `db_format_global`), nicht zusammen mit Instanz-DBs — beide werden vom Anwender frei benannt, im Gegensatz zum meist automatisch abgeleiteten Namen eines Instanz-DBs.

Empfehlung zur Behebung Datenbaustein gemäß der für seine Art (Global-/Array-DB bzw. Instanz-DB) geltenden Namenskonvention umbenennen.

Prüfpunkt 6 — PLC-Tag-Namenskonvention (Eingänge/Ausgänge)

Kategorie	Namenskonventionen
Standard-Schweregrad	Fehler
Config-Schlüssel	<code>checks.namenskonventionen.plc_tag_eingaenge</code> und <code>checks.namenskonventionen.plc_tag_ausgaenge</code>

Dieser eine Prüfpunkt ist in der Oberfläche als **zwei getrennte Kontrollkästchen** abgebildet — eines für Eingänge, eines für Ausgänge — damit sich beide Richtungen unabhängig voneinander aktivieren und mit einem eigenen Muster konfigurieren lassen.

Was wird geprüft? Für jeden PLC-Tag, der über eine feste Peripherie-Adresse verfügt (z. B. `%I0.0` für einen Eingang oder `%Q4.1` für einen Ausgang), wird der Tag-Name gegen das zur jeweiligen Richtung passende Muster geprüft. Ob ein Tag als Eingang oder Ausgang gilt, wird dabei **anhand seiner Adresse** bestimmt — nicht anhand des Namens selbst.

Warum ist das wichtig? Ein Blick auf den Namen genügt, um zu erkennen, ob ein Signal von der Anlage kommt oder zu ihr geht — das beschleunigt Fehlersuche und Verdrahtungsabgleich erheblich.

Parameter

Parameter	Standardwert	Bedeutung
regex (Eingänge)	<code>^I_</code>	Muster, dem Eingangs-Tags entsprechen müssen.
regex (Ausgänge)	<code>^Q_</code>	Muster, dem Ausgangs-Tags entsprechen müssen.

Beispiel

```
PLC_1 > Variablentabellen > Tags_Eingaenge > Sensor_Endlage  
→ Tag 'Sensor_Endlage' entspricht nicht dem Muster '^I_'.
```

Besonderheiten

- Tags **ohne feste Peripherie-Adresse** (z. B. rein interne Merker ohne Bezug zu einem Ein- oder Ausgang) werden von diesem Prüfpunkt automatisch nicht geprüft, da sie weder eindeutig Eingang noch Ausgang sind.

Empfehlung zur Behebung Tag gemäß Namenskonvention für Eingänge bzw. Ausgänge umbenennen (siehe Konfiguration).

Prüfpunkt 7 — Bausteinname-Konvention (FB/FC)

Kategorie	Namenskonventionen
Standard-Schweregrad	Fehler
Config-Schlüssel	<code>checks.namenskonventionen.fb_prefix</code> und <code>checks.namenskonventionen.fc_prefix</code>

Auch dieser Prüfpunkt ist als **zwei getrennte Kontrollkästchen** abgebildet — eines für Funktionsbausteine (FB), eines für Funktionen (FC).

Was wird geprüft? Für jeden Funktionsbaustein bzw. jede Funktion wird der Bausteinname gegen das jeweils konfigurierte Muster geprüft. Organisationsbausteine (OB) sind von diesem Prüfpunkt nicht betroffen; Datenbausteine werden bereits über [Prüfpunkt 5](#) geprüft.

Warum ist das wichtig? Ein einheitliches Präfix macht auf den ersten Blick erkennbar, ob ein Baustein einen eigenen Instanz-Datenbaustein besitzt (FB) oder zustandslos arbeitet (FC) — ein relevanter Unterschied beim Verständnis des Programmablaufs.

Parameter

Parameter	Standardwert	Bedeutung
regex (FB)	<code>^FB_\S*\$</code>	Muster, dem Funktionsbaustein-Namen entsprechen müssen.
regex (FC)	<code>^FC_\S*\$</code>	Muster, dem Funktions-Namen entsprechen müssen.

Beispiel

```
PLC_1 > Programmbausteine > Motor_Steuerung
→ Bausteinname 'Motor_Steuerung' entspricht nicht dem Muster '^FB_\S*$'.
```

Besonderheiten

- Anders als bei Prüfpunkt 5 endet das Standardmuster hier mit einem Endanker (`$`) — geprüft wird also der **gesamte** Bausteinname, nicht nur der Anfang.

Empfehlung zur Behebung Funktionsbaustein bzw. Funktion gemäß Namenskonvention umbenennen (siehe Konfiguration).

Prüfpunkt 8 — Konstanten-Namensformat

Kategorie	Namenskonventionen
Standard-Schweregrad	Warnung
Config-Schlüssel	<code>checks.namenskonventionen.konstanten_format</code>

Was wird geprüft? Für globale Konstanten innerhalb einer Variablentabelle wird der Name gegen das konfigurierte Muster geprüft — im Standard: nur Großbuchstaben, Ziffern und Unterstriche, beginnend mit einem Buchstaben.

Warum ist das wichtig? Durchgängig großgeschriebene Konstanten sind auf den ersten Blick von normalen (veränderlichen) Variablen zu unterscheiden — eine in der Programmierung weit verbreitete Konvention.

Parameter

Parameter	Standardwert	Bedeutung
regex	<code>^[A-Z][A-Z0-9_]*\$</code>	Muster, dem der Konstantenname entsprechen muss.

Beispiel

```
PLC_1 > Variablentabellen > Tags_Allgemein > MaxDrehzahl
→ Konstante 'MaxDrehzahl' entspricht nicht dem Muster '^[A-Z][A-Z0-9_]*$'.
```

Besonderheiten

- Enthält eine Variablen-tabelle keine globalen Konstanten, wird sie für diesen Prüfpunkt übersprungen, ohne einen Befund zu erzeugen.

Empfehlung zur Behebung Konstantenname gemäß Namenskonvention anpassen (siehe Konfiguration).

Prüfpunkt 9 — Testvariablen vorhanden

Kategorie	Namenskonventionen
Standard-Schweregrad	Warnung
Config-Schlüssel	checks.namenskonventionen.testvariablen

Was wird geprüft? Anders als die übrigen Prüfpunkte dieser Kategorie prüft dieser Punkt nicht, ob ein Name einem Muster entspricht, sondern ob im Projekt überhaupt noch PLC-Tags vorhanden sind, deren Name mit einem der konfigurierten “Test-Präfixe” beginnt.

Warum ist das wichtig? Test- und Debug-Variablen werden während der Entwicklung oft temporär angelegt und danach vergessen. Sie blähen das Projekt unnötig auf und bergen das Risiko, versehentlich mit produktiven Variablen verwechselt zu werden.

Parameter

Parameter	Standardwert	Bedeutung
prefixe	["TEST_", "DEBUG_", "_TEMP"]	Liste von Namenspräfixen, die als Testvariable gelten.

Beispiel

```
PLC_1 > Variablentabellen > Tags_Allgemein > TEST_Handbetrieb
→ Testvariable 'TEST_Handbetrieb' ist noch im Projekt vorhanden.
```

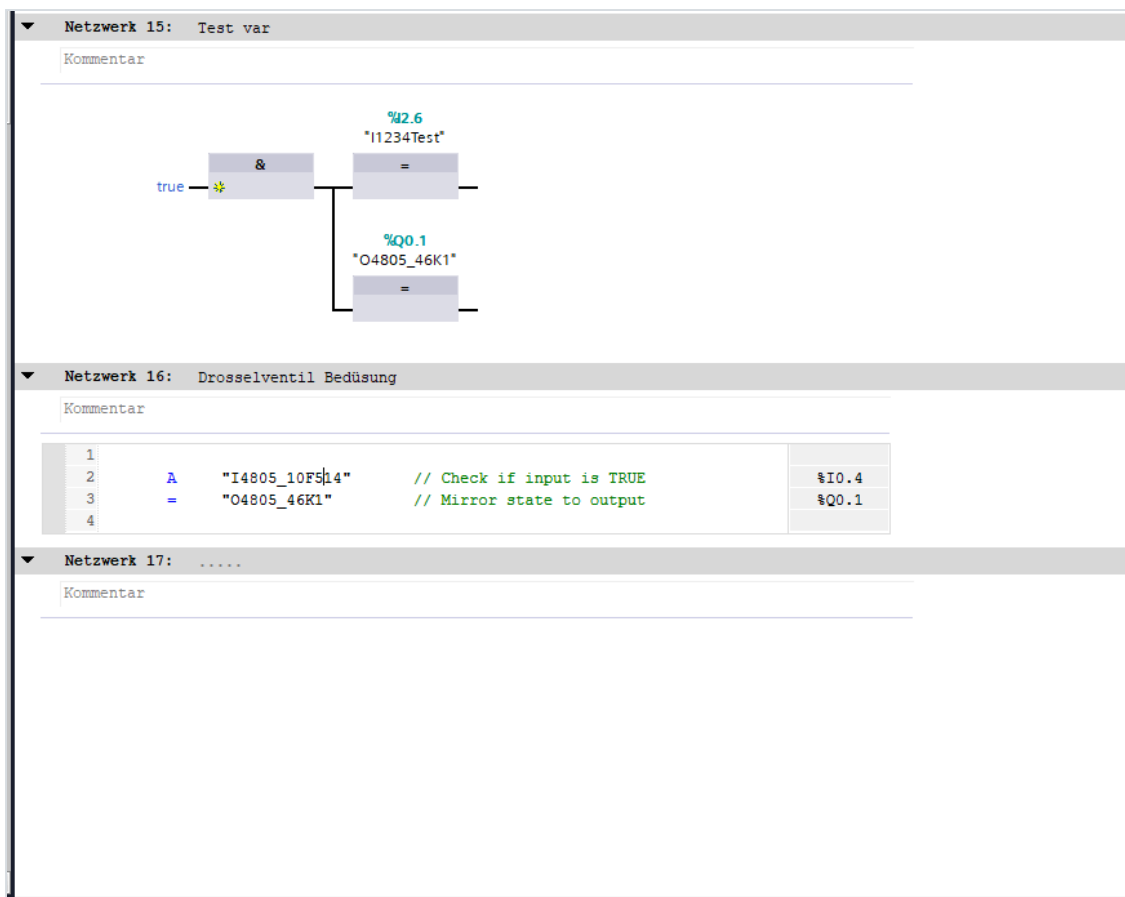
Besonderheiten

- Ist die Präfixliste in der Konfiguration leer, liefert dieser Prüfpunkt grundsätzlich keine Befunde.

Empfehlung zur Behebung Prüfen, ob die Testvariable noch benötigt wird — andernfalls entfernen.

10.3 Programmstruktur (Prüfpunkte 10-16)

Diese Kategorie prüft den eigentlichen Programmaufbau: Gibt es toten Code (leere Netzwerke, unbenutzte Variablen und Bausteine)? Werden Ein- und Ausgänge so verwendet, wie man es von ihnen erwartet? Ist die Struktur einheitlich und überschaubar (Sprache, Netzwerkgröße)? Diese Prüfpunkte zielen direkt auf die **Lesbarkeit und Verlässlichkeit der Programmlogik** ab — nicht nur auf äußere Form wie Kommentare oder Namen.



Beispiel aus TIA Portal: ein leeres Netzwerk sowie ein Ausgang, der an mehreren Stellen im Programm beschrieben wird

Prüfpunkt 10 — Leere Netzwerke

Kategorie	Programmstruktur
Standard-Schweregrad	Warnung
Config-Schlüssel	checks.programmstruktur.leere_netzwerke

Was wird geprüft? Für jedes Netzwerk eines grafisch programmierten Bausteins wird geprüft, ob es überhaupt Programmelemente enthält (Kontakte, Spulen, Bausteinaufrufe usw.). Ist ein Netzwerk vollständig leer, wird ein Befund erzeugt.

Warum ist das wichtig? Leere Netzwerke sind meist Überbleibsel aus der Entwicklung — etwa Platzhalter oder Reste gelöschter Logik. Sie tragen nichts zur Funktion bei, verlängern den Baustein unnötig und sorgen bei der Fehlersuche für die Frage: “Fehlt hier etwas, oder ist das Netzwerk absichtlich leer?”

Parameter

Parameter	Standardwert	Bedeutung
ausnahme_titel_regex	"" (deaktiviert)	Passt der Netzwerktitel eines sonst leeren Netzwerks auf dieses Muster, wird es nicht gemeldet.

Beispiel

```
PLC_1 > Programmbausteine > FB_Motor > Netzwerk 5  
→ Netzwerk ist leer (keine Programmelemente).
```

Besonderheiten

- Wie bei [Prüfpunkt 3](#) gibt es das Konzept “Netzwerk” nur in den grafischen Sprachen (KOP/FUP/GRAPH) — Bausteine in SCL oder AWL/STL werden automatisch nicht geprüft.
- TIA erlaubt gemischte Programmiersprachen innerhalb eines Bausteins (siehe [Prüfpunkt 15](#)) — ein einzelnes Netzwerk kann z. B. SCL sein, obwohl der Baustein insgesamt als FBD geführt wird. Ein solches Netzwerk wird ebenfalls automatisch nicht geprüft, unabhängig von der Sprache des Bausteins insgesamt.
- Ein Netzwerk, das ausschließlich einen einzigen Bausteinaufruf enthält (keinen Kontakt, keine Spule), zählt als nicht-leer — ein Bausteinaufruf ist ein eigenständiges Programmelement.
- Ein leeres Netzwerk wird in der Praxis gelegentlich absichtlich verwendet, um mit seinem Netzwerktitel eine Art Kapitelüberschrift innerhalb eines Bausteins zu setzen (z. B. "##### Kapitel-Titel #####"). Passt der Titel eines sonst leeren Netzwerks auf `ausnahme_titel_regex` (`re.match`, wie bei allen anderen Regex-Parametern dieses Projekts), wird es trotz fehlender Programmelemente nicht gemeldet. Ein Netzwerk mit echten Programmelementen ist davon unabhängig ohnehin nie betroffen.

Empfehlung zur Behebung Leeres Netzwerk mit Logik befüllen oder entfernen — oder, falls absichtlich als Kapitelüberschrift verwendet, den Titel auf `ausnahme_titel_regex` abstimmen.

Prüfpunkt 11a — Unbenutzte Variablen (Dead Code)

Kategorie	Programmstruktur
Standard-Schweregrad	Warnung
Config-Schlüssel	<code>checks.programmstruktur.unbenutzte_variablen</code>

Was wird geprüft? Für jeden PLC-Tag, jede Variable innerhalb eines Global-/Array-Datenbausteins und jedes Interface-Member (Input/Output/InOut/Static/Temp) eines FB/FC/OB wird geprüft, ob eine Verwendung vorliegt. Für FB/FC/OB zählt dabei **ausschließlich die Verwendung innerhalb des Bausteins selbst** — ein Zugriff von außen (z. B. eine andere Stelle im Programm, die direkt auf `"Instanz".Member` zugreift) macht ein Member hier nicht “benutzt”; solche externen Zugriffe sind ohnehin unerwünscht und werden separat von [Prüfpunkt 25](#) gemeldet.

Warum ist das wichtig? Unbenutzte Variablen sind “toter Code”: Sie belegen Speicher und Adressraum, tauchen unnötig in Kreuzreferenzen und Exporten auf und können bei einer späteren Aufräumaktion fälschlich für “wird noch gebraucht” gehalten werden.

Parameter

Parameter	Standard	Bedeutung
unterelemente_pruefen	false	Bei <code>false</code> gilt ein UDT-/Strukturiertes Member (in DBs wie in FB/FC/OB) als Ganzes als verwendet, sobald irgendein Unterfeld irgendwo referenziert wird — einzelne, weiterhin ungenutzte Unterfelder werden dann nicht einzeln gemeldet. Ist die Variable dagegen komplett unbenutzt (kein einziges Unterfeld irgendwo referenziert), werden ihre Unterfelder trotzdem einzeln gemeldet, unabhängig von diesem Parameter. Bei <code>true</code> werden immer alle Unterfelder einzeln geprüft, ohne diese Pauschal-Ausnahme. Skalare Variablen sind von diesem Parameter nicht betroffen (für sie ist “als Ganzes” ohnehin gleichbedeutend mit “als einziges Blatt”).

Beispiel

```
PLC_1 > Variablentabellen > Tags_Allgemein > Merker_Testlauf
→ Variable 'Merker_Testlauf' wird im gesamten Programm nicht verwendet.
```

```
PLC_1 > Programmbausteine > OrgPrg > Member > test
→ Variable 'test' wird im Baustein 'OrgPrg' nirgends verwendet.
```

Besonderheiten

- PLC-Tags, Global-/Array-DB-Variablen und FB-/FC-/OB-Interface-Member werden technisch auf drei unterschiedlichen Wegen geprüft. Für die Nutzung der Oberfläche macht das keinen Unterschied — das Ergebnis ist in allen Fällen ein Befund mit derselben Aussage: “wird nirgends verwendet”.
- **Instanz-Datenbausteine werden hier bewusst nicht mehr eigenständig geprüft** (frühere Handbuch-Versionen taten dies noch): Eine Instanz-DB ist reiner Speicher ohne eigene Logik — ob ein Member “benutzt” ist, entscheidet sich im Code des zugehörigen FB, nicht in der DB. Die Interface-Member werden deshalb direkt an der FB-Definition geprüft, unabhängig davon, wie viele Instanzen davon im Projekt existieren.
- Global- und Array-Datenbausteine haben dagegen keinen “Besitzer-Baustein” — dort zählt weiterhin jede Verwendung im gesamten Projekt (nicht nur intern), analog zum bisherigen Verhalten. Die von Openness gelieferten Kreuzreferenz-Ergebnisse liegen als Baum vor: verschachtelte Struct-/UDT-typisierte Member (z. B. `DiagCpu`) enthalten selbst wieder Member (z. B. `DiagCpu.DNNmode`). Diese Prüfung steigt rekursiv bis zu den tatsächlich unbenutzten Blatt-Variablen ab, meldet aber nicht die

durchlaufenen Zwischen-Member selbst und nicht den Datenbaustein als Ganzes — ob ein Datenbaustein komplett unbenutzt ist, prüft ohnehin eigenständig [Prüfpunkt 11b](#).

- Wie bei Prüfpunkt 1a gilt: Ein Kommentar bzw. hier eine Verwendung auf dem Array selbst reicht — einzelne Array-Elemente (z. B. `Rezepte[3]`) werden nicht separat als unbenutzt gemeldet, ein großes Array-Member könnte sonst tausende Einzelbefunde erzeugen.
- Bei FC/OB zählt auch **Temp** als Interface-Section (anders als bei den Kommentar-Prüfpunkten 1c/26/27, wo Temp bewusst ausgenommen ist) — eine nie verwendete Temp-Variable ist eindeutig totes Gerümpel, und OBs haben meist ausschließlich Temp-Variablen als lokale Deklarationen.
- Wird ein UDT-/Struct-typisiertes Member als Ganzes an einen anderen Baustein übergeben (z. B. `MeinFb(duStruct := "MeinDb".MeinStruct)`), markieren sowohl `CrossReferenceService` (bei DBs) als auch der XML-Zugriffs-Scan (bei FB/FC/OB) typischerweise nur diesen Knoten selbst als referenziert. Bei `unterelemente_pruefen: false` (Standard) genügt das, damit die Variable insgesamt als verwendet gilt — praxisnah für Projekte, die UDT-Variablen überwiegend als Ganzes weiterreichen statt feldweise einzeln zuzugreifen.

Empfehlung zur Behebung Unbenutzte Variable entfernen oder die fehlende Verwendung ergänzen.

Prüfpunkt 11b — Unbenutzte Bausteine

Kategorie	Programmstruktur
Standard-Schweregrad	Warnung
Config-Schlüssel	<code>checks.programmstruktur.unbenutzte_bausteine</code>

Was wird geprüft? Ergänzend zu Prüfpunkt 11a, aber auf Bausteinebene: Für jeden Funktionsbaustein (FB), jede Funktion (FC) und jeden Datenbaustein (DB) wird geprüft, ob er von irgendeiner Stelle im Projekt aufgerufen bzw. referenziert wird.

Warum ist das wichtig? Ungenutzte Bausteine erschweren die Übersicht über ein Projekt und können bei künftigen Erweiterungen versehentlich für aktive Logik gehalten werden, obwohl sie längst funktionslos sind.

Parameter Dieser Prüfpunkt hat keine konfigurierbaren Parameter.

Beispiel

```
PLC_1 > Programmbausteine > FB_Reserve_Alt
→ Baustein 'FB_Reserve_Alt' wird von keiner Stelle im Projekt referenziert.
```

Besonderheiten

- **Organisationsbausteine (OB)** sind von diesem Prüfpunkt ausgenommen: Sie werden planmäßig vom Betriebssystem der Steuerung aufgerufen (z. B. OB1 zyklisch), nicht von anderem Anwendercode. Eine fehlende Referenz im Programmcode ist bei ihnen normal und kein Hinweis auf toten Code.
- **FB/FC und Instanz-Datenbausteine** werden anhand einer direkten Kreuzreferenz auf den Baustein selbst geprüft — ein `CALL` referenziert den Baustein bzw. die Instanz-DB als Ganzes, das ist ein zuverlässiges Signal. Bei Instanz-Datenbausteinen wird dabei ein permanenter, vorhandener Meta-

Eintrag (Typbeziehung zur eigenen FB, kein echter Aufruf) herausgefiltert — sonst würde jede Instanz-DB fälschlich als “benutzt” gelten, selbst wenn ihre FB nie aufgerufen wird, ein einzelnes Member der Instanz-DB von außen aber doch zugegriffen wird (was ohnehin unerwünscht ist und separat von [Prüfpunkt 25](#) gemeldet wird).

- **Global- und Array-Datenbausteine** werden dagegen anhand ihrer Member geprüft (wie bei [Prüfpunkt 11a](#)): Ein normaler DB wird nie “als Ganzes” aufgerufen, sondern immer nur über einzelne Variablen gelesen/geschrieben — eine direkte Referenz auf den DB selbst gibt es dafür in der Kreuzreferenz-Baumstruktur nicht. Der DB gilt hier als verwendet, sobald mindestens ein Member irgendwo im Projekt referenziert ist.

Empfehlung zur Behebung Unbenutzten Baustein entfernen oder den fehlenden Aufruf ergänzen.

Prüfpunkt 12a — Eingänge mindestens einmal gelesen

Kategorie	Programmstruktur
Standard-Schweregrad	Warnung
Config-Schlüssel	<code>checks.programmstruktur.eingaenge_gelesen</code>

Was wird geprüft? Für jeden Eingangs-Tag (siehe [Prüfpunkt 6](#)) wird geprüft, ob er im Programm mindestens einmal **lesend** verwendet wird. Wird er nirgends gelesen, wird ein Befund erzeugt.

Warum ist das wichtig? Ein Eingang, der nie gelesen wird, hat keinerlei Einfluss auf das Programmverhalten. Entweder wurde seine Auswertung schlicht vergessen, oder das Signal wird gar nicht mehr benötigt — dann sind Verdrahtung und Projektierung dieses Eingangs überflüssig.

Parameter Dieser Prüfpunkt hat keine konfigurierbaren Parameter.

Beispiel

```
PLC_1 > Variablentabellen > Tags_Eingaenge > I_Reserve_03
→ Eingang 'I_Reserve_03' wird im Programm nie gelesen.
```

Besonderheiten

- Anders als Prüfpunkt 11a (irgendeine Verwendung genügt) unterscheidet dieser Prüfpunkt gezielt zwischen **Lese-** und **Schreibzugriffen** — geprüft wird ausschließlich, ob tatsächlich gelesen wird. Ob ein Eingang fälschlich auch **beschrieben** wird, prüft der separate [Prüfpunkt 12b](#).
- Geprüft werden ausschließlich **PLC-Tags mit fester Hardware-Adresse** (%I / %Q) aus den Variablentabellen — nicht DB-Member, selbst wenn sie wie ein Eingang benannt sind. In Projekten, die Hardware-Ein-/Ausgänge über einen Organisationsbaustein in einen Datenbaustein spiegeln (dort arbeitet dann die eigentliche Logik weiter), sieht dieser Prüfpunkt nur die ursprünglichen PLC-Tags, nicht die gespiegelten DB-Member — die tatsächliche Anzahl geprüfter Ein-/Ausgänge kann dadurch deutlich kleiner ausfallen als die Anzahl der physisch verdrahteten Signale.

Empfehlung zur Behebung Prüfen, ob der Eingang tatsächlich benötigt wird — andernfalls Beschaltung bzw. Tag entfernen.

Prüfpunkt 12b — Eingänge dürfen nicht beschrieben werden

Kategorie	Programmstruktur
Standard-Schweregrad	Fehler
Config-Schlüssel	checks.programmstruktur.eingaenge_nicht_beschrieben

Was wird geprüft? Für jeden Eingangs-Tag wird gezählt, an wie vielen Stellen im Programm **schreibend** darauf zugegriffen wird. Wird er auch nur an einer einzigen Stelle beschrieben, wird ein Befund erzeugt.

Warum ist das wichtig? Eingänge werden bei jedem SPS-Zyklus vom Prozessabbild automatisch aus der Hardware überschrieben. Ein Schreibzugriff aus dem Anwenderprogramm hat dadurch keinerlei dauerhafte Wirkung — der geschriebene Wert wird spätestens im nächsten Zyklus wieder verworfen. Bestenfalls täuscht er also einen Wert vor, der so real nie ankommt; meist ist er schlicht eine Verwechslung mit einem Merker. Diese Regel gehört zu den grundlegendsten Konventionen der SPS-Programmierung, war in der ursprünglichen Liste der Prüfpunkte aber nicht als eigener Punkt enthalten — dieser Prüfpunkt schließt die Lücke.

Parameter Dieser Prüfpunkt hat keine konfigurierbaren Parameter.

Beispiel

```
PLC_1 > Variablentabellen > Tags_Eingaenge > I_Sensor_01
→ Eingang 'I_Sensor_01' wird im Programm an 1 Stelle(n) beschrieben –
  Eingänge dürfen nicht beschrieben werden.
```

Besonderheiten

- Anders als bei [Prüfpunkt 13](#) (Ausgänge dürfen mehrfach, aber nicht **mehrfach** beschrieben werden) genügt hier bereits ein **einzig** Schreibzugriff für einen Befund — Eingänge dürfen grundsätzlich gar nicht beschrieben werden.
- Der Standard-Schweregrad ist **Fehler**, nicht Warnung wie bei den meisten übrigen Prüfpunkten dieser Kategorie.
- Wie bei Prüfpunkt 12a: Es werden ausschließlich echte PLC-Tags mit Hardware-Adresse geprüft, keine DB-Member (siehe dortige Besonderheiten zum I/O-Spiegel-Pattern).

Empfehlung zur Behebung Schreibzugriff auf den Eingang entfernen — Eingänge sind nur lesend zu verwenden. Falls ein veränderbarer Wert benötigt wird, eine separate Merker- bzw. Hilfsvariable verwenden.

Prüfpunkt 13 — Ausgänge maximal einmal geschrieben

Kategorie	Programmstruktur
Standard-Schweregrad	Fehler
Config-Schlüssel	checks.programmstruktur.ausgaenge_mehrfach_schreiben

Was wird geprüft? Für jeden Ausgangs-Tag wird gezählt, an wie vielen Stellen im Programm **schreibend** darauf zugegriffen wird. Wird er an mehr als einer Stelle beschrieben, wird ein Befund erzeugt.

Warum ist das wichtig? Wird derselbe Ausgang an mehreren Stellen im Programm beschrieben, überschreiben sich die Zuweisungen je nach Ausführungsreihenfolge gegenseitig — das Ergebnis hängt dann von der Bearbeitungsreihenfolge der Bausteine ab und ist kaum noch vorhersehbar. Das gilt als einer der klassischen, schwer zu findenden Programmierfehler in der SPS-Technik.

Parameter Dieser Prüfpunkt hat keine konfigurierbaren Parameter.

Beispiel

```
PLC_1 > Variablentabellen > Tags_Ausgaenge > Q_Ventil_02  
→ Ausgang 'Q_Ventil_02' wird an 3 Stellen beschrieben.
```

Besonderheiten

- Der Standard-Schweregrad ist hier **Fehler** statt Warnung wie bei den meisten übrigen Prüfpunkten dieser Kategorie — das unterstreicht, wie ernst dieser Verstoß typischerweise einzustufen ist.
- Wie bei Prüfpunkt 12a: Es werden ausschließlich echte PLC-Tags mit Hardware-Adresse geprüft, keine DB-Member (siehe dortige Besonderheiten zum I/O-Spiegel-Pattern).

Empfehlung zur Behebung Schreibzugriffe auf den Ausgang auf eine einzige Stelle im Programm konsolidieren.

Prüfpunkt 14 — AWL-Code vorhanden

Kategorie	Programmstruktur
Standard-Schweregrad	Warnung
Config-Schlüssel	checks.programmstruktur.awl_code

Was wird geprüft? Für jeden Baustein wird geprüft, ob er in AWL/STL (“Anweisungsliste” / “Statement List”) programmiert ist.

Warum ist das wichtig? AWL gilt bei Siemens als veraltete Programmiersprache: TIA Portal unterstützt sie zwar weiterhin, entwickelt sie aber nicht mehr aktiv weiter und empfiehlt sie nicht für neue Projekte. AWL-Code gilt zudem als deutlich schwerer lesbar als die grafischen Sprachen (KOP/FUP) oder SCL.

Parameter Dieser Prüfpunkt hat keine konfigurierbaren Parameter.

Besonderheiten - Ist die Grundsprache eines Bausteins insgesamt AWL/STL, wird der ganze Baustein als ein Befund gemeldet. - TIA Portal erlaubt es aber auch, innerhalb eines Bausteins mit anderer Grundsprache (z. B. KOP/FUP) einzelne Netzwerke auf AWL umzuschalten. Ein solcher Baustein bleibt bei `block.ProgrammingLanguage` auf seiner Grundsprache stehen — nur der XML-Export je Netzwerk zeigt die abweichende Sprache. Dieser Prüfpunkt exportiert daher jeden Baustein, dessen Grundsprache nicht STL/SCL ist, und prüft zusätzlich jedes einzelne Netzwerk; ein gefundenes AWL-Netzwerk wird mit Netzwerknummer gemeldet, unabhängig von der Grundsprache des Bausteins.

Beispiel

```
PLC_1 > Programmbausteine > FC_Altcode
→ Baustein 'FC_Altcode' ist in AWL (STL) programmiert.

PLC_1 > Programmbausteine > 010rgPrg > Netzwerk 16
→ Netzwerk 16 in Baustein '010rgPrg' ist in AWL (STL) programmiert.
```

Empfehlung zur Behebung Baustein bzw. betroffenes Netzwerk nach KOP, FUP oder SCL migrieren.

Prüfpunkt 15 — Gemischte Programmiersprachen

Kategorie	Programmstruktur
Standard-Schweregrad	Warnung
Config-Schlüssel	checks.programmstruktur.gemischte_sprachen

Was wird geprüft? Für jeden Baustein wird geprüft, welche Programmiersprachen innerhalb seiner einzelnen Netzwerke verwendet werden. Kommt mehr als eine Sprache vor (z. B. ein Teil in KOP, ein anderer in FUP), wird ein Befund erzeugt.

Warum ist das wichtig? Wechselt die Sprache innerhalb eines Bausteins, muss man beim Lesen ständig zwischen unterschiedlichen Darstellungsweisen “umschalten” — das erschwert das Verständnis unnötig. Ein Baustein sollte durchgängig in einer Sprache programmiert sein.

Parameter

Parameter	Standardwert	Bedeutung
scl_in_fup_ignorieren	true	SCL-Netzwerke innerhalb eines ansonsten in FUP programmierten Bausteins sind in der betrieblichen Praxis weit verbreitet und gelten nicht als Problem. Ist die Baustein-Grundsprache FUP und die gefundene Sprachmischung genau FUP+SCL, wird der Baustein bei <code>true</code> nicht gemeldet. Jede andere Kombination (z. B. KOP+SCL, FUP+AWL, oder mehr als zwei Sprachen) bleibt unabhängig davon gemeldet. Bei <code>false</code> gilt das ursprüngliche Verhalten: jede Sprachmischung wird gemeldet.

Beispiel

```
PLC_1 > Programmbausteine > FB_Mischbetrieb
```

→ Baustein 'FB_Mischbetrieb' mischt mehrere Sprachen: FBD, LAD.

Besonderheiten

- Wie bei Prüfpunkt 10 und 16 gilt auch dieser Prüfpunkt nur für Bausteine mit einzelnen Netzwerken (KOP/FUP/GRAPH). Bausteine, die komplett in SCL oder AWL/STL programmiert sind, werden automatisch nicht geprüft.
- Mit dem Standardparameter `scl_in_fup_ignorieren: true` wird die betrieblich übliche Kombination FUP+SCL nicht mehr gemeldet — live an Salzmaschine verifiziert: 70 → 2 Befunde (die 2 verbleibenden sind echte Mischfälle: ein Baustein mit drei Sprachen sowie ein Baustein mit FUP+AWL).

Empfehlung zur Behebung Baustein auf eine einheitliche Programmiersprache vereinheitlichen.

Prüfpunkt 16 — Zu komplexe Netzwerke

Kategorie	Programmstruktur
Standard-Schweregrad	Warnung
Config-Schlüssel	<code>checks.programmstruktur.max_netzwerk_elemente</code>

Was wird geprüft? Für jedes grafische Netzwerk (KOP/FUP/GRAPH) wird die Anzahl der enthaltenen Programmelemente (Kontakte, Spulen, Bausteinaufrufe, Verknüpfungen usw.) gezählt und mit dem konfigurierten Schwellenwert verglichen. Für SCL-Netzwerke (eigenständige SCL-Bausteine sowie einzelne SCL-Netzwerke innerhalb eines sonst grafischen Bausteins) wird stattdessen die Anzahl der Code-Zeilen gezählt und mit einem eigenen Schwellenwert verglichen. Wird der jeweilige Schwellenwert überschritten, wird ein Befund erzeugt.

Warum ist das wichtig? Sehr umfangreiche Netzwerke sind auf einen Blick schwer zu erfassen und deuten häufig darauf hin, dass mehrere Teilfunktionen in ein einziges Netzwerk gequetscht wurden, statt sie sinnvoll aufzuteilen.

Parameter

Parameter	Standardwert	Bedeutung
<code>max_elemente</code>	50	Maximale Anzahl an Programmelementen je grafischem Netzwerk (KOP/FUP/GRAPH).
<code>max_zeilen_scl</code>	50	Maximale Anzahl an Code-Zeilen je SCL-Netzwerk.

Beispiel

PLC_1 > Programmbausteine > FB_Rezeptverwaltung > Netzwerk 2
→ Netzwerk hat 87 Elemente (Schwellenwert: 50).

PLC_1 > Programmbausteine > FC_Berechnung > Netzwerk 1
→ SCL-Netzwerk hat 74 Zeilen (Schwellenwert: 50).

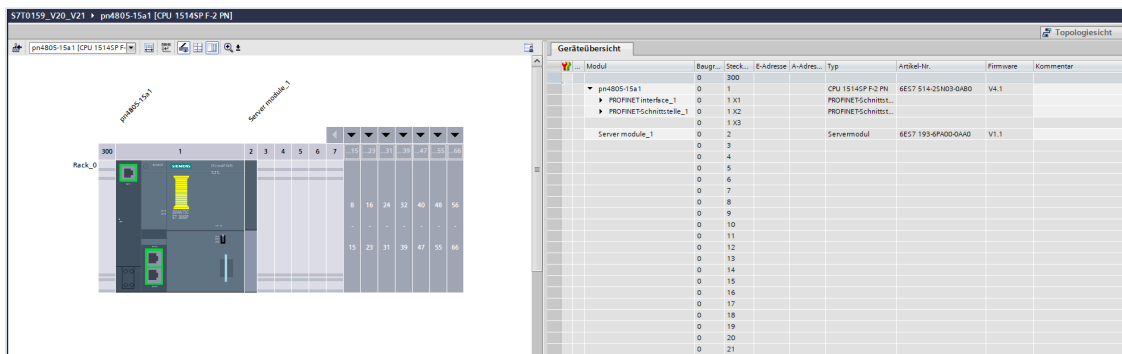
Besonderheiten

- AWL/STL wird weiterhin komplett nicht geprüft (gilt ohnehin als veraltet, siehe Prüfpunkt 14) — auch nicht als einzelnes AWL-Netzwerk innerhalb eines sonst nicht-AWL-Bausteins, da für Text-Netzwerke keine sinnvolle “Elementanzahl” existiert und AWL zur Migration ansteht statt weiter ausgebaut zu werden.
- Bis Version 0.41 wurden SCL-Netzwerke (weder als eigenständiger SCL-Baustein noch als einzelnes SCL-Netzwerk innerhalb eines sonst grafischen Bausteins) überhaupt nicht auf Komplexität geprüft — die Elementzählung liefert für SCL-Text immer 0. Seit Version 0.41 zählt ein SCL-Netzwerk stattdessen seine Code-Zeilen. Live an Salzmaschine verifiziert: 0 → 119 Befunde (Zeilenzahlen 50–459, überwiegend in der Siemens-Standardbibliothek `PrgBibSiemens/LGF`).

Empfehlung zur Behebung Netzwerk aufteilen oder die Teilfunktion in einen eigenen Baustein auslagern.

10.4 Hardware & Konfiguration (Prüfpunkte 17-18c)

Diese Kategorie verlässt die Ebene des Programmcodes und prüft die **Projektierung der Anlage selbst**: Ist überhaupt Peripherie-Hardware konfiguriert, sind CPU-Typ und Firmware eindeutig dokumentiert, ist ein eventuell vorhandenes Sicherheitsprogramm gegen unautorisierte Änderungen geschützt, und sind die Kommunikationszertifikate gültig?



Beispiel aus TIA Portal: die Hardwarekonfiguration einer PLC ohne zusätzliche Ein-/Ausgangsmodule

Prüfpunkt 17 — Hardware vorhanden und aktiviert

Kategorie	Hardware & Konfiguration
Standard-Schweregrad	Fehler
Config-Schlüssel	<code>checks.hardware.hardware_vorhanden</code>

Was wird geprüft? Für jede projektierte PLC wird geprüft, ob außer der CPU selbst noch mindestens ein weiteres Hardware-Modul konfiguriert ist (z. B. eine Ein-/Ausgangsbaugruppe). Ist die PLC komplett ohne

solche Zusatzmodule projektiert, wird ein Befund erzeugt.

Warum ist das wichtig? Ein Programm mit Ein-/Ausgangs-Tags setzt in aller Regel voraus, dass auch die passende Hardware dafür projektiert ist. Fehlt jegliche Zusatz-Hardware, deutet das entweder auf eine unvollständige Hardware-Konfiguration hin, oder darauf, dass die reale Anlage über ein anderes Gerät angesprochen wird als im Projekt hinterlegt.

Parameter Dieser Prüfpunkt hat keine konfigurierbaren Parameter.

Beispiel

```
PLC_1 > Hardwarekonfiguration
→ PLC 'PLC_1' hat keine zusätzlichen Hardware-Module konfiguriert
(nur die CPU selbst).
```

Besonderheiten

- Dieser Prüfpunkt ist bewusst vereinfacht: Er gleicht **nicht** ab, ob für jeden einzelnen I/O-Tag exakt das passende Adressbereichs-Modul vorhanden ist — das wäre ein deutlich aufwendigerer Abgleich. Geprüft wird nur, ob überhaupt zusätzliche Hardware projektiert ist. Eine PLC ganz ohne jede Zusatz-Hardware gilt als auffällig genug, um gemeldet zu werden.
- Bei der Zählung werden von TIA automatisch angelegte Strukturelemente herausgerechnet, die kein echtes I/O-Modul sind: der Baugruppenträger selbst sowie bei ET200SP-Stationen zusätzlich genau ein Busadapter/Schnittstellenmodul und genau ein Server-/Abschlussmodul. Ohne diese Korrektur hätte der Prüfpunkt bei ET200SP-Stationen (die in der Praxis immer mindestens Rack + Busadapter + Server-Modul als Zusatzelemente zeigen) nie anschlagen können, selbst wenn die PLC komplett ohne I/O-Hardware projektiert ist.

Empfehlung zur Behebung Hardware-Konfiguration prüfen — fehlendes Modul projektieren/aktivieren oder nicht mehr benötigten Tag entfernen.

Prüfpunkt 18a — CPU-Typ und Firmware-Version dokumentiert

Kategorie	Hardware & Konfiguration
Standard-Schweregrad	Warnung
Config-Schlüssel	checks.hardware.cpu_firmware_dokumentiert

Was wird geprüft? Für jede CPU wird geprüft, ob sowohl die Bestellnummer (CPU-Typ) als auch die Firmware-Version auslesbar hinterlegt sind. Fehlt eine der beiden Angaben, wird ein Befund erzeugt, der konkret benennt, welche Angabe fehlt.

Warum ist das wichtig? Bei Rückfragen, Ersatzteilbeschaffung oder einem Firmware-Update muss bekannt sein, um welche exakte CPU-Variante und Firmware-Version es sich handelt — ohne diese Angabe im Projekt muss dafür extra die reale Anlage kontaktiert werden.

Parameter Dieser Prüfpunkt hat keine konfigurierbaren Parameter.

Beispiel

```
PLC_1 > Hardwarekonfiguration  
→ Nicht dokumentiert: Firmware-Version.
```

Besonderheiten

- Der Befundtext benennt konkret, welche der beiden Angaben fehlt (CPU-Typ/Bestellnummer, Firmware-Version oder beide) — betroffen kann auch nur eine der beiden sein.

Empfehlung zur Behebung CPU-Typ und Firmware-Version in den Projekteigenschaften dokumentieren.

Prüfpunkt 18b — Passwortschutz bei Sicherheits-SPS

Kategorie	Hardware & Konfiguration
Standard-Schweregrad	Fehler
Config-Schlüssel	<code>checks.hardware.safety_passwort</code>

Was wird geprüft? Für jede CPU mit aktiviertem Sicherheitsprogramm (F-CPU, “Fail-safe”) wird geprüft, ob ein Safety-Offline-Passwort gesetzt ist. Fehlt es, wird ein Befund erzeugt.

Warum ist das wichtig? Das Safety-Offline-Passwort schützt das Sicherheitsprogramm einer F-CPU vor unautorisierten Änderungen — ein zentraler Baustein der funktionalen Sicherheit. Ohne dieses Passwort könnte sicherheitsrelevante Logik ohne Kontrolle verändert werden.

Parameter Dieser Prüfpunkt hat keine konfigurierbaren Parameter.

Beispiel

```
PLC_1 > Sicherheitsprogramm  
→ F-CPU 'PLC_1' hat kein Safety-Offline-Passwort gesetzt.
```

Besonderheiten

- Dieser Prüfpunkt betrifft ausschließlich CPUs mit lizenziertem und aktiviertem Sicherheitsprogramm (F-CPU). Bei gewöhnlichen Standard-CPU's ohne Sicherheitsprogramm wird er automatisch nicht ausgeführt — es entsteht in diesem Fall auch kein Befund.

Empfehlung zur Behebung F-Passwort für die Sicherheits-CPU vergeben (über die Safety-Administration in TIA Portal).

Prüfpunkt 18c — Kommunikationszertifikat

Kategorie	Hardware & Konfiguration
Standard-Schweregrad	siehe Besonderheiten — fest vorgegeben, nicht konfigurierbar

Config-Schlüssel

checks.hardware.zertifikat

Was wird geprüft? Für jede CPU wird der lokale Zertifikatsspeicher geprüft. Ist gar kein Kommunikationszertifikat vorhanden, oder ist eines bereits abgelaufen, wird ein Befund erzeugt. Läuft ein noch gültiges Zertifikat innerhalb der konfigurierten Restlaufzeit ab, wird ebenfalls ein Befund erzeugt.

Warum ist das wichtig? Kommunikationszertifikate sichern beispielsweise die verschlüsselte Verbindung zwischen CPU und Engineering-Station ab. Ein abgelaufenes oder fehlendes Zertifikat kann zu Kommunikationsausfällen führen, die im laufenden Betrieb überraschend auftreten, wenn sie nicht vorab erkannt werden.

Parameter

Parameter	Standardwert	Bedeutung
min_restlaufzeit_monate	6	Anzahl Monate vor Ablauf, ab der ein noch gültiges Zertifikat bereits als Warnung gemeldet wird.

Beispiel

```
PLC_1 > Zertifikate > <Zertifikats-ID>  
→ Zertifikat läuft bald ab (gültig bis 2026-12-01, Schwellenwert 6 Monate).
```

Besonderheiten

- Dieser Prüfpunkt ist der einzige in der gesamten Kategorie, dessen Status **nicht** vom konfigurierten Standard-Schweregrad abhängt, sondern für jeden Fall fest vorgegeben ist: kein Zertifikat vorhanden → **Fehler**, Zertifikat abgelaufen → **Fehler**, Zertifikat läuft innerhalb der Restlaufzeit ab → **Warnung**, sonst → **OK**. Die Einstellung `severity` in der Konfiguration hat für diesen Prüfpunkt also keine Wirkung — nur `min_restlaufzeit_monate` ist relevant.
- Dieser Prüfpunkt ist außerdem der einzige im gesamten Programm, der für einzelne, gültige, unauffällige Zertifikate ausdrücklich einen eigenen OK-Befund **pro Zertifikat** erzeugt. Bei allen anderen Prüfpunkten führt "kein Verstoß gefunden" dazu, dass für das jeweilige Objekt gar kein Eintrag in der Befundliste entsteht — läuft ein solcher Prüfpunkt aber komplett ohne Fehler/Warnung durch, erzeugt er (anders als 18c) nur einen einzigen zusammenfassenden OK-Befund für sich als Ganzes, nicht pro Objekt (siehe [Abschnitt 4.4](#)).
- Bis Version 0.43 importierte der Check `LocalCertificateManager` aus dem falschen Namespace (`Siemens.Engineering.SW.Security` statt `Siemens.Engineering.Security`) — der dadurch entstehende `ImportError` wurde von einer bewusst breiten Fehlerbehandlung (Dienst evtl. nicht verfügbar) stillschweigend verschluckt, sodass **jede** PLC unabhängig vom tatsächlichen Zertifikatsstatus als "kein Zertifikat vorhanden" gemeldet wurde. Seit Version 0.44 behoben.

Empfehlung zur Behebung Zertifikat einspielen bzw. rechtzeitig vor Ablauf erneuern.

10.5 Projektmetadaten (Prüfpunkte 19-21)

Diese Kategorie prüft die “Rahmendaten” eines Projekts — alles, was nicht in einzelnen Bausteinen steht, sondern das Projekt als Ganzes betrifft: Projekteigenschaften, aktive Sprachen, Übersetzbarkeit und Versionierung.

The screenshot shows a software configuration window titled "S7T0159_V20_V21 [Projekt]". It has two tabs: "Allgemein" (selected) and "Schutz". On the left is a tree view with "Projekt" selected. The main area is divided into sections: "Projekt" with fields for Name, Erstzeitpunkt, Letzte Änderung, Autor, Zuletzt geändert von, Version, Familie, Kommentar, and Copyright; "Details" with Speicherort and Größe in KB; and "Sprachen" with a list of active languages. The "Autor" field is empty, and the "Version" field is also empty. The "OK" button is highlighted in green.

Beispiel aus TIA Portal: die Projekteigenschaften mit leerem Feld “Author” und nicht gesetzter Versionsnummer

Prüfpunkt 19 — Kundeninformation in Projekteigenschaften

Kategorie	Projektmetadaten
Standard-Schweregrad	Warnung
Config-Schlüssel	checks.projektmetadaten.pflichtfelder

Was wird geprüft? Für jedes in der Konfiguration hinterlegte Pflichtfeld wird geprüft, ob es in den Top-Level-Projekteigenschaften ausgefüllt ist. Ist ein Feld leer, wird ein Befund erzeugt.

Warum ist das wichtig? Grundlegende Projektinformationen wie Autor oder Version sollten für jedes Projekt gepflegt sein, damit auch nach längerer Zeit oder bei Weitergabe klar ist, wer das Projekt erstellt hat und um welchen Stand es sich handelt.

Parameter

Parameter	Standardwert	Bedeutung
felder	["Author", "Version"]	Liste der zu prüfenden Projekteigenschaften-Felder (echte

Parameter	Standardwert	Bedeutung
		Openness-Attributnamen, siehe Tabelle unten).

Beispiel

Projekt > Eigenschaften > Author
→ Pflichtfeld 'Author' ist nicht ausgefüllt.

Besonderheiten

- Wichtig bei der Konfiguration: In der Liste `felder` müssen die echten, **englischen** internen Attributnamen der Openness-Schnittstelle stehen (z. B. `Author`) — nicht die deutsche Bezeichnung (“Autor”), wie sie in der TIA-Portal-Oberfläche angezeigt wird. Ein falsch geschriebener Feldname wird stillschweigend als leer gewertet und meldet dauerhaft einen Befund.
- **Vollständige Liste** der von der Openness-API als Top-Level- Projektattribute gelieferten Felder (live per `project.GetAttributeInfos()` gegen ein echtes Projekt geprüft, deutsche Bezeichnung laut V21-Referenz, Manual 03/2026, Abschnitt “Projektbezogene Attribute lesen”):

Attributname (für <code>felder</code>)	Deutsche Bedeutung	Als Pflichtfeld geeignet?
<code>Author</code>	Autor des Projekts	Ja (Standard)
<code>Comment</code>	Kommentar des Projekts	Ja — mehrsprachig, wird intern automatisch aufgelöst (siehe unten)
<code>Copyright</code>	Copyright-Hinweis des Projekts	Ja, falls im Projekt genutzt (siehe Hinweis unten)
<code>Family</code>	Familie des Projekts	Ja, falls im Projekt genutzt (siehe Hinweis unten)
<code>Version</code>	Version des Projekts	Ja (Standard)
<code>CreationTime</code>	Erstellungszeitpunkt	Nein — technisch, von TIA automatisch gesetzt, nie leer
<code>LastModified</code>	Zeitpunkt der letzten Änderung	Nein — technisch, von TIA automatisch gesetzt, nie leer
<code>LastModifiedBy</code>	Autor der letzten Änderung	Nein — technisch, von TIA automatisch gesetzt
<code>Name</code>	Projektname	Nein — kann in TIA gar nicht leer sein
<code>Path</code>	Absoluter Projektpfad	Nein — kein Textfeld (<code>FileInfo</code> - Objekt)
<code>Size</code>	Projektgröße in KB	Nein — kein Textfeld (Zahl)

Attributname (für <code>felder</code>)	Deutsche Bedeutung	Als Pflichtfeld geeignet?
<code>IsModified</code>	Projekt seit letztem Speichern geändert?	Nein — kein Textfeld (Wahrheitswert)
<code>LanguageSettings</code>	Sprachverwaltung des Projekts	Nein — kein Textfeld (eigenes Objekt, siehe Prüfpunkt 20)

`Copyright` und `Family` tauchen im TIA-Portal-Standarddialog “Projekteigenschaften” möglicherweise gar nicht als editierbares Feld auf — vor Aufnahme in `felder` in TIA prüfen, ob sich das Feld überhaupt befüllen lässt (sonst meldet der Prüfpunkt dauerhaft einen nicht behebbaren Befund).

- `Comment` wird — anders als die übrigen Textfelder — nicht direkt als `System.String` geliefert, sondern als mehrsprachiges `Siemens.Engineering.MultilingualText`-Objekt (`GetAttribute("Comment")` liefert dafür sogar `None`). Der Prüfpunkt behandelt `Comment` deshalb intern als Sonderfall und liest den Text der Projekt-Referenzsprache über dieselbe Hilfsfunktion, die bereits für mehrsprachige Kommentarfelder auf Baustein-/Tag-Ebene verwendet wird — bei der Konfiguration selbst ist dadurch keine Sonderbehandlung nötig, `Comment` wird wie jedes andere Feld einfach in `felder` eingetragen.

Empfehlung zur Behebung Pflichtfeld in den Projekteigenschaften ausfüllen.

Prüfpunkt 20 — Anzahl Sprachen

Kategorie	Projektmetadaten
Standard-Schweregrad	Warnung
Config-Schlüssel	<code>checks.projektmetadaten.max_sprachen</code>

Was wird geprüft? Es wird gezählt, wie viele Sprachen im Projekt aktiv sind (also für Kommentare, Netzwerktitle und andere Texte zur Verfügung stehen), und mit dem konfigurierten Maximum verglichen. Wird das Maximum überschritten, wird ein Befund mit der vollständigen Liste aller aktiven Sprachen erzeugt.

Warum ist das wichtig? Zusätzliche Sprachen werden während der Entwicklung häufig testweise aktiviert und danach nicht mehr deaktiviert. Jede zusätzliche Sprache bedeutet doppelte Pflege aller Kommentare und Texte — bleiben nicht mehr benötigte Sprachen aktiv, drohen sie leer oder veraltet zu bleiben.

Parameter

Parameter	Standardwert	Bedeutung
<code>max</code>	2	Maximale Anzahl gleichzeitig aktiver Sprachen.

Beispiel

```
Projekt > Eigenschaften > Sprachen
→ 4 aktive Sprachen konfiguriert (Schwellenwert: 2):
    Deutsch, Englisch, Französisch, Italienisch.
```

Empfehlung zur Behebung Nicht mehr benötigte Sprachen aus den Projekteigenschaften entfernen.

Prüfpunkt 21 — Kompilierfehler und Warnungen

Kategorie	Projektmetadaten
Standard-Schweregrad	siehe Besonderheiten — fest vorgegeben, nicht konfigurierbar
Config-Schlüssel	<code>checks.projektmetadaten.kompilierfehler</code>

Was wird geprüft? Für jede PLC-Software im Projekt wird ein vollständiger Übersetzungsvorgang (Kompilieren) angestoßen. Jede dabei zurückgemeldete Compiler-Meldung wird als eigener Befund aufgenommen: Meldungen mit mindestens einem Fehler gelten als Fehler-Befund, reine Warnmeldungen als Warnung.

Warum ist das wichtig? Kompilierfehler bedeuten, dass sich ein Baustein nicht in ein lauffähiges Programm übersetzen lässt — der schwerwiegendste denkbare Verstoß, denn ein nicht übersetzbares Programm lässt sich gar nicht erst auf die Steuerung laden. Auch Compiler-Warnungen weisen häufig auf reale Probleme hin (z. B. implizite Typumwandlungen), auch wenn sie die Übersetzung selbst nicht verhindern.

Parameter

Parameter	Standardwert	Bedeutung
<code>ignorierte_meldungen</code>	<code>[]</code>	Liste literaler Teiltexthe (kein Regex); ein Treffer irgendwo im Meldungstext unterdrückt die betreffende Compiler-Meldung vollständig und dauerhaft.

Beispiel

```
PLC_1 > Compiler-Meldung > FB_Motor\Netzwerk 3  
→ Adressbereich überschritten.
```

Besonderheiten

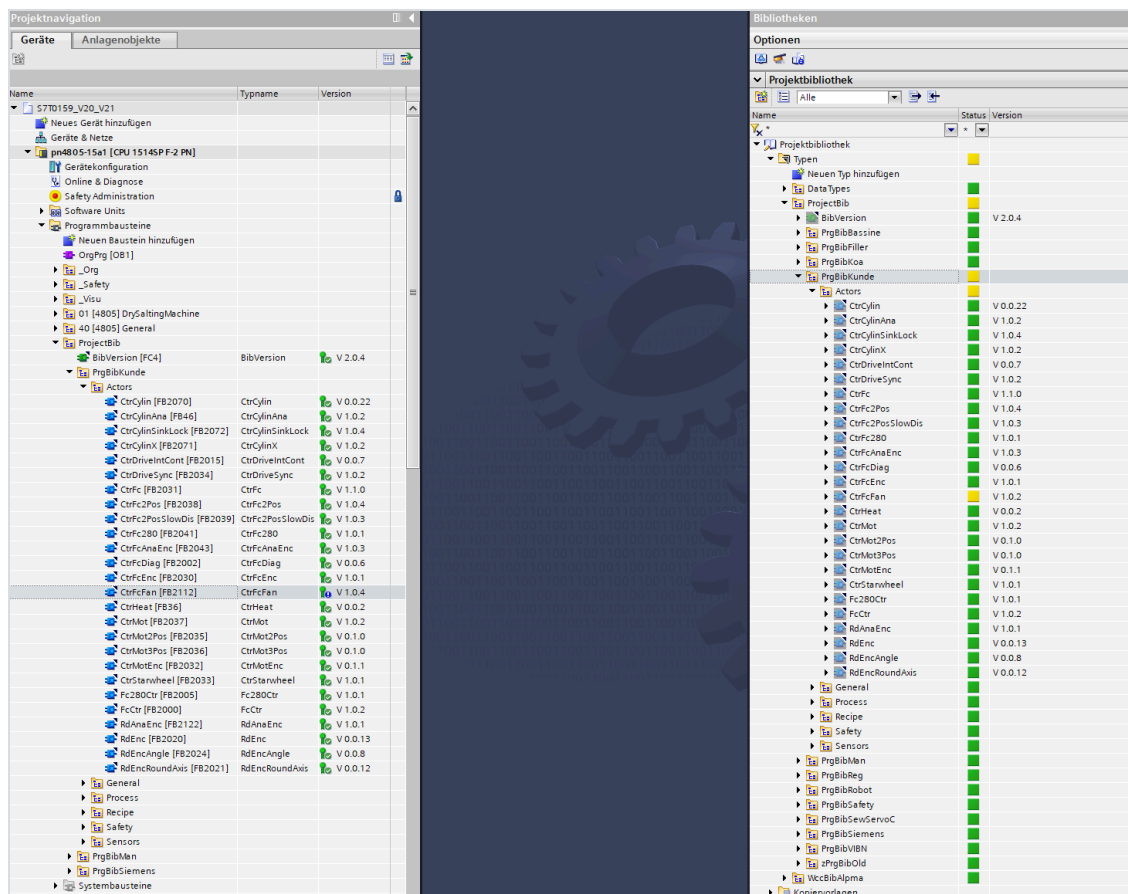
- Lläuft — fest vorgegeben, nicht konfigurierbar — immer als **letzter** Prüfpunkt eines Laufs, nachdem alle anderen Prüfpunkte bereits abgeschlossen sind. Grund: Das Kompilieren ist der einzige Vorgang im gesamten Programm, der den Übersetzungsstand des Projekts im TIA-Portal-Speicher verändert (siehe “Was das Programm ausdrücklich nicht tut” in [Kapitel 1](#)) — würde er zwischen anderen Prüfpunkten laufen, könnten nachfolgende Prüfpunkte auf einem anderen Projektstand aufsetzen als die vorherigen. Das Projekt wird dabei nicht gespeichert; die Änderung betrifft nur die laufende, headless geöffnete Session.
- Für das Übersetzen müssen laut TIA-Portal-Vorgabe alle Geräte offline sein. Läuft die Prüfung z. B. gegen ein Projekt mit online geschalteten Geräten, kann bereits der Übersetzungsvorgang selbst fehlschlagen — auch das wird als eigener Fehler-Befund gemeldet (“Übersetzen fehlgeschlagen: ...”).

- `ignorierte_meldungen` prüft — anders als sonst bei ähnlichen Ausnahme- Parametern in diesem Programm üblich (siehe z. B. `ausnahme_titel_regex` bei Prüfpunkt 10) — bewusst **kein** Regex, sondern reinen, case- insensitive Teiltext-Abgleich: Ein Eintrag muss nur irgendwo im Meldungstext vorkommen, egal an welcher Stelle. Grund: Reale Compiler- Meldungen enthalten sehr häufig Klammern und Anführungszeichen (z. B. `"(Project > Properties > Protection)"`) — als Regex- Muster verwendet, würden diese Zeichen als Gruppierungs-Metazeichen interpretiert statt als literale Zeichen und den Abgleich dadurch **stillschweigend** (ohne Fehlermeldung) zum Scheitern bringen, selbst wenn exakt der komplette, live beobachtete Meldungstext eingetragen wurde. Da der praktische Anwendungsfall ohnehin immer “genau diese eine bekannte Meldung dauerhaft unterdrücken” lautet, wird hier nie ein Wildcard-Muster benötigt — reiner Teiltext ist deshalb robuster als Regex. Gedacht für Meldungen, die bewusst und dauerhaft als irrelevant eingestuft werden, typischerweise weil sie bei sehr vielen schreibgeschützten Bausteinen auftreten (z. B. aus zugekauften/bereits fertigen Bibliotheken) — Beispiele aus der Praxis: `"since it is write-protected"`, `"because it is not editable"`. Ein Treffer unterdrückt die Meldung unabhängig von Baustein und Fehler-/ Warnungsstatus — im Zweifel gilt weiterhin: lieber Rücksprache mit dem Entwickler halten, statt eine unbekannte Meldung hier pauschal einzutragen.
- Ähnlich wie bei [Prüfpunkt 18c](#) hat der konfigurierte Standard-Schweregrad hier keine Wirkung auf einzelne Befunde: Ob ein Befund als Fehler oder als Warnung erscheint, hängt ausschließlich davon ab, ob der Compiler dazu mindestens einen Fehler oder nur Warnungen zurückgemeldet hat.

Empfehlung zur Behebung Compiler-Meldung beheben und Baustein neu übersetzen.

10.6 Bibliotheken & Typen (Prüfpunkte 22-23)

Diese Kategorie prüft den Umgang mit Bibliotheken und wiederverwendeten Typinstanzen: Sind eingesetzte Bibliotheksbausteine noch auf dem aktuellen Stand, und gibt es “verwaiste” Datenbausteine, deren zugehöriger Funktionsbaustein nicht mehr existiert?



Beispiel aus TIA Portal: die Aktualitätsprüfung der Projektbibliothek mit einer als veraltet markierten Typinstanz

Prüfpunkt 22 — Bibliotheksbausteine auf aktuellem Stand

Kategorie	Bibliotheken & Typen
Standard-Schweregrad	Warnung
Config-Schlüssel	checks.bibliotheken.veraltete_bibliotheken

Was wird geprüft? Für die Projektbibliothek wird ein Aktualitätscheck durchgeführt: Für jeden aus der Bibliothek abgeleiteten Typ (z. B. einen Baustein oder Datentyp, der auf einem Bibliothekstyp basiert) wird geprüft, ob die im Projekt verwendete Instanz noch der aktuellen Version des Bibliothekstyps entspricht. Jede veraltete Instanz wird als eigener Befund gemeldet.

Warum ist das wichtig? Wird ein Bibliothekstyp weiterentwickelt — etwa weil ein Fehler behoben oder eine Funktion ergänzt wurde —, wirkt sich das nur dann auf das Projekt aus, wenn die verwendeten Instanzen auch tatsächlich aktualisiert werden. Veraltete Instanzen laufen unbemerkt mit dem alten, womöglich fehlerhaften Stand weiter.

Parameter Dieser Prüfpunkt hat keine konfigurierbaren Parameter.

Beispiel

Projektbibliothek

→ Typinstanz 'FB_Ventil' ist veraltet (aktuelle Bibliotheksversion: 2.1).

Besonderheiten

- Dieser Prüfpunkt deckt ausschließlich die **Projektbibliothek** ab (die im Projekt selbst gespeicherte Bibliothek). Globale Bibliotheken, die z. B. zentral im Netzwerk liegen und von mehreren Projekten gemeinsam genutzt werden, werden derzeit nicht geprüft.

Empfehlung zur Behebung Bibliothekstyp in der Projektbibliothek aktualisieren und betroffene Instanzen neu generieren.

Prüfpunkt 23 — Instanz-DBs ohne zugehörigen FB

Kategorie	Bibliotheken & Typen
Standard-Schweregrad	Fehler
Config-Schlüssel	<code>checks.bibliotheken.verwaiste_instanz_dbs</code>

Was wird geprüft? Für jeden Instanz-Datenbaustein (den einem konkreten Funktionsbaustein- Aufruf zugeordneten DB) wird geprüft, ob der zugehörige Funktionsbaustein (FB) noch im Projekt existiert. Ist der FB nicht mehr vorhanden, wird ein Befund erzeugt.

Warum ist das wichtig? Ein Instanz-DB ohne zugehörigen FB ("verwaist") ist funktionslos — er kann nicht mehr korrekt genutzt werden, belegt aber weiterhin Platz im Projekt und kann bei einer Sichtprüfung fälschlich für aktive Logik gehalten werden. Dieser Zustand entsteht meist, wenn ein FB gelöscht oder umbenannt wurde, ohne die zugehörigen Instanz-DBs zu bereinigen.

Parameter Dieser Prüfpunkt hat keine konfigurierbaren Parameter.

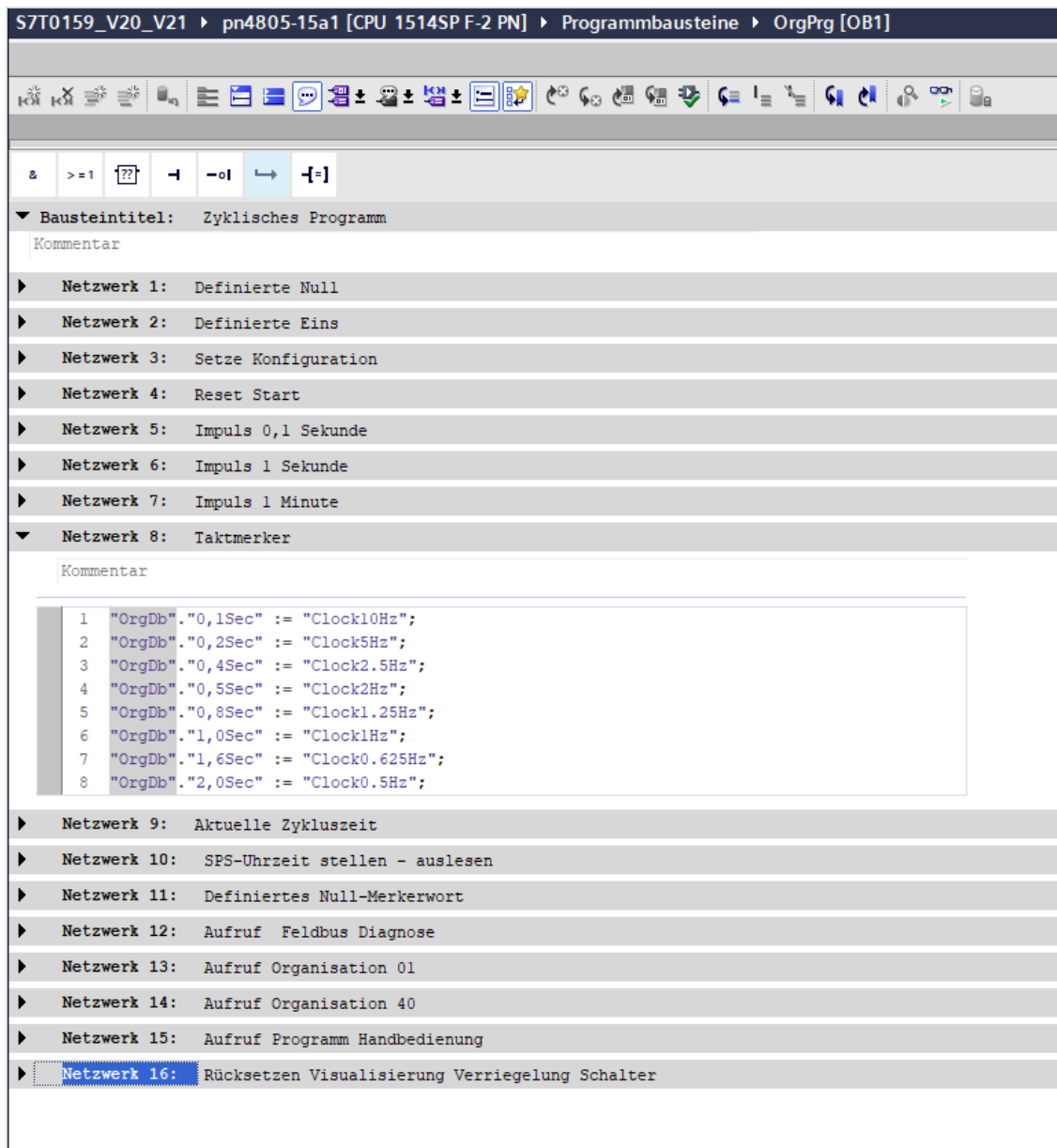
Beispiel

```
PLC_1 > Datenbaustein > DB_Ventil_Instance
→ Instanz-DB 'DB_Ventil_Instance' referenziert FB 'FB_Ventil_Alt',
  der nicht mehr im Projekt existiert.
```

Empfehlung zur Behebung Verwaisten Instanz-DB entfernen oder den zugehörigen FB wiederherstellen.

10.7 Siemens Styleguide & Best Practices (Prüfpunkte 24-33)

Diese letzte und umfangreichste Kategorie fasst Empfehlungen aus dem Siemens Standardisierungsleitfaden sowie allgemein anerkannte Best Practices der SPS-Programmierung zusammen. Sie deckt ein breites Spektrum ab — von der sauberen Kapselung interner Bausteindaten über die Vermeidung unnötiger Instanz-Datenbausteine bis zur Projektorganisation.



Beispiel aus TIA Portal: der Organisationsbaustein OB1 mit auffällig vielen Netzwerken, die eigene Logik statt reiner Bausteinaufrufe enthalten

Prüfpunkt 24 — Sprachen konsistent

Kategorie	Siemens Styleguide & Best Practices
Standard-Schweregrad	Warnung
Config-Schlüssel	checks.styleguide.sprachen_konsistent

Was wird geprüft? Es wird geprüft, ob die im Projekt hinterlegte Referenzsprache (die Sprache, in der Kommentare und Texte primär verfasst werden sollen) der konfigurierten Erwartungssprache entspricht.

Warum ist das wichtig? Wechselt die Referenzsprache unbemerkt — etwa weil beim Anlegen des Projekts versehentlich Englisch statt Deutsch gewählt wurde —, kann das dazu führen, dass neue Kommentare in der “falschen” Sprache erfasst werden, ohne dass es sofort auffällt.

Parameter

Parameter	Standardwert	Bedeutung
erwartete_sprache	de	Sprachkürzel, dem die Referenzsprache des Projekts entsprechen soll.

Beispiel

```
Projekt > Eigenschaften > Sprache  
→ Referenzsprache des Projekts ('en-US') weicht von 'de' ab.
```

Besonderheiten

- Dieser Prüfpunkt ist bewusst vereinfacht: Er prüft die **projektweite** Referenzsprache, nicht die tatsächlich in jedem einzelnen Kommentar oder Netzwerktitel verwendete Sprache — das würde eine echte Spracherkennung pro Text erfordern und ist nicht Teil dieses Prüfpunkts.

Empfehlung zur Behebung Kommentare und Netzwerktitel einheitlich in der Projektsprache verfassen bzw. die Referenzsprache im Projekt korrigieren.

Prüfpunkt 25 — Direkter Zugriff auf Static-Tags von außen

Kategorie	Siemens Styleguide & Best Practices
Standard-Schweregrad	Warnung
Config-Schlüssel	checks.styleguide.static_zugriff_extern

Was wird geprüft? Für jeden Instanz-Datenbaustein wird geprüft, ob eine seiner “Static”-Variablen (interne, dauerhaft gespeicherte Variablen des zugehörigen Funktionsbausteins) von einer anderen Stelle im Projekt als dem Funktionsbaustein selbst direkt gelesen oder beschrieben wird.

Warum ist das wichtig? Static-Variablen sind als interner Zustand eines Funktionsbausteins gedacht — der vorgesehene Weg, mit einem FB zu kommunizieren, führt über seine Ein- und Ausgangsparameter. Greift Code von außen direkt auf den Instanz-DB zu, umgeht das diese Schnittstelle und macht den Baustein schwerer wartbar: Änderungen an seiner internen Struktur können dann unerwartet andere Stellen im Programm beeinflussen.

Parameter Dieser Prüfpunkt hat keine konfigurierbaren Parameter.

Beispiel

```
PLC_1 > Datenbaustein > DB_Ventil_Instance > Member > InternerZaehler  
→ Static-Tag 'InternerZaehler' wird von außerhalb ('FC_Diagnose / NW6')  
direkt zugegriffen.
```

Besonderheiten

- Dieser Prüfpunkt meldete seit seiner Einführung nie einen Treffer (zwei unabhängige, mittlerweile behobene Fehler in der Auswertung der Kreuzreferenz-Rohdaten). Live an Salzmaschine verifiziert (`01PrgDb > 1x_30M1StopGap` , extern zugegriffen von `01Vis`).
- Multiinstanz-typisierte Static-Member (deren Datentyp selbst ein FB ist) tragen in der Kreuzreferenz zusätzlich einen technischen Metaeintrag zur eigenen Typbeziehung — dieser wird herausgefiltert und nicht als externer Zugriff missverstanden.
- Neben dem zugreifenden Baustein wird zusätzlich die Netzwerk- bzw. Codestelle innerhalb dieses Bausteins angegeben (z. B. `010rg / NW6`) — diese Information steckt bereits in denselben Kreuzreferenz-Rohdaten wie der Bausteinname und wurde bislang nur nicht mit ausgewertet. Je nach Programmiersprache des zugreifenden Netzwerks unterscheidet sich das Format: `NW6` bei einem grafischen Netzwerk (KOP/FUP), `Ln: 12 C1: 4` bei einem eigenständigen SCL-Baustein ohne Netzwerkgliederung, oder `NW 10 Ln: 45 C1: 43` bei einem einzelnen SCL-Netzwerk innerhalb eines sonst grafischen Bausteins. Ein eventueller Netzwerktitel wird dabei bewusst nicht mit übernommen (oft lang, für die Meldung nicht nötig).

Empfehlung zur Behebung Zugriff über Ein-/Ausgangsparameter des FB kapseln, statt direkt auf den Instanz-DB zuzugreifen.

Prüfpunkt 26 — Output-/InOut-Tag pro Zyklus nur einmal beschrieben

Kategorie	Siemens Styleguide & Best Practices
Standard-Schweregrad	Fehler
Config-Schlüssel	<code>checks.styleguide.output_mehrfach_beschrieben</code>

Was wird geprüft? Für jeden `VAR_OUTPUT`- und `VAR_IN_OUT`-Parameter eines Funktionsbausteins oder einer Funktion wird gezählt, an wie vielen Stellen **innerhalb** des Bausteins er beschrieben wird. Wird er an mehr als einer Stelle beschrieben, wird ein Befund erzeugt.

Warum ist das wichtig? Analog zu [Prüfpunkt 13](#) bei Ausgangs-Tags führt das mehrfache Beschreiben desselben Output- bzw. InOut-Parameters innerhalb eines Bausteins zu unvorhersehbarem Verhalten, das von der internen Ausführungsreihenfolge abhängt.

Parameter Dieser Prüfpunkt hat keine konfigurierbaren Parameter.

Beispiel

```
PLC_1 > Programmbausteine > FB_Regler > Member > Fehlercode
→ Output-Parameter 'Fehlercode' wird an 2 Stellen beschrieben.
```

Besonderheiten

- Dieser Prüfpunkt meldete seit seiner Einführung nie einen Treffer (die Kreuzreferenzabfrage lieferte am FB/FC direkt keinen auswertbaren Ergebnisbaum). Die Prüfung erfolgt jetzt stattdessen direkt anhand des Baustein-codes (sprachunabhängig, SCL wie FBD/LAD) und deckt damit erstmals auch Funktionen (FC) ab, nicht nur Funktionsbausteine (FB). Live an Salzmaschine verifiziert, u. a. `LSNTP_Server` (`status` , `error` , `statusID` je 4 Schreibzugriffe).

- Der Standard-Schweregrad ist hier **Fehler** statt Warnung — wie beim eng verwandten Prüfpunkt 13.
- Auf Nutzerwunsch werden neben VAR_OUTPUT- auch VAR_IN_OUT-Parameter geprüft (dieselbe Logik gilt hier gleichermaßen: ein InOut-Parameter, der intern an mehreren Stellen überschrieben wird, hat dasselbe Ausführungsreihenfolge-Problem). Die Meldung unterscheidet die Parameterart im Text (Output-Parameter '...' bzw. InOut-Parameter '...').

Empfehlung zur Behebung Schreibzugriffe auf den Output-Parameter auf eine Stelle im Baustein konsolidieren.

Prüfpunkt 27 — UDT für wiederkehrende Strukturen

Kategorie	Siemens Styleguide & Best Practices
Standard-Schweregrad	Warnung
Config-Schlüssel	checks.styleguide.udt_wiederkehrende_strukturen

Was wird geprüft? Für alle Datenbausteine wird die jeweilige Struktur (die Liste aller Variablennamen mit ihrem Datentyp) verglichen. Kommt exakt dieselbe Struktur in mindestens zwei verschiedenen Datenbausteinen vor, ohne dass sie als eigener PLC-Datentyp (UDT) ausgelagert ist, wird für jeden betroffenen DB ein Befund erzeugt.

Warum ist das wichtig? Kommt dieselbe Datenstruktur mehrfach unabhängig voneinander vor, muss bei einer notwendigen Änderung (z. B. ein zusätzliches Feld) jede einzelne Fundstelle manuell angepasst werden — vergisst man dabei eine Stelle, laufen die Strukturen unbemerkt auseinander. Ein gemeinsamer PLC-Datentyp (UDT) muss dagegen nur an einer Stelle geändert werden.

Parameter Dieser Prüfpunkt hat keine konfigurierbaren Parameter.

Beispiel

```
PLC_1 > Datenbaustein > DB_Rezept_A
→ Struktur von 'DB_Rezept_A' ist identisch zu 2 weiteren DBs
(DB_Rezept_B, DB_Rezept_C) – als UDT auslagern.
```

Besonderheiten

- Datenbausteine mit weniger als zwei Variablen gelten als zu trivial, um als “wiederkehrende Struktur” gezählt zu werden, und werden von diesem Prüfpunkt automatisch ausgenommen.

Empfehlung zur Behebung Wiederkehrende Struktur als PLC-Datentyp (UDT) anlegen und in den betroffenen Datenbausteinen referenzieren.

Prüfpunkt 28 — OB1 (Main) Komplexität

Kategorie	Siemens Styleguide & Best Practices
Standard-Schweregrad	Warnung

Config-Schlüssel	checks.styleguide.ob1_komplexitaet
------------------	------------------------------------

Was wird geprüft? Für den zyklischen Hauptorganisationsbaustein OB1 wird gezählt, wie viele seiner Netzwerke eigene Logik enthalten (also mehr als nur einen einzelnen Bausteinaufruf). Übersteigt diese Zahl den konfigurierten Schwellenwert, wird ein Befund erzeugt.

Warum ist das wichtig? OB1 sollte idealerweise als reine “Aufrufzentrale” dienen, die andere Bausteine in der richtigen Reihenfolge aufruft. Enthält OB1 selbst viel eigene Logik, wird er unübersichtlich, und die eigentliche Programmstruktur ist schwerer zu erkennen.

Parameter

Parameter	Standardwert	Bedeutung
max_netzwerke_mit_logik	5	Maximale Anzahl an Netzwerken mit eigener Logik in OB1.

Beispiel

```
PLC_1 > Programmbausteine > Main [OB1]
→ OB1 enthält 8 Netzwerke mit eigener Logik (Schwellenwert: 5) –
  Logik in eigene Bausteine auslagern.
```

Besonderheiten

- Ein Netzwerk, das nur einen einzigen Bausteinaufruf enthält (also nur den Aufruf selbst, ohne zusätzliche Verknüpfungen), zählt **nicht** als “eigene Logik” — reine Aufrufnetzwerke wirken sich also nicht negativ auf diesen Prüfpunkt aus.

Empfehlung zur Behebung Logik aus OB1 in eigene Bausteine auslagern — OB1 sollte primär Bausteine aufrufen.

Prüfpunkt 29 — Know-How-Schutz dokumentiert

Kategorie	Siemens Styleguide & Best Practices
Standard-Schweregrad	Warnung
Config-Schlüssel	checks.styleguide.know_how_schutz

Was wird geprüft? Für jeden Baustein mit aktiviertem Know-how-Schutz (Kopierschutz, der den Bausteininhalt vor dem Einsehen durch Dritte schützt) wird geprüft, ob die Kopfbeschreibung einen entsprechenden Hinweis enthält (die Wörter “know-how” oder “knowhow”). Fehlt dieser Hinweis, wird ein Befund erzeugt.

Warum ist das wichtig? Ein know-how-geschützter Baustein lässt sich von außen nicht einsehen — ohne einen Hinweis in der Kopfbeschreibung ist auf den ersten Blick nicht erkennbar, dass es sich überhaupt um einen geschützten Baustein handelt, was bei der Fehlersuche oder Projektübergabe zu Verwirrung führen kann.

Parameter

Parameter	Standardwert	Bedeutung
dokumentations_hinweise	["know-how", "knowhow"]	Liste literaler Teiltexthe; mindestens einer muss case-insensitiv in der Kopfbeschreibung vorkommen, damit der Schutz als dokumentiert gilt.

Beispiel

```
PLC_1 > Programmbausteine > FB_Rezeptalgorithmus
→ Baustein 'FB_Rezeptalgorithmus' ist know-how-geschützt, aber nicht
als solcher dokumentiert.
```

Besonderheiten

- Die Prüfung auf den Dokumentationshinweis erfolgt als einfache, case-insensitive Textsuche in der Kopfbeschreibung (kein Regex, analog zu `ignorierte_meldungen` bei Prüfpunkt 21) — eine Formulierung, die nicht in `dokumentations_hinweise` eingetragen ist, wird nicht erkannt.

Empfehlung zur Behebung Know-how-Schutz im Bausteinkopf bzw. in der Projektdokumentation vermerken.

Prüfpunkt 30 — Tag-Tabellen nur I/O-Tags

Kategorie	Siemens Styleguide & Best Practices
Standard-Schweregrad	Warnung
Config-Schlüssel	<code>checks.styleguide.tag_tabellen_nur_io</code>

Was wird geprüft? Für jede Variablentabelle wird geprüft, ob sie sowohl echte I/O-Tags (mit Peripherie-Adresse, siehe [Prüfpunkt 6](#)) als auch andere Tags (z. B. interne Merker ohne Adresse) enthält. Ist das der Fall, wird ein Befund für die gesamte Tabelle erzeugt.

Warum ist das wichtig? Werden I/O-Tags und interne Merker in derselben Tabelle vermischt, ist auf einen Blick schwerer erkennbar, welche Signale tatsächlich mit der physischen Anlage verbunden sind und welche rein programmintern sind.

Parameter Dieser Prüfpunkt hat keine konfigurierbaren Parameter.

Beispiel

```
PLC_1 > Variablentabellen > Tags_Allgemein
→ Tag-Tabelle 'Tags_Allgemein' enthält sowohl I/O-Tags als auch
andere Tags – in getrennte Tabellen aufteilen.
```

Besonderheiten

- Leere Variablentabellen werden von diesem Prüfpunkt automatisch übersprungen.
- Eine Tag-Tabelle, die ausschließlich Nicht-I/O-Tags enthält (also gar keine I/O-Tags gemischt sind), wird bewusst **nicht** gemeldet — geprüft wird ausschließlich die Mischung, nicht das bloße Vorhandensein von Nicht-I/O-Tags an sich (Review-General.md, Befund 10b).

Empfehlung zur Behebung Nicht-I/O-Tags (z. B. Merker) in eine eigene Tag-Tabelle verschieben.

Prüfpunkt 31 — Nicht-optimierte Bausteine

Kategorie	Siemens Styleguide & Best Practices
Standard-Schweregrad	Warnung
Config-Schlüssel	<code>checks.styleguide.nicht_optimierte_bausteine</code>

Was wird geprüft? Für jeden Baustein wird geprüft, ob er mit “Standard”-Bausteinzugriff (statt “Optimiert”) projiziert ist. Ist das der Fall, wird ein Befund erzeugt.

Warum ist das wichtig? Optimierter Bausteinzugriff ist bei modernen S7-1200/1500-Steuerungen der empfohlene Standard: TIA Portal verwaltet dabei die Adressierung der Variablen automatisch, was Adressierungsfehler ausschließt und in der Regel performanter ist. Standard-Zugriff mit fester, manuell vergebener Adressierung gilt heute meist nur noch aus Kompatibilitätsgründen als notwendig.

Parameter Dieser Prüfpunkt hat keine konfigurierbaren Parameter.

Beispiel

```
PLC_1 > Programmbausteine > DB_Altsystem  
→ Baustein 'DB_Altsystem' ist nicht optimiert (MemoryLayout = Standard).
```

Besonderheiten

- Es gibt durchaus legitime technische Gründe für Standard-Zugriff (z. B. bestimmte Kommunikationsbausteine oder ältere Steuerungsfamilien) — ein Befund bei diesem Prüfpunkt ist als Hinweis zum Prüfen zu verstehen, nicht automatisch als Fehler.

Empfehlung zur Behebung Bausteinzugriff auf “Optimiert” umstellen, sofern kein technischer Grund dagegenspricht.

Prüfpunkt 32 — Bausteine im Root ohne Ordnerstruktur

Kategorie	Siemens Styleguide & Best Practices
Standard-Schweregrad	Warnung
Config-Schlüssel	<code>checks.styleguide.bausteine_im_root</code>

Was wird geprüft? Es wird gezählt, wie viele Bausteine direkt im Wurzelordner der Programmbausteine liegen (also nicht in einem Unterordner gruppiert sind), und mit dem konfigurierten Schwellenwert verglichen. Wird er überschritten, wird ein Befund erzeugt.

Warum ist das wichtig? Liegen sehr viele Bausteine unstrukturiert direkt im Wurzelordner, wird die Navigation im Projekt zunehmend unübersichtlich — eine thematische Gruppierung in Unterordner (z. B. nach Anlagenteil oder Funktion) erleichtert das Wiederfinden erheblich.

Parameter

Parameter	Standardwert	Bedeutung
<code>max_bausteine_root</code>	<code>20</code>	Maximale Anzahl an Bausteinen direkt im Wurzelordner.

Beispiel

```
PLC_1 > Programmbausteine
→ 34 Bausteine liegen direkt im Root-Ordner (Schwellenwert: 20) –
  in Unterordner gruppieren.
```

Empfehlung zur Behebung Bausteine in thematische Unterordner/Gruppen einsortieren.

Prüfpunkt 33 — Schreibschutz von Bausteinen

Kategorie	Siemens Styleguide & Best Practices
Standard-Schweregrad	Warnung
Config-Schlüssel	<code>checks.styleguide.schreibschutz</code>

Was wird geprüft? Für jeden Baustein mit aktiviertem Schreibschutz wird geprüft, ob die Kopfbeschreibung einen entsprechenden Hinweis enthält (die Begriffe “schreibschutz” oder “write-protect”). Fehlt dieser Hinweis, wird ein Befund erzeugt.

Warum ist das wichtig? Analog zu [Prüfpunkt 29](#) lässt sich ein schreibgeschützter Baustein zwar weiterhin einsehen, aber nicht mehr verändern. Ohne Dokumentationshinweis ist nicht sofort erkennbar, dass Änderungsversuche am Baustein absichtlich blockiert sind, was bei einer geplanten Anpassung zunächst für Verwirrung sorgen kann.

Parameter

Parameter	Standardwert	Bedeutung
<code>dokumentations_hinweise</code>	<code>["schreibschutz", "write-protect"]</code>	Liste literaler Teiltexthe; mindestens einer muss case-insensitiv in der Kopfbeschreibung vorkommen, damit der Schutz als dokumentiert gilt.

Beispiel

```
PLC_1 > Programmbausteine > FB_Kalibrierung  
→ Baustein 'FB_Kalibrierung' ist schreibgeschützt, aber nicht  
entsprechend dokumentiert.
```

Besonderheiten

- Dieser Prüfpunkt ist **neu in TIA Portal V21** — das zugrunde liegende Attribut (`IsWriteProtected`) ist erst seit dieser Version über die Openness-Schnittstelle verlässlich auslesbar.
- Die Prüfung auf den Dokumentationshinweis erfolgt als einfache, case-insensitive Textsuche in der Kopfbeschreibung (kein Regex, analog zu `ignorierte_meldungen` bei Prüfpunkt 21 sowie demselben Parameter bei Prüfpunkt 30) — eine Formulierung, die nicht in `dokumentations_hinweise` eingetragen ist, wird nicht erkannt.

Empfehlung zur Behebung Schreibschutz im Bausteinkopf bzw. in der Projektdokumentation vermerken.

Kapitel 10 ist damit vollständig: Alle 34 aktuellen Prüfpunkte (inkl. der Unterpunkte 11b, 18b und 18c) sind ausgearbeitet — ergänzt um Prüfpunkt 1b (UDT ohne Kommentar), Prüfpunkt 1c (FB-Interface-Member ohne Kommentar) und Prüfpunkt 12b (Eingänge dürfen nicht beschrieben werden), die alle drei nachträglich als sinnvolle Ergänzung hinzugekommen sind, sowie um den ursprünglichen Prüfpunkt 22 (Projektversion) bereinigt, der sich als redundant zu Prüfpunkt 19 herausstellte (siehe Änderungshistorie in Anhang C). Rückmeldungen und Korrekturen sind jederzeit willkommen — dieses Handbuch bleibt bis zu einer ersten Durchsicht als Entwurf gekennzeichnet (siehe Kopf des Dokuments).

Anhang A: Glossar

Begriff	Erklärung
TIA Portal	Engineering-Software von Siemens zur Projektierung von SPS-Steuerungen.
TIA Portal Openness API	Programmierschnittstelle von Siemens, über die externe Programme wie der TIA Linter lesend (und grundsätzlich auch schreibend) auf TIA-Portal-Projekte zugreifen können.
Baustein	Sammelbegriff für die Programmeinheiten eines SPS-Programms (z. B. Funktionsbaustein FB, Funktion FC, Organisationsbaustein OB).
DB (Datenbaustein)	Baustein, der ausschließlich Daten (Variablen) speichert, keine Programmlogik.
PLC-Tag	Eine benannte, projektweit gültige Variable, meist mit Bezug zu einem Ein- oder Ausgang der Steuerung.
Netzwerk	Ein logischer Programmierabschnitt innerhalb eines Bausteins.
Variablentabelle	Eine Sammlung von PLC-Tags, meist thematisch gruppiert (z. B. Eingänge, Ausgänge).
Headless	Betrieb eines Programms ohne sichtbare Benutzeroberfläche im Hintergrund — hier: Zugriff auf TIA Portal, ohne dass sich das TIA-Portal-Fenster öffnet.

Anhang B: Aus diesem Handbuch ein PDF erstellen

Dieses Handbuch liegt als reine Markdown-Datei (`Handbuch.md`) vor und wurde bewusst so geschrieben, dass sich daraus ohne Nacharbeit ein sauberes PDF erzeugen lässt: einfache Überschriften, einfache Tabellen, Bilder über Standard-Markdown-Syntax. Die einzige Ausnahme sind die unsichtbaren Seitenumbruch-Marker vor jedem Kapitel (siehe unten) — reines HTML, das in jeder normalen Markdown-Ansicht ohnehin nicht sichtbar ist.

Bilder ergänzen

An vielen Stellen im Handbuch steht bereits ein Bild-Platzhalter in der Form `![Beschreibung]` (`images/dateiname.png`) . Um ein solches Bild einzufügen:

1. Screenshot bzw. Grafik als `.png` speichern, mit demselben Dateinamen, der im Platzhalter steht (z. B. `gui-eingabeseite-gesamt.png`).
2. Die Datei in den Ordner `docs/images/` legen — also **neben** die Datei `Handbuch.md` , nicht in einen weiteren Unterordner.
3. Sobald die Datei dort liegt, wird sie beim nächsten PDF-Export (siehe unten) automatisch mit eingebunden — an der Markdown-Datei selbst muss nichts mehr geändert werden.

Bis ein Bild tatsächlich vorhanden ist, zeigen die meisten Markdown-Ansichten (auch diese Vorschau) an der jeweiligen Stelle nur den Beschreibungstext oder ein Platzhalter-Symbol an — das ist normal und kein Fehler.

Seitenumbrüche

Vor jedem Kapitel (`##` -Überschrift, also Kapitel 1–10 sowie Anhang A–C) steht im Quelltext eine unsichtbare Zeile:

```
<div style="page-break-after: always;"></div>
```

Das ist reines HTML und in normalen Markdown-Ansichten (Obsidian, GitHub, VS-Code-Vorschau) unsichtbar — es verändert also nichts am Lesefluss am Bildschirm. Markdown selbst kennt keinen eigenen Befehl für Seitenumbrüche; dieses HTML-Snippet ist der gebräuchlichste Ersatz dafür.

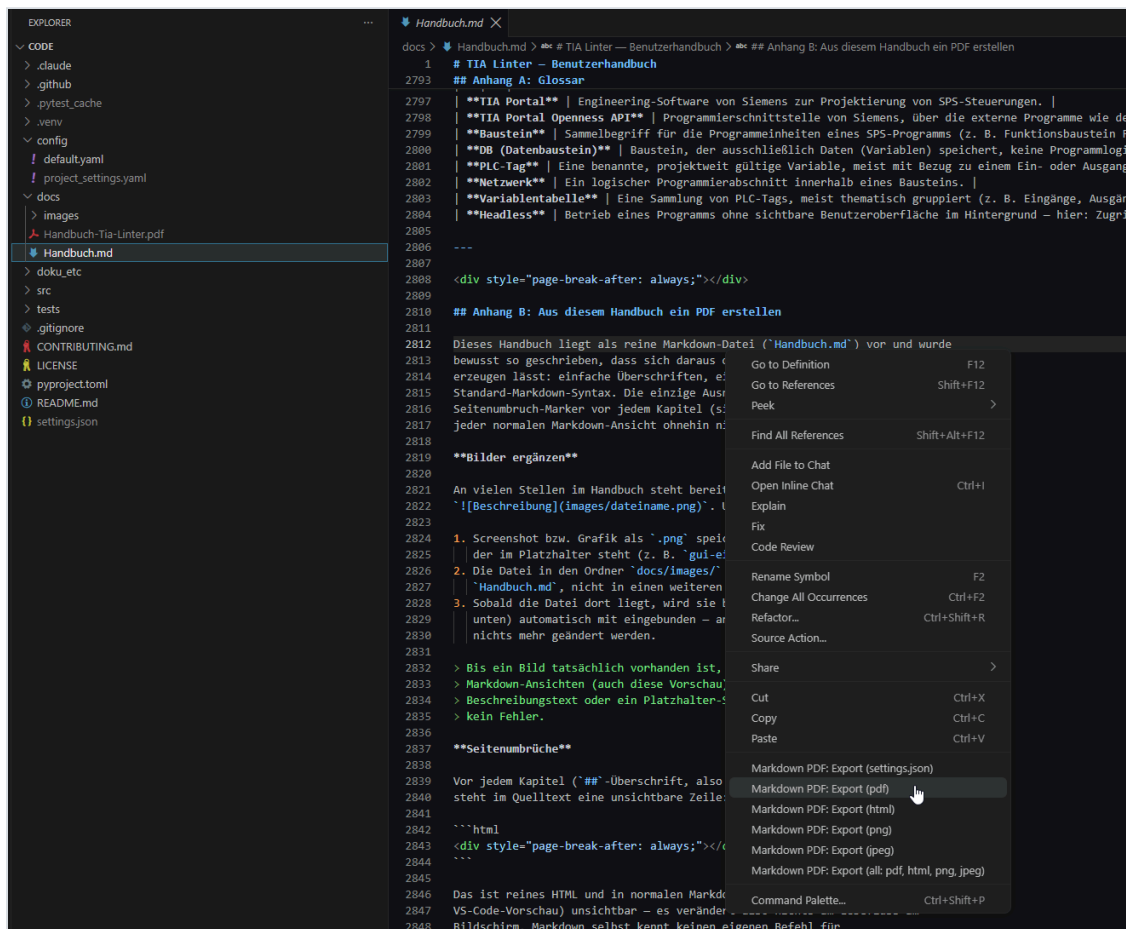
Wichtig: Dieser Marker funktioniert zuverlässig mit **Variante 1** unten (VS Code, Chrome-basiert) — dort beginnt jedes Kapitel auf einer neuen PDF-Seite. Bei **Variante 2** (Pandoc) wird er dagegen in der Standardeinstellung stillschweigend **ignoriert** (kein Fehler, aber auch kein Umbruch), da Pandoc PDFs standardmäßig über LaTeX erzeugt und LaTeX rohes HTML nicht auswertet. Wer feste Seitenumbrüche über Pandoc braucht, müsste stattdessen an jeder Stelle einen rohen LaTeX-Block (````\{=latex\} / \newpage / ````) einfügen — das wiederum würde in VS Code/Obsidian/GitHub als

sichtbarer Codeblock erscheinen. Beide Mechanismen gleichzeitig zu pflegen war für dieses Handbuch nicht vorgesehen; falls du überwiegend über Pandoc exportierst, sag Bescheid.

Es gibt mehrere gleichwertige Wege, aus der Datei ein PDF zu erzeugen — hier die zwei einfachsten:

Variante 1 — Visual Studio Code (am einfachsten, keine Kommandozeile nötig)

1. Visual Studio Code öffnen und die Erweiterung **“Markdown PDF”** (von yzane) installieren.
2. Die Datei `Handbuch.md` in VS Code öffnen.
3. Rechtsklick im Dokument → **“Markdown PDF: Export (pdf)”**.
4. Das PDF wird automatisch im selben Ordner wie die Markdown-Datei erzeugt.



Rechtsklick-Kontextmenü in Visual Studio Code mit dem Eintrag **“Markdown PDF: Export (pdf)”**

Variante 2 — Pandoc (Kommandozeile, für technisch versierte Nutzer)

Mit installiertem [Pandoc](#) genügt im Ordner mit der Datei folgender Befehl:

```
pandoc Handbuch.md -o Handbuch.pdf --toc -V lang=de
```

Die Option `--toc` erzeugt dabei automatisch ein anklickbares, mit Seitenzahlen versehenes Inhaltsverzeichnis am Anfang des PDFs — zusätzlich zu der Übersicht, die bereits am Anfang der Markdown-Datei steht.

Beide Varianten erzeugen aus derselben Datei ein vollständiges PDF, **inklusive** aller Bilder, die zu diesem Zeitpunkt bereits unter `docs/images/` liegen. Wird das Handbuch später um weitere Kapitel oder Bilder ergänzt, müssen diese Schritte einfach nur erneut ausgeführt werden.

Anhang C: Änderungshistorie dieses Handbuchs

Version	Datum	Änderung
0.1	17.07.2026	Ersteinrichtung: Programmübersicht, Voraussetzungen, Grundprinzip der Prüfung, vollständige Bedienungsanleitung der Oberfläche, PDF-Report, Pfad-Format, Log-Datei, Glossar. Kapitel 10 ("Die Prüfpunkte im Detail") ist als Platzhalter angelegt und noch nicht ausgefüllt.
0.2	17.07.2026	Kapitel 10 begonnen: Abschnitt 10.1 "Kommentare & Beschreibungen" (Prüfpunkte 1–4) vollständig ausgearbeitet, inkl. Parametern, Beispielen und Besonderheiten. Übersichtstabelle für den Bearbeitungsstand von Kapitel 10 ergänzt.
0.3	17.07.2026	Abschnitt 10.2 "Namenskonventionen" (Prüfpunkte 5–9) ausgearbeitet, inkl. der beiden geteilten Prüfpunkte 6 (Eingänge/Ausgänge) und 7 (FB/FC).
0.4	17.07.2026	Abschnitt 10.3 "Programmstruktur" (Prüfpunkte 10–16, inkl. 11b "Unbenutzte Bausteine") ausgearbeitet.
0.5	17.07.2026	Abschnitt 10.4 "Hardware & Konfiguration" (Prüfpunkte 17–18c) ausgearbeitet, inkl. Hinweis auf die Sonderrolle von Prüfpunkt 18c (fest vorgegebener Status statt konfigurierbarem Schweregrad, einziger Prüfpunkt mit explizitem OK-Befund).
0.6	17.07.2026	Abschnitt 10.5 "Projektmetadaten" (Prüfpunkte 19–22) ausgearbeitet, inkl. Hinweis auf englische vs. deutsche Feldnamen bei Prüfpunkt 19 und die Sonderrolle von Prüfpunkt 21 (Status abhängig vom Compiler-Ergebnis statt konfigurierbarem Schweregrad).
0.7	17.07.2026	Abschnitt 10.6 "Bibliotheken & Typen" (Prüfpunkte 23–24) ausgearbeitet.
0.8	17.07.2026	Abschnitt 10.7 "Siemens Styleguide & Best Practices" (Prüfpunkte 25–35) ausgearbeitet. Damit ist Kapitel 10 mit allen 35 Prüfpunkten vollständig — das Handbuch trägt bis zu einer ersten Durchsicht weiterhin die Kennzeichnung "Entwurf".
0.9	17.07.2026	Über 20 Bild-Platzhalter (! [Beschreibung] (images/dateiname.png)) an allen Stellen ergänzt, an denen ein Screenshot oder eine Grafik sinnvoll ist (GUI-Seiten, PDF-Report-Beispiele, Pfad-Format, Log-Datei, je ein Beispielbild pro Prüfpunkt-Kategorie in Kapitel 10). Ordner docs/images/ samt README für die Bilddateien angelegt; Anhang B um eine Anleitung zum Ergänzen der Bilder erweitert.
0.10	17.07.2026	Abschnitt 5.2 um die neue Programmerweiterung ausgeschlossene_ordner ergänzt (Ordner samt Unterordnern von der Prüfung ausnehmen, z. B. bereits geprüfte Bibliotheksordner).

Version	Datum	Änderung
0.11	17.07.2026	Seitenumbruch-Marker (<code><div style="page-break-after: always;"></code>) vor allen 14 Kapitel-/Anhang-Überschriften ergänzt, damit jedes Kapitel im PDF auf einer neuen Seite beginnt. In Anhang B dokumentiert, inkl. Hinweis, dass der Marker nur mit Variante 1 (VS Code) zuverlässig funktioniert und bei Variante 2 (Pandoc/LaTeX) standardmäßig folgenlos ignoriert wird.
0.12	17.07.2026	Prüfpunkt 5 (DB-Namensformat) in Abschnitt 10.2 an die Programmerweiterung angepasst: jetzt als zwei unabhängig konfigurierbare Einträge für Global-/Array-DB und Instanz-DB beschrieben (analog zu Prüfpunkt 6/7), inkl. Hinweis zur Einordnung von Array-DBs.
0.13	17.07.2026	Abschnitt 5.2 um die neue Programmerweiterung <code>ausgeschlossene_bausteine</code> ergänzt (einzelne Bausteine per Name komplett von jedem Prüfpunkt ausnehmen, unabhängig vom Ordner — Ergänzung zu <code>ausgeschlossene_ordner</code>).
0.14	17.07.2026	Neuen Prüfpunkt 12b ("Eingänge dürfen nicht beschrieben werden") in Abschnitt 10.3 ergänzt — schließt eine Lücke in der ursprünglichen Prüfpunkte-Liste, Standard-Schweregrad Fehler. Querverweis bei Prüfpunkt 12 ergänzt, Abschlusshinweis am Ende von Kapitel 10 aktualisiert.
0.15	18.07.2026	Neuen Prüfpunkt 1b ("UDT ohne Kommentar") in Abschnitt 10.1 ergänzt — schließt die Lücke, die dadurch entsteht, dass Prüfpunkt 1 Items innerhalb eines UDT-typisierten DB-Members ab dieser Version bewusst nicht mehr einzeln prüft (analog zu Array-Elementen genügt dort ein Kommentar auf der Variable selbst). Prüfpunkt 1b prüft stattdessen sowohl den Kommentar des UDT selbst als auch die Kommentare aller seiner Items direkt an der UDT-Definition; verschachtelte UDT-Items werden dabei nicht rekursiv mitgeprüft, da das verschachtelte UDT eigenständig geprüft wird. Querverweis bei Prüfpunkt 1 ergänzt, Abschlusshinweis am Ende von Kapitel 10 aktualisiert.
0.16	18.07.2026	Neuen Parameter <code>ausnahme_variables</code> bei Prüfpunkt 1 in Abschnitt 10.1 dokumentiert — erlaubt das Ausnehmen einzelner Variablen (PLC-Tags oder DB-Member) anhand des vollständigen Namens, exakte Übereinstimmung, unabhängig von <code>ausnahme_prefixe</code> .
0.17	18.07.2026	Neuen Parameter <code>ausnahme_udts</code> bei Prüfpunkt 1 in Abschnitt 10.1 dokumentiert — erlaubt das manuelle Ausnehmen von Datentypnamen (UDTs), deren Items nicht geprüft werden sollen, gedacht vor allem für System-/Bibliotheksdantypen ohne sichtbare Definition in TIA Portal. Standardwert von <code>ausnahme_prefixe</code> in der Parameter-Tabelle auf <code>["___"]</code> korrigiert (Programmänderung: einfacher Unterstrich als Präfix in der Praxis oft ungeeignet).

Version	Datum	Änderung
0.18	18.07.2026	Besonderheiten von Prüfpunkt 1 in Abschnitt 10.1 klargestellt: die UDT-Erkennung schützt die Items einer UDT-typisierten Variable auch dann, wenn die Variable selbst über <code>ausnahme_prefixe</code> / <code>ausnahme_variables</code> ausgenommen ist (Programmänderung: vorher wurde eine so ausgenommene, aber UDT-typisierte Variable nie als UDT erkannt, wodurch ihre Items fälschlich einzeln geprüft wurden).
0.19	18.07.2026	Neuen Prüfpunkt 1c ("FB-Interface-Member ohne Kommentar") in Abschnitt 10.1 ergänzt — schließt die Lücke, die dadurch entsteht, dass Prüfpunkt 1 Items innerhalb einer Multi-Instanz-FB-Variable ab dieser Version ebenfalls bewusst nicht mehr einzeln prüft (analog zur UDT-Behandlung aus Prüfpunkt 1b). Prüfpunkt 1c prüft die Interface-Member (Input/Output/InOut/Static) direkt an der FB-Definition, nicht den FB-Kopfkommentar (bleibt Sache von Prüfpunkt 2). Technischer Hinweis ergänzt: die Openness-API liefert für FBs keine Interface-Member über die direkte Objektnavigation (anders als bei DBs/UDTs) — der Prüfpunkt liest sie stattdessen aus dem XML-Export. Querverweis bei Prüfpunkt 1 ergänzt, veralteten <code>ausnahme_prefixe</code> -Standardwert bei Prüfpunkt 1b auf <code>["__"]</code> korrigiert, Abschlusshinweis am Ende von Kapitel 10 aktualisiert.
0.20	18.07.2026	Redundanz zwischen Prüfpunkt 1 und 1c behoben (User-Brainstorming): Prüfpunkt 1 prüfte bislang zusätzlich jedes Instanz-DB-Member einzeln (mit Fallback auf den FB-Kommentar), was bei fehlendem FB-Kommentar zu doppelten Befunden führte. Prüfpunkt 1 überspringt Instanz-DBs jetzt komplett — Prüfpunkt 1c ist die alleinige Quelle für FB-Interface-Member-Kommentare, egal ob als Multi-Instanz oder als eigenständige Instanz-DB verwendet. Tradeoff dokumentiert: ein an einer einzelnen Instanz überschriebener, abweichender Kommentar wird dadurch nicht mehr erkannt. Besonderheiten bei Prüfpunkt 1 und 1c entsprechend ergänzt.
0.21	18.07.2026	Zwei Programmänderungen dokumentiert: (1) Läuft ein Prüfpunkt komplett ohne Fehler/Warnung durch, meldet er jetzt genau einen zusammenfassenden OK-Befund statt gar keinen — Abschnitt 4.4 entsprechend überarbeitet, Besonderheiten von Prüfpunkt 18c klargestellt (bleibt einziger Prüfpunkt mit Befunden pro Einzelobjekt statt nur einem zusammenfassenden OK). (2) In der Eingabeseite steht jetzt links neben jedem Kontrollkästchen die zugehörige Prüfpunkt-Nummer aus diesem Handbuch — Abschnitt 6.2 entsprechend ergänzt.
0.22	18.07.2026	Neunter Kommentar-Bug behoben (User-Meldung, live an ausgeschlossenen Ordner "ProjectBib" verifiziert): Bei Prüfpunkt 1/1b wurden UDT-/FB-Namen zur Typ-Erkennung bislang mit <code>ausgeschlossene_ordner</code> / <code>ausgeschlossene_bausteine</code> gefiltert, wodurch ein UDT oder FB aus einem ausgeschlossenen Ordner dem Linter als solcher unbekannt wurde — eine damit typisierte Variable wurde dadurch fälschlich doch wieder in ihre (nicht einsehbaren) Items hinein geprüft. Fix: die Typ-Erkennungslisten

Version	Datum	Änderung
		umfassen jetzt immer alle UDTs/FBs der PLC-Software, unabhängig von Ausschlüssen — nur die eigentliche Prüfung selbst bleibt davon betroffen. Besonderheiten von Prüfpunkt 1 in Abschnitt 10.1 entsprechend ergänzt.
0.23	19.07.2026	Zehnter Kommentar-Bug behoben (User-Meldung, live an FB <code>Ø10rgPrg</code> und dessen Multi-Instanz <code>iou_ReqRcp / lu_RcpReq</code> verifiziert): <code>interface_section_members()</code> (<code>_tia_helpers.py</code>), gemeinsam genutzt von Prüfpunkt 1c sowie den Prüfpunkten 26 und 27 (<code>static_zugriff_extern / output_mehrfach_beschrieben</code>), suchte im XML-Export bislang nach <i>jeder</i> Section mit passendem Namen im gesamten Dokument — dadurch wurde auch das verschachtelte Sub-Interface eines Multi-Instanz-Members fälschlich als Teil des äußeren Bausteins behandelt, seine Sub-Member wurden mitgeprüft, obwohl sie eigenständig geprüft werden, sobald die Prüfung beim referenzierten FB selbst ankommt. Fix: die Funktion steigt beim Baumdurchlauf nicht mehr in <code><Member></code>-Elemente hinein ab, wodurch nur noch die Sections des Bausteins selbst gefunden werden. Live verifiziert an <code>Ø10rgPrg</code>: Anzahl der Static-Interface-Member sank von fälschlich 13 (4 direkte + 9 aus dem verschachtelten Sub-Interface) auf korrekt 4; projektweit sank Prüfpunkt 1c von 256 auf 45 Befunde. Docstring von <code>FbMemberKommentarCheck</code> korrigiert (die dortige Annahme “keine rekursive Auflösung nötig” war zuvor schlicht falsch).
0.24	19.07.2026	Neuer Parameter <code>check_db</code> bei Prüfpunkt 2 (User-Meldung: Kopfbeschreibungen sind bei Datenbausteinen unüblich) — Standardwert <code>false</code>: Datenbausteine werden von der Kopfbeschreibungs-Prüfung ausgenommen, FB/FC/OB bleiben davon unberührt und werden weiterhin immer geprüft; <code>true</code> stellt das bisherige Verhalten (alle Bausteintypen) wieder her. Parameter-Tabelle und Besonderheiten bei Prüfpunkt 2 entsprechend ergänzt. Außerdem: <code>config/default.yaml</code> durchgehend um kurze, einzeilige Prüfpunkt-Kommentare vor jedem Konfigurationsbereich ergänzt (analog zu den bereits bestehenden, ausführlicheren Kommentaren bei Prüfpunkt 1b/1c) — jeder der 35 Prüfpunkte lässt sich damit direkt in der YAML-Datei anhand seiner Nummer wiederfinden.
0.25	19.07.2026	Zwölfter Kommentar-Bug behoben (User-Meldung, live an FC <code>400rg</code> verifiziert): Prüfpunkt 2 las bislang nur das <code>Comment</code>-Attribut eines Bausteins — <code>PlcBlock</code> hat laut V21-Openness-Referenz aber zwei unabhängige mehrsprachige Felder, <code>Title</code> und <code>Comment</code>, beide im Bausteinkopf sichtbar. Bei <code>400rg</code> war <code>Comment</code> für jede Sprache leer, obwohl <code>Title</code> die sichtbare Kopfbeschreibung “Organisation Störungen Allgemein” trug — der Baustein wurde fälschlich als unbeschrieben gemeldet. Fix: Es zählt jetzt das längere der beiden Felder. Live verifiziert: 48 → 34 Befunde über alle Bausteintypen

Version	Datum	Änderung
		hinweg (isoliert vom <code>check_db</code> -Filter aus Version 0.24 gemessen), mit der Standardkonfiguration (<code>check_db: false</code>) 48 → 3. “Was wird geprüft?” und Besonderheiten bei Prüfpunkt 2 entsprechend ergänzt.
0.26	19.07.2026	Dreizehnter Kommentar-Bug behoben (User-Meldung: “fast alle Warnungen sind falsch”): Prüfpunkt 3 las den Netzwerktitel bislang aus der <code>AttributeList</code> eines <code>CompileUnit</code> -XML-Elements (analog zu einfachen Attributen wie <code>ProgrammingLanguage</code>) — Titel/Kommentar eines Netzwerks liegen im XML-Export aber als eigene, mehrsprachige <code>MultilingualText</code> -Komposition im <code>ObjectList</code> , strukturell identisch zu <code>PlcBlock.Title / .Comment</code> (siehe Version 0.25). Die alte Leselogik fand dadurch nie einen Titel, unabhängig davon, ob einer gepflegt war. Zusätzlicher, davon unabhängiger Fallstrick beim Fix selbst: <code>reference_language(project).Culture</code> liefert ein <code>System.Globalization.CultureInfo</code> -NET-Objekt, kein <code>System.String</code> — ein direkter Vergleich gegen die aus dem XML gelesenen reinen Text-Kultur-Codes (z. B. “de-DE”) schlug deshalb zunächst trotz des Strukturfixes weiterhin fehl, bis <code>str(...)</code> ergänzt wurde. Live verifiziert: 148 → 1 Befund (der verbleibende ist ein echter “Titel zu lang”-Fall). Neue Hilfsfunktion <code>compile_unit_multilingual_text()</code> in <code>_tia_helpers.py</code> . Besonderheiten bei Prüfpunkt 3 entsprechend ergänzt.
0.27	19.07.2026	Vierzehnter Kommentar-Bug behoben (User-Meldung, live an FB <code>Ø10rgPrg</code> verifiziert, dort absichtlich weder Autor noch Version gesetzt): Prüfpunkt 4 meldete trotz fehlender Angaben keinen Befund. Ursache: <code>GetAttribute("HeaderVersion")</code> liefert ein <code>System.Version</code> -NET-Objekt statt eines Strings, dessen <code>ToString()</code> bei nicht gesetzter Version “0.0.0.0” ergibt (.NET-Standardwert) — ein nicht-leerer String, wodurch die Version fälschlich als vorhanden galt und praktisch jeder Baustein den Prüfpunkt bestand. Live an allen 288 Bausteinen des Salzmaschine-Projekts bestätigt: 234 tragen exakt “0.0.0.0” , echte Versionen sehen dagegen wie “0.1” / “1.0” / “2.1” aus (im TIA-UI ohnehin nicht als vierteiliger Wert eingebbar). Fix: “0.0.0.0” wird wie eine leere Version behandelt. Live verifiziert: 0 → 104 Befunde projektweit. Besonderheiten bei Prüfpunkt 4 entsprechend ergänzt.
0.28	19.07.2026	Neuer Parameter <code>check_idb</code> bei Prüfpunkt 4 (User-Auftrag, analog zu <code>check_db</code> bei Prüfpunkt 2) — Standardwert <code>false</code> : Instanz-Datenbausteine werden von der Autor/Version-Prüfung ausgenommen, Global-/Array-DBs sowie FB/FC/OB bleiben unverändert und werden weiterhin immer geprüft; <code>true</code> stellt das bisherige Verhalten (inkl. Instanz-DBs) wieder her. Anders als <code>check_db</code> bei Prüfpunkt 2 betrifft dieser Parameter gezielt nur Instanz-DBs, nicht alle Datenbausteine. Parameter-Tabelle und Besonderheiten bei Prüfpunkt 4 entsprechend ergänzt. Live verifiziert

Version	Datum	Änderung
		(2 Instanz-DBs ohne Autor/Version standardmäßig ausgenommen, z. B. <code>PrgFieldbusOkDb</code> , <code>PlcTimeDb</code>).
0.29	19.07.2026	Zwei GUI-Änderungen (User-Auftrag) dokumentiert: (1) Die Prüfpunkt-Kategorien stehen auf der Eingabeseite jetzt zu je drei nebeneinander (Grid-Layout) statt strikt untereinander (Pack-Layout) — nutzt die verfügbare Fensterbreite besser aus. (2) Das Mausrad scrollt jetzt überall im Prüfpunkte-Bereich, nicht mehr nur direkt über der Bildlaufleiste (rekursive <code><MouseWheel></code> -Bindung auf Canvas und alle Kind-Widgets, da Enter/Leave-basierte Umschaltung wegen der eingebetteten Checkbox-Frame als eigenes Kind-Fenster nicht robust funktioniert hätte). Abschnitt 6.2 entsprechend ergänzt.
0.30	19.07.2026	Drei weitere GUI-Änderungen (User-Auftrag): (1) “Alle auswählen”/“Alle abwählen” sowie “Prüfung starten”/“Abbrechen” sind jetzt ca. 50 % größer (Schrift relativ zur tatsächlichen Basisschriftgröße skaliert, nicht hart kodiert) und horizontal zentriert statt linksbündig. (2) “Prüfung starten” färbt sich hellgrün, sobald mindestens ein Prüfpunkt angehakt ist — dafür auf ein klassisches <code>tk.Button</code> statt <code>ttk.Button</code> umgestellt, da <code>ttk</code> -Buttons unter den nativen Windows-Themes eine gesetzte Hintergrundfarbe ignorieren. Abschnitt 6.2 entsprechend ergänzt.
0.31	19.07.2026	Fünftehnter Kommentar-/Struktur-Bug behoben (User-Meldung, live an FC <code>OrgPrg</code> verifiziert, Netzwerke 3/8/9/12/13/14/15): Prüfpunkt 10 meldete massenhaft nicht-leere Netzwerke fälschlich als leer, aus zwei unabhängigen Gründen. (1) Ein Netzwerk mit einem einzigen Bausteinanruf und sonst keiner Logik wird im XML-Export als <code><Call></code> -Element dargestellt, nicht als <code><Part></code> — <code>compile_unit_element_count()</code> (<code>_tia_helpers.py</code> , gemeinsam genutzt mit Prüfpunkt 16 und 30) zählte bislang nur <code><Part></code> -Elemente, ein reiner Call-Aufruf ergab dadurch Elementanzahl 0. Fix: <code><Call></code> zählt jetzt ebenfalls als Element. (2) TIA erlaubt gemischte Programmiersprachen innerhalb eines Bausteins (siehe Prüfpunkt 15) — ein einzelnes Netzwerk kann SCL sein, obwohl der Baustein insgesamt z. B. FBD ist; der bisherige Skip anhand der Baustein- <code>ProgrammingLanguage</code> griff dann nicht, ein SCL-Netzwerk (<code><StructuredText></code> statt <code><FlgNet></code> / <code><Part></code>) wurde fälschlich als leer gezählt. Fix: zusätzlicher Skip anhand der Netzwerk-eigenen <code>ProgrammingLanguage</code> . Live verifiziert: 55 → 19 Befunde projektweit, alle 7 gemeldeten <code>OrgPrg</code> -Netzwerke korrekt behoben. Besonderheiten bei Prüfpunkt 10 entsprechend ergänzt.
0.32	19.07.2026	Neuer Parameter <code>ausnahme_titel_regex</code> bei Prüfpunkt 10 (User-Auftrag: leere Netzwerke werden gelegentlich absichtlich als Kapitelüberschrift innerhalb eines Bausteins verwendet, per Netzwerktitel statt Inhalt) — Standardwert <code>""</code> (deaktiviert): Passt der Titel eines sonst leeren Netzwerks auf das konfigurierte Muster, wird es nicht gemeldet, unabhängig davon, ob es tatsächlich leer ist. Live an einem echten Fall im Salzmaschine-Projekt verifiziert (<code>40Prg</code> >

Version	Datum	Änderung
		Netzwerk 1 , Titel "##### Schaltgerüst/ Bedienschrank =4805 #####"): mit passendem Regex 19 → 18 Befunde, mit nicht-passendem Regex unverändert 19. Parameter-Tabelle und Besonderheiten bei Prüfpunkt 10 entsprechend ergänzt.
0.33	19.07.2026	Siebzehnter Kommentar-/Struktur-Bug behoben (User-Meldung, live an Instanz-DB DB_PrgFieldbusOkDb verifiziert): Prüfpunkt 11 meldete bei manchen Instanz-DBs einen nicht existierenden "Member" mit demselben Namen wie die DB selbst (... > DB_PrgFieldbusOkDb > Member > DB_PrgFieldbusOkDb). Ursache: GetCrossReferences(CrossReferenceFilter.UnusedObjects) liefert für eine Instanz-DB nicht ausschließlich echte Member als Source — ein zusätzlicher Source -Eintrag mit Name == Name der DB und TypeName "Instance DB of ..." repräsentiert die DB selbst, kein Member. Der bisherige Code behandelte jeden Source blind als DB-Variable. Fix: Source s, deren Name exakt dem Namen der DB entspricht, werden übersprungen — Prüfpunkt 11b prüft ohnehin eigenständig, ob eine DB als Ganzes unbenutzt ist. Live verifiziert: 23 → 5 Befunde projektweit — alle 18 zuvor gemeldeten "DB-Member"-Befunde erwiesen sich als exakt dieser Selbst-Eintrag-Fall, keine echten Member betroffen. Besonderheiten bei Prüfpunkt 11 entsprechend ergänzt.
0.34	19.07.2026	Achtzehnter Bug behoben, korrigiert/erweitert Version 0.33 (User-Meldung: "meiner Meinung nach wird DB_PrgFieldbusOkDb -> DiagCpu -> DNNmode nicht verwendet", per Rückfrage bestätigt anhand des offiziellen Beispielcodes der V21-Openness-Referenz, Manual 03/2026, "Querverweise für STEP 7 abrufen"): Die Annahme aus Version 0.33, jeder Source in unused_result.Sources sei bereits ein fertiges unbenutztes Member, war falsch — Sources ist die Wurzel eines Baums; der in 0.33 übersprungene Source (Name == DB-Name) ist genau dieser Wurzelknoten, dessen .Children rekursiv die tatsächlich unbenutzten Member enthalten. Der bloße Skip aus 0.33 behob zwar den irreführenden Phantom-Befund, verschluckte dabei aber auch sämtliche echten, tiefer verschachtelten Treffer — der Prüfpunkt meldete dadurch de facto nie ein einziges echtes unbenutztes DB-Member. Fix: neue Hilfsfunktion unused_cross_reference_leaf_names() steigt rekursiv durch .Children ab und liefert nur echte Blattknoten; Array-Elemente werden dabei übersprungen (ein einzelnes großes Array-Member lieferte live tausende Einzelindizes als separate Blätter — analog zum Array-Skip bei Prüfpunkt 1). Live verifiziert: DiagCpu.DNNmode wird jetzt korrekt gemeldet; projektweit 5 → 3.210 Befunde (vorher blind durch den reinen Root-Skip, nach Array-Filter aber realistisch). Besonderheiten bei Prüfpunkt 11 entsprechend überarbeitet.
0.35	21.07.2026	Grundlegende Überarbeitung von Prüfpunkt 11, Neunzehnter bis Dreiundzwanzigster Bug (User-Einwand: Instanz-DBs haben keine

Version	Datum	Änderung
		eigene Logik, die Prüfung sollte sich auf die interne Verwendung im FB/FC/OB selbst beschränken, externe Zugriffe sind ohnehin unerwünscht und gehören zu Prüfpunkt 26). Instanz-DBs werden bei Prüfpunkt 11 jetzt komplett übersprungen; FB-/FC-/OB-Interface-Member (inkl. Temp) werden stattdessen direkt anhand des Baustein-codes geprüft (neue Hilfsfunktion <code>local_variable_access_names()</code> : <code><Access> / <Instance Scope="LocalVariable"></code> im XML-Export, sprachunabhängig für SCL und FBD/LAD identisch strukturiert) — garantiert strukturell, dass nur interne Verwendung zählt. Global-/Array-DBs bleiben unverändert über <code>CrossReferenceService</code> geprüft. Live verifiziert: 163 Befunde (5 PLC-Tags, 140 Global-/Array-DB-Member, 18 neue FB/FC/OB-Member-Treffer wie <code>OrgPrg > test</code>). Im Zuge dieser Überarbeitung zwei weitere, unabhängige Bugs entdeckt und behoben: Prüfpunkt 26 las den zugreifenden Baustein-namen aus dem immer leeren <code>Location.Name</code> statt <code>Location.ReferenceLocation</code>, und <code>find_source_child_by_name</code> fand Member nie, weil Kindknoten im Kreuzreferenzbaum mit dem DB-Namen als Präfix qualifiziert sind — beide behoben, zusätzlich ein Multiinstanz-Metaeintrag-Fehlalarm (<code>ReferenceType.InstanceType</code>) gefiltert. Prüfpunkt 26 meldet dadurch jetzt erstmals einen echten Treffer (<code>01PrgDb.lx_30M1StopGap</code>). Prüfpunkt 27 war ebenfalls seit Einführung wirkungslos (<code>CrossReferenceService</code> liefert am FB/FC keinen Member-Baum) — auf denselben XML-Mechanismus umgestellt (neue Hilfsfunktion <code>local_variable_write_counts()</code>, inkl. Pin-Rollen-Tabelle für FBD/LAD-Boxen wie Coil/Move/TON und SCL-Zuweisungserkennung), kreuzvalidiert gegen die alte Instanz-DB-Methode (<code>LSNTP_Server : status / error / statusID</code> je 4 Schreibzugriffe, exakte Übereinstimmung) und deckt jetzt erstmals auch FC ab. Besonderheiten bei Prüfpunkt 11, 26 und 27 entsprechend überarbeitet.
0.36	21.07.2026	Prüfpunkte 12/12b/13 auf User-Auftrag hin überprüft (kein Bug gefunden, reine Untersuchung): Rohdaten-Dump und Skalen-Check gegen Salzmaschine zeigen, dass die Kreuzreferenz-Auswertung technisch korrekt ist (keine Meta-Eintrag-Artefakte wie bei den DB-Membern in Prüfpunkt 26, <code>Location.Access</code> liefert zuverlässig Read/Write) — die 0 Befunde bei allen drei Prüfpunkten sind kein Scan-Fehler. Auffällig dabei: Das Projekt hat insgesamt nur 42 PLC-Tags mit fester Hardware-Adresse (19 Eingänge, 7 Ausgänge, exakt passend zur Namenskonvention aus Prüfpunkt 6) — die eigentliche Prozess-I/O läuft offenbar größtenteils über ein I/O-Spiegel-Pattern (Hardware-Tags werden von einem Organisationsbaustein in Datenbaustein-Member kopiert, z. B. <code>I4805_33S1</code> als Static-Member von <code>01Prg</code>), die diese drei Prüfpunkte naturgemäß nicht erfassen, da sie ausschließlich Variablentabellen-Tags mit <code>%I / %Q</code>-Adresse durchlaufen. Auf Rückfrage entschied der User, den Geltungsbereich unverändert auf echte PLC-Tags zu beschränken — die Einschränkung wurde

Version	Datum	Änderung
		stattdessen in den Besonderheiten von Prüfpunkt 12/12b/13 dokumentiert. Keine Code-Änderung in dieser Runde.
0.37	22.07.2026	Neuer Parameter <code>unterelemente_pruefen</code> bei Prüfpunkt 11 (User-Auftrag, betriebliche Praxis: UDT-/Struct-typisierte Variablen werden bei diesem Anwender oft als Ganzes an einen anderen Baustein übergeben statt feldweise einzeln zugegriffen — die bisherige feldgranulare Meldung einzelner, technisch ungenutzter Unterfelder wurde als Fehlalarm-Rauschen empfunden). Standardwert <code>false</code> : Ist ein UDT-/Struct-typisiertes Top-Level-Member auf irgendeiner Tiefe referenziert, gilt es als Ganzes als verwendet und einzelne, weiterhin ungenutzte Unterfelder werden nicht mehr gemeldet; ist es komplett unbenutzt, werden die Unterfelder trotzdem einzeln gemeldet, unabhängig vom Parameter. Bei <code>true</code> bleibt es beim bisherigen, feldgranularen Verhalten. Für DBs neue Hilfsfunktion <code>cross_reference_referenced_top_level_names()</code> (Filter <code>CrossReferenceFilter.ObjectsWithReferences</code> , das Gegenstück zu <code>UnusedObjects</code>) — Baumform live gegen Salzmaschine verifiziert (identisch zu <code>UnusedObjects : Sources[0] = DB-Wurzel, .Children = Top-Level-Member</code>). Für FB/FC/OB, wo bislang gar keine Unterfeld-Granularität existierte (nur die erste <code>Component</code> unter <code>Symbol</code> wurde je erkannt), zwei neue Hilfsfunktionen: <code>interface_section_member_paths()</code> (löst die verschachtelte <code><Sections></code> -Deklaration eines Members zu dotted Blattpfaden auf) und <code>local_variable_access_paths()</code> (löst tatsächliche Zugriffe entsprechend auf). Bei der ersten Implementierung fälschlich angenommen, TIA exportiere mehrstufige Struct-Zugriffe als ineinander verschachtelte <code><Component></code> -Elemente — live an FB <code>PrgFieldbusOk</code> (<code>DiagCpu.SubordinateIOState / DiagCpu.SubordinateState</code>) und FC <code>CtrFcParaRdWr</code> (<code>lx_Step.Sc</code>) widerlegt: TIA exportiert sie tatsächlich als flache Geschwister- <code>Component</code> -Liste direkt unter <code>Symbol</code> — vor Dokumentation dieser Runde korrigiert. Kompletter Testlauf gegen Salzmaschine bestätigt den praktischen Nutzen: <code>unterelemente_pruefen: false</code> liefert zunächst 174 Befunde (132 DB-Member, 37 FB/FC/OB-Member), <code>true</code> dagegen 8.023 (140 DB-Member, 7.878 FB/FC/OB-Member) — der überwiegende Teil des Unterschieds entsteht bei FB/FC/OB, wo die neue Feld-Granularität sonst massives Rauschen erzeugen würde. Parameter-Tabelle und Besonderheiten bei Prüfpunkt 11 entsprechend ergänzt. Direkt im Anschluss vom User an einem konkreten Fall widerlegt (DB <code>V01St</code> , Variable <code>4805_27M11</code> , Typ <code>U_VisStFcFan</code> , komplett verwendet als Aufrufparameter von <code>CtrFcFan</code> in <code>01Prg /Netzwerk 16</code> — trotzdem wurden alle Sub-Member gemeldet): Ursache war eine Quotierungs-Inkonsistenz. Der DB-Zweig übernimmt den von <code>CrossReferenceService</code> gelieferten Blattnamen ungefiltert (z. B. <code>"4805_27M11".M</code> , mit TIA-Anführungszeichen um Ziffernbeginn-Segmente, siehe <code>normalize_member_path()</code>), während die neue <code>cross_reference_referenced_top_level_names()</code> ihre Namen

Version	Datum	Änderung
		bereits normalisiert (unquoted) liefert — beim Vergleich <code>name.split(".", 1)[0]</code> not in <code>used_top_level</code> verglich der Code dadurch <code>"4805_27M11"</code> (mit Anführungszeichen) gegen <code>4805_27M11</code> (ohne), was für jeden Ziffernbeginn-Namen nie traf. Betraf projektweit sehr viele DB-Member, da im Salzmaschine-Projekt die meisten Variablennamen mit Ziffern beginnen (<code>4805_...</code>). Fix: <code>normalize_member_path()</code> jetzt auch beim DB-Zweig direkt nach dem Abschneiden des DB-Präfixes angewendet (zusätzlicher, willkommener Nebeneffekt: die angezeigten Pfade sind jetzt auch ohne Anführungszeichen lesbar). Live erneut verifiziert: <code>false</code> → 54 Befunde (12 DB-Member statt zuvor 132, <code>4805_27M11</code> korrekt nicht mehr gemeldet), <code>true</code> unverändert 8.023 (Detailmodus zeigt weiterhin korrekt alle sieben tatsächlich unbenutzten Blätter unter <code>4805_27M11.M</code>, z. B. <code>4805_27M11.M.FcPI.StWord</code>).
0.38	22.07.2026	Sechszwanzigster Bug behoben (User-Einwand direkt im Anschluss an Prüfpunkt 11: "ein normaler DB wird niemals als Ganzes verwendet, immer nur die Items in ihm"): Prüfpunkt 11b prüfte FB/FC/DB bislang einheitlich über <code>cross_reference_locations</code> (direkte Kreuzreferenz auf den Baustein selbst) — das liest nur <code>References</code>, die direkt am Root- <code>SourceObject</code> hängen. Bei FB/FC/Instanz-DB ist das korrekt, weil ein <code>CALL</code> den Baustein/die Instanz selbst als Ziel referenziert (live bestätigt: FB <code>PrgFieldbusOk</code> 541, FC <code>BibVersion</code> 0 Root-Referenzen — exakt passend zur tatsächlichen Nutzung; genutzte Instanz-DBs durchgängig 2 Root-Referenzen). Bei einem normalen Global-/Array-DB gibt es aber keine "Aufruf"-Semantik — Verwendung bedeutet dort immer Lesen/Schreiben einzelner Member, was ausschließlich unter <code>.Children</code> auftaucht, nie als direkte Referenz auf die DB-Wurzel selbst. <code>cross_reference_locations(db)</code> lieferte dadurch für jeden Global-/Array-DB immer 0 Treffer, unabhängig von der tatsächlichen Nutzung — live an 7 Stichproben (u. a. <code>V01St</code> mit 17 tatsächlich benutzten Top-Level-Membern) mit durchgängig 0 Root-Referenzen bestätigt. Der Prüfpunkt hätte dadurch praktisch jeden Global-/Array-DB im Projekt fälschlich als unbenutzt gemeldet. Fix: Global-/Array-DBs (nicht Instanz-DBs) werden jetzt über dieselbe Hilfsfunktion wie Prüfpunkt 11 geprüft (<code>cross_reference_referenced_top_level_names()</code>) — ein DB gilt als verwendet, sobald irgendein Member irgendwo im Projekt referenziert ist. Live verifiziert: 23 → 2 Befunde projektweit (alle 21 Global-/Array-DB-Fehlalarme verschwunden, die verbleibenden 2 — <code>BibVersion</code>, <code>LGF_Description</code> — sind echte unbenutzte FCs). Besonderheiten bei Prüfpunkt 11b entsprechend ergänzt. Im Zuge dieser Runde außerdem eine Nummerierungs-Kollision bei den fortlaufenden Bug-Ordinalzahlen korrigiert: die in Version 0.37 als "Zwanzigster" bzw. "Vierundzwanzigster" bezeichneten Bugs/Features kollidierten mit bereits in Version 0.35 vergebenen Nummern (19–23) — rückwirkend auf Vierundzwanzigster (<code>unterelemente_pruefen</code> -

Version	Datum	Änderung
		Feature) und Fünfundzwanzigster (Quotierungs-Fix) korrigiert, dieser PP11b-Fix ist entsprechend der Sechszwanzigste.
0.39	22.07.2026	Siebenundzwanzigster Bug behoben (User-Meldung mit selbst angelegtem Testfall: eine Instanz-DB, deren FB nirgends per CALL aufgerufen wird, deren 2 Static-Member aber extern von einem anderen Baustein zugegriffen werden — wurde von Prüfpunkt 11b trotzdem nicht als unbenutzt markiert): Jede Instanz-DB trägt unabhängig von echter Verwendung einen permanenten Meta-Eintrag in ihren Root- Locations (ReferenceType.InstanceType , @<DBName> ► Type), der nur die Typbeziehung zur eigenen FB beschreibt, keine echte Codestelle — dasselbe Muster, das in Version 0.35 bereits bei Prüfpunkt 26 auf Member-Ebene gefiltert wurde, hier aber auf Root-Ebene der Instanz-DB selbst und bislang ungefiltert. Live an der vom User angelegten Instanz-DB 01TestOrgPrgDb (FB 01OrgPrg , nie aufgerufen, aber 2 Member extern in 01Org /Netzwerk 6 zugegriffen) verifiziert: cross_reference_locations lieferte trotzdem genau 1 Eintrag — exakt dieser Meta-Eintrag, kein echter Treffer —, wodurch die Instanz-DB fälschlich immer als “benutzt” galt. Zum Vergleich liefert eine tatsächlich per CALL verwendete Instanz-DB (DB_PrgFieldbusOkDb) 2 Locations: den echten Aufruf (ReferenceType.UsedBy) plus denselben Meta-Eintrag. Fix: Bei Instanz-DBs werden InstanceType -Locations vor der Verwendungsprüfung herausgefiltert. Live verifiziert: 2 → 3 Befunde (01TestOrgPrgDb kommt korrekt hinzu, alle zuvor bestätigten Fälle unverändert). Auf User-Wunsch zusätzlich die Befundmeldung für Instanz-DBs spezifischer formuliert: "Baustein '<Name>' wird an keinem FB verwendet." statt der generischen Meldung, die weiterhin bei FB/FC/Global-/Array-DB gilt. Besonderheiten bei Prüfpunkt 11b entsprechend ergänzt.
0.40	23.07.2026	Achtundzwanzigster Bug behoben (User-Meldung: ein selbst eingefügtes AWL-Netzwerk in 010OrgPrg /Netzwerk 16 wurde von Prüfpunkt 14 nicht erkannt): Prüfpunkt 14 prüfte bislang ausschließlich die Grundsprache des Bausteins (block.ProgrammingLanguage). TIA Portal erlaubt aber, innerhalb eines Bausteins mit anderer Grundsprache (z. B. KOP/FUP) einzelne Netzwerke auf AWL umzuschalten (dieselbe Grundmechanik, die Prüfpunkt 15 — GemischteSprachenCheck — bereits über den XML-Export je CompileUnit korrekt erkennt) — die Grundsprache des Bausteins bleibt dabei unverändert, sodass der bisherige Check ein solches Netzwerk nie sah. Fix: Für Bausteine, deren Grundsprache nicht STL/SCL ist, wird jetzt zusätzlich jedes Netzwerk einzeln per XML-Export auf ProgrammingLanguage == "STL" geprüft und bei Treffer mit Netzwerknummer gemeldet; komplette AWL-Bausteine (Grundsprache STL) werden wie bisher als Ganzes gemeldet. Live gegen Salzmaschine verifiziert: 0 → 2 Befunde (010OrgPrg /Netzwerk 16 wie vom User gemeldet, plus ein bislang unentdeckter echter Fall in

Version	Datum	Änderung
		CtrlFcParaRdWr /Netzwerk 4). Besonderheiten bei Prüfpunkt 14 entsprechend ergänzt.
0.41	23.07.2026	Neunundzwanzigster Bug behoben, direkt im Anschluss an Version 0.40 entdeckt (User-Auftrag: neuer Parameter <code>scl_in_fup_ignorieren</code> bei Prüfpunkt 15, da SCL-Netzwerke in FUP-Bausteinen betrieblich weit verbreitet sind — beim Testen des neuen Parameters live gegen Salzmaschine zeigte sich, dass er wirkungslos blieb, unabhängig vom eingestellten Wert). Root Cause: <code>GetAttribute("ProgrammingLanguage")</code> auf einem Baustein liefert kein <code>System.String</code>, sondern ein <code>Siemens.Engineering.SW.Blocks.ProgrammingLanguage</code> - .NET-Enum-Objekt (repr zeigt <code><ProgrammingLanguage.FBD: 3></code>) — jeder Vergleich wie <code>get_attribute(block, "ProgrammingLanguage") == "STL"</code> oder <code>in ("SCL", "STL")</code> war dadurch immer <code>False</code>, unabhängig von der tatsächlichen Sprache. Live bestätigt: <code>str(...)</code> liefert zuverlässig den reinen Namen (<code>"FBD"</code>, <code>"SCL"</code>, <code>"DB"</code>, ...), identisch zu den bereits als Text vorliegenden Netzwerk-eigenen <code>ProgrammingLanguage</code> -Werten aus dem XML-Export. Dasselbe Muster (.NET-Objekt statt Text an einer String-Vergleichsstelle) war bereits einmal bei <code>reference_language(project).Culture</code> aufgetreten und dort schon korrekt mit <code>str(...)</code> gelöst (siehe <code>SprachenKonsistentCheck / NetzwerkBeschreibungCheck</code>) — hier aber an fünf weiteren Stellen unbemerkt, weil im Salzmaschine-Projekt bislang kein Baustein existierte, dessen Grundsprache tatsächlich SCL oder STL ist (die betroffenen Skips liefen dadurch immer ins Leere, ohne dass es auffiel). Betraf Prüfpunkt 3, 10, 14, 15 und 16. Fix: neue zentrale Hilfsfunktion <code>block_programming_language()</code> (<code>_tia_helpers.py</code>) kapselt <code>str(get_attribute(block, "ProgrammingLanguage"))</code>, an allen fünf Stellen eingesetzt. Live gegen Salzmaschine vollständig neu verifiziert (alle fünf betroffenen Prüfpunkte, Vorher/Nachher): Prüfpunkt 3 194 → 46 (reine SCL-Bausteine wurden bislang fälschlich mitgeprüft, jetzt korrekt ausgenommen), Prüfpunkt 10 unverändert 167 (hatte bereits einen funktionierenden Netzwerk-eigenen Fallback-Skip), Prüfpunkt 14 unverändert 2, Prüfpunkt 15 70 → 2 (der neue Parameter <code>scl_in_fup_ignorieren</code> greift jetzt wie beabsichtigt), Prüfpunkt 16 unverändert 0. Besonderheiten bei Prüfpunkt 3 und Parameter-Tabelle/Besonderheiten bei Prüfpunkt 15 entsprechend ergänzt.
0.42	23.07.2026	Neues Feature bei Prüfpunkt 16 (User-Frage im Anschluss an Prüfpunkt 15/16: "checkst du auch SCL Netzwerke mit zu vielen Code-Zeilen?" — Antwort war bislang nein): <code>compile_unit_element_count</code> zählt ausschließlich grafische <code><Part></code> / <code><Call></code> -Elemente und liefert für SCL-Text immer 0 — ein SCL-Netzwerk wurde dadurch unabhängig von seiner tatsächlichen Zeilenzahl nie gemeldet, egal ob als eigenständiger SCL-Baustein (bislang komplett übersprungen, da der

Version	Datum	Änderung
		Skip bislang auf "SCL" UND "STL" griff) oder als einzelnes SCL-Netzwerk innerhalb eines sonst grafischen Bausteins (nicht übersprungen, aber mit Elementanzahl 0 gezählt). Neuer Parameter <code>max_zeilen_scl</code> (Standard 50, analog zu <code>max_elemente</code>): SCL-Netzwerke werden jetzt anhand ihrer Code-Zeilen geprüft — neue Hilfsfunktion <code>compile_unit_scl_line_count()</code> (<code>_tia_helpers.py</code>) zählt die <code><NewLine></code>-Elemente im XML-Export (N Zeilenumbrüche = N+1 Zeilen; ein Netzwerk ganz ohne <code><Token></code> / <code><Access></code>-Inhalt gilt als leer, nicht als eine Zeile). Der Skip in <code>MaxNetzwerkElementeCheck</code> greift jetzt nur noch bei "STL" (AWL bleibt bewusst komplett ausgenommen, siehe Prüfpunkt 14 — gilt als veraltet statt weiter ausgebaut zu werden); pro Netzwerk entscheidet die Netzwerk-eigene <code>ProgrammingLanguage</code>, ob Elementzählung oder Zeilenzählung greift. Beide YAML-Konfigurationsdateien (<code>default.yaml</code> und <code>project_settings.yaml</code>) um den neuen Parameter ergänzt. Live gegen Salzmaschine verifiziert: 0 → 119 Befunde (Zeilenzahlen 50–459, weit überwiegend in der Siemens-Standardbibliothek <code>PrgBibSiemens/LGF</code>, einige in eigenen Bausteinen wie <code>PrgFieldbusOk</code> / Netzwerk 4 mit 256 Zeilen). <code>pytest</code> weiterhin 51/51 grün. Parameter-Tabelle und Besonderheiten bei Prüfpunkt 16 entsprechend ergänzt.
0.43	24.07.2026	Dreißigster Bug behoben (User-Meldung anhand eines eigens angelegten Testprojekts <code>S7T0159_V21_NoHW</code>, das nur eine CPU ohne jede zusätzliche I/O-Hardware enthalten sollte: "warum läuft PP17 hier trotz fehlender Hardware ohne Probleme durch?"): Prüfpunkt 17 zählte bislang schlicht <code>len(device.DeviceItems)</code> und meldete nur bei <code><= 1</code>. <code>device.DeviceItems</code> enthält aber neben der CPU immer weitere, von TIA automatisch angelegte Strukturelemente — den Baugruppenträger (<code>Rack_0</code>) sowie bei ET200SP-Stationen zusätzlich genau einen Busadapter/Schnittstellenmodul und genau ein Server-/Abschlussmodul (physisch zwingend als letztes Modul im Baugruppenträger). Live an der Salzmaschine-CPU (ET200SP, <code>pn4805-15a1</code>) bestätigt: <code>device.DeviceItems</code> liefert 10 Einträge (Rack + CPU + 6 echte I/O-Module + Server-Modul + Busadapter) — der Schwellenwert <code><= 1</code> konnte bei dieser Stationsart dadurch grundsätzlich nie unterschritten werden, selbst bei einer PLC komplett ohne I/O-Hardware. Fix: neue Hilfsfunktion <code>count_additional_hardware_modules()</code> (<code>_tia_helpers.py</code>) rechnet Baugruppenträger (<code>TypeIdentifizier</code> beginnt mit "System:Rack"), CPU selbst sowie bei ET200SP den Busadapter (erkennbar an der für diesen Zweck immer festen <code>PositionNumber == 127</code>, unabhängig vom konkreten Adaptertyp/Bestellnummer) und das Server-/Abschlussmodul (immer die höchste "normale" Positionsnummer unterhalb 127) heraus; die Prüfung greift jetzt bei <code>module_count == 0</code>. Live verifiziert (isolierter Vergleich, ohne die eigentliche Prüfpunkt-17-Meldung auszulösen, da beide Test- und Originalprojekt tatsächlich dieselben 6 I/O-Module enthielten — User-

Version	Datum	Änderung
		Fehler beim Anlegen des Testprojekts, kein Linter-Bug): Modulzahl korrekt von 10 auf 6 reduziert (nur die echten I/O-Module verbleiben), Rack/Busadapter/Server-Modul zuverlässig herausgerechnet. <code>pytest</code> weiterhin 51/51 grün. Besonderheiten bei Prüfpunkt 17 entsprechend ergänzt.
0.44	24.07.2026	Einunddreißigster Bug behoben, direkt im Anschluss an Version 0.43 am selben Testprojekt <code>S7T0159_V21_NoHW</code> gefunden (User hatte die CPU dort gegen eine F-CPU getauscht und zunächst PP18b getestet — dabei stellte sich per Rückfrage heraus, dass die F-Fähigkeit in TIA noch nicht aktiviert war, <code>Failsafe_FCapabilityActivated == False</code>, wodurch <code>SafetyAdministration</code> korrekterweise <code>None</code> lieferte; kein Linter-Bug, sondern ein noch fehlender TIA-Konfigurationsschritt. Nach Aktivierung der F-Fähigkeit funktionierte PP18b wie erwartet. User meldete danach: “PP18c leider nicht. Es ist ein Zertifikat vorhanden ... wird auch im Kommunikationsmodus verwendet. Aber trotzdem wird PP18c als Fehler markiert.”): <code>ZertifikatCheck</code> importierte <code>LocalCertificateManager</code> bislang aus <code>Siemens.Engineering.SW.Security</code> — laut Typenverzeichnis der installierten <code>Siemens.Engineering.Base.xml</code> (V21) liegt die Klasse aber unter <code>Siemens.Engineering.Security</code>, ohne <code>SW</code>-Zwischen-Namespace. Der resultierende <code>ModuleNotFoundError</code> wurde von der bewusst breiten Fehlerbehandlung (<code>except Exception: cert_manager = None</code>, gedacht für “Dienst evtl. nicht verfügbar/keine Lizenz”) stillschweigend verschluckt — <code>cert_manager</code> war dadurch projektweit und unabhängig vom tatsächlichen Zertifikatsstatus immer <code>None</code>, jede PLC wurde als “kein Zertifikat vorhanden” (Fehler) gemeldet, selbst wenn (wie hier) ein gültiges Zertifikat existierte und aktiv genutzt wurde. Root Cause per Reflection über die geladenen Assemblies gefunden (Suche nach <code>"CertificateManager"</code> über alle Typen brachte zunächst nichts, weil der Import schlicht nie geklappt hatte — erst der Namensabgleich in der XML-Dokumentation zeigte den korrekten Namespace). Fix: Import auf <code>Siemens.Engineering.Security</code> korrigiert. Live gegen <code>S7T0159_V21_NoHW</code> und zur Kontrolle zusätzlich gegen das Original-Salzmaschine-Projekt verifiziert: beide liefern jetzt identisch 1 Befund <code>[OK] pn4805-15a1 > Zertifikate > 2: Zertifikat ist gültig (gültig bis 2057-10-28)</code> — exakt das vom User genannte Zertifikat <code>pn4805-15a1/Communication-1</code>, ID 2. <code>pytest</code> weiterhin 51/51 grün. Besonderheiten bei Prüfpunkt 18c entsprechend ergänzt.
0.45	24.07.2026	Neues Feature bei Prüfpunkt 19 (User-Wunsch: “ich würde bei PP19 gerne noch mehr Felder prüfen. Mein TIA ist auf Deutsch, d. h. meine Namen in den Eigenschaften stimmen nicht mit den Strings überein”): Vollständige Liste aller von der Openness-API als Top-Level-Projektattribute gelieferten Felder ermittelt — live per <code>project.GetAttributeInfos()</code> gegen ein echtes Projekt (12 Attribute) sowie per Namensabgleich mit der deutschen V21-Referenz

Version	Datum	Änderung
		(Manual 03/2026, Abschnitt "Projektbezogene Attribute lesen"). Dabei zusätzlichen, bislang unbemerkten Bug gefunden: <code>Comment</code> (Kommentar des Projekts) wird nicht als <code>System.String</code> geliefert, sondern als mehrsprachiges <code>Siemens.Engineering.MultilingualText</code>-Objekt — dieselbe Fallstrick-Klasse wie beim mehrfach behobenen <code>Comment</code>-Attribut auf Baustein-/Tag-Ebene (siehe u. a. Runde 13/14/15), hier aber am Projekt-Objekt selbst und bislang nie aufgefallen, weil <code>Comment</code> bisher nicht Teil der Standard-<code>felder</code>-Liste war. <code>GetAttribute("Comment")</code> liefert dafür sogar direkt <code>None</code> statt des Objekts — ein generischer <code>str(get_attribute(...))</code>-Zugriff (wie ihn <code>PflichtfelderCheck</code> bislang einheitlich für alle Felder verwendete) hätte <code>Comment</code> dadurch immer als leer gemeldet, unabhängig vom tatsächlichen Inhalt, sobald ein Nutzer es zu <code>felder</code> hinzugefügt hätte. Fix: <code>PflichtfelderCheck</code> behandelt <code>Comment</code> jetzt als Sonderfall und liest es über die bereits bestehenden Hilfsfunktionen <code>read_comment()</code> / <code>reference_language()</code> (dieselbe Infrastruktur, die den ursprünglichen <code>Comment</code>-Bug an anderer Stelle bereits gelöst hat) — bei der Konfiguration selbst ist keine Sonderbehandlung nötig, <code>Comment</code> wird wie jedes andere Feld einfach in <code>felder</code> eingetragen. Live gegen <code>S7T0159_V21_NoHW</code> verifiziert: mit <code>felder: ["Author", "Comment", "Copyright", "Family", "Version"]</code> liefert der Check 3 Befunde (<code>Comment</code> , <code>Copyright</code> , <code>Family</code> leer — <code>Author</code> = "Mukara", <code>Version</code> = "Ende Dev TBE" korrekt erkannt als gefüllt), keine Exception beim <code>Comment</code>-Zugriff. <code>project_settings.yaml</code> auf Wunsch direkt um die drei neuen Felder erweitert (<code>felder: ["Author", "Version", "Comment", "Copyright", "Family"]</code>); <code>default.yaml</code> unverändert bei den bisherigen zwei Standardfeldern, aber beide Dateien um die vollständige Feldliste als Kommentar ergänzt. <code>pytest</code> weiterhin 51/51 grün. Parameter-Tabelle und Besonderheiten bei Prüfpunkt 19 um die vollständige Attributtabelle (inkl. Eignung als Pflichtfeld) und den <code>Comment</code>-Sonderfall ergänzt.
0.46	24.07.2026	Neuer Parameter bei Prüfpunkt 21 (User-Auftrag: "ich habe PP21 getestet und er scheint gut zu funktionieren. Aber ich würde gerne 2 Warnungen unterdrücken (dauerhaft). Und zwar tauchen für sehr viele schreibgeschützte Bausteine 2 Warnungen auf: '...since it is write-protected' und '...because it is not editable'. Bitte dokumentiere in den YAML und definitiv im Handbuch, dass diese Warnungen unterdrückt werden, da sie für irrelevant empfunden werden. Sonst Kontaktaufnahme mit Entwickler."): Neuer Parameter <code>ignorierte_meldungen</code> (Standard <code>[]</code>) — Liste von Regex-Mustern, die per <code>re.search</code> (bewusst nicht <code>re.match</code> wie sonst bei Regex-Parametern in diesem Projekt üblich, da ein Muster irgendwo im oft langen freien Compiler-Meldungstext treffen darf, nicht nur an dessen Anfang) gegen jede Compiler-Meldung geprüft werden; ein Treffer unterdrückt die Meldung vollständig. Regex-Filterlogik isoliert gegen Beispieltex te getestet (kein Live-Recompile

Version	Datum	Änderung
		nötig, da der Nutzer PP21 bereits selbst erfolgreich gegen das echte Projekt getestet hatte): beide Muster treffen zuverlässig auf realistische Beispielmeldungen, unbeteiligte Meldungen bleiben unverändert. Beide YAML-Dateien ergänzt: <code>default.yaml</code> mit leerer Liste plus den beiden Mustern als auskommentiertes Beispiel, <code>project_settings.yaml</code> mit beiden Mustern aktiv gesetzt (dauerhaft unterdrückt, laut Nutzer als irrelevant eingestuft). <code>pytest</code> weiterhin 51/51 grün. Parameter-Tabelle und Besonderheiten bei Prüfpunkt 21 entsprechend ergänzt.
0.47	24.07.2026	Zweiunddreißigster Bug behoben, direkt im Anschluss an Version 0.46 gefunden (User-Meldung: "kannst du checken warum meine 2 Parameter in PP21 nicht greifen?", mit den beiden vollständigen, live aus TIA kopierten Meldungstexten). Live-Recompile des Testprojekts <code>S7T0159_V21_NoHW</code> bestätigte zunächst, dass die in <code>project_settings.yaml</code> hinterlegten Muster Zeichen für Zeichen exakt mit den echten Compiler-Meldungen übereinstimmen (per <code>repr()</code> -Vergleich verifiziert) — der Fehler lag also nicht an falsch abgetippten Mustern. Ursache stattdessen im Regex-Mechanismus selbst: Beide (und, wie sich zeigte, auch das schon vorher aktive dritte Muster <code>"Compiling finished (errors: 0; warnings: 0)"</code>) enthalten literale, unescaped Klammern — als Regex sind <code>(/)</code> aber Gruppierungs-Metazeichen, keine literalen Zeichen. Isoliert nachgewiesen: Selbst ein Muster gegen den exakt eigenen Wortlaut als Suchtext (<code>re.search(muster, muster)</code>) liefert <code>False</code> , sobald eine echte Klammer enthalten ist — der Abgleich schlägt komplett stillschweigend fehl, ohne Fehlermeldung, unabhängig davon, wie exakt der Meldungstext übernommen wurde. Sowohl <code>default.yaml</code> als auch <code>project_settings.yaml</code> waren betroffen, da beide dieselbe Regex-Semantik dokumentierten. Fix: <code>ignorierte_meldungen</code> von <code>Regex-Matching (re.search)</code> auf reinen, case-insensitiven Teiltext-Abgleich (<code>text.casefold()</code> in <code>description.casefold()</code>) umgestellt — der praktische Anwendungsfall (eine konkrete, bekannte Meldung dauerhaft unterdrücken) braucht ohnehin nie Wildcards, reiner Teiltext ist hier robuster als Regex. <code>re-Import</code> in <code>metadata.py</code> entsprechend entfernt (nicht mehr benötigt). Live erneut mit allen 5 hinterlegten Mustern gegen die beiden problematischen Meldungen sowie drei weitere Beispielmeldungen getestet: alle 5 Muster treffen jetzt zuverlässig, eine unbeteiligte Beispielmeldung bleibt unverändert erhalten. <code>pytest</code> weiterhin 51/51 grün. Beide YAML-Dateien (Kommentar von "Regex-Muster" auf "literale Teiltexte, kein Regex" korrigiert) sowie Parameter-Tabelle und Besonderheiten bei Prüfpunkt 21 entsprechend aktualisiert.
0.48	24.07.2026	Prüfpunkt 22 ("Projektversion vorhanden") komplett entfernt und alle nachfolgenden Prüfpunkte um 1 nach vorne verschoben (User-Auftrag im Anschluss an eine Nachfrage zu Prüfpunkt 22: "Ist der nicht mit PP19 schon erschlagen?" — bestätigt: <code>ProjektversionCheck</code> prüfte

Version	Datum	Änderung
		lediglich, ob <code>project.Version</code> leer ist, exakt dieselbe Prüfung, die Prüfpunkt 19 bereits über sein <code>felder</code>-Array abdeckt, sobald <code>"Version"</code> darin steht — was seit jeher der Standardwert ist. “Option 4”: Prüfpunkt 22 ersatzlos streichen statt ihn auszubauen oder Prüfpunkt 19 einzuschränken). Betraf durchgängig Code, beide YAML-Dateien und Handbuch: <code>ProjektversionCheck</code>-Klasse und Registry-Eintrag entfernt (<code>metadata.py</code> , <code>registry.py</code>); alle Prüfpunkt-Nummern 23–35 in Docstrings/Kommentaren um 1 verringert (22–34), betroffen <code>registry.py</code> , <code>libraries.py</code> (Modul- und Klassen-Docstrings, inkl. Kreuzverweise wie “Prüfpunkt-11/26-Überarbeitung” → “11/25”), <code>structure.py</code> und <code>_tia_helpers.py</code> (Kreuzverweise auf die verschobenen Styleguide-Prüfpunkte 25–27 → 24–26). Beide YAML-Dateien: <code>projektversion</code>-Konfigurationsblock entfernt, alle Kommentare der Kategorien Bibliotheken/Styleguide neu nummeriert, Kopfkomentar “35 Prüfpunkte” → “34”. Handbuch: Abschnitt 10.5 (jetzt “Prüfpunkte 19-21”) um Prüfpunkt 22 gekürzt, Abschnitte 10.6/10.7 samt aller Unterüberschriften, internen Anker-Links (u. a. auf den ehemaligen Prüfpunkt 26) und der Übersichtstabelle am Kapitelanfang neu nummeriert; alle “35 Prüfpunkte”-Erwähnungen (Kapitelübersicht, Abschlusshinweis, Besonderheiten bei Prüfpunkt 1b/1c) auf “34” korrigiert. Auf Nutzerwunsch zusätzlich die ursprüngliche, außerhalb des Code-Repos liegende Checkliste <code>doku_etc/Pruefpunkte.md</code> (in <code>tia-linter/doku_etc/</code> , nicht <code>tia-linter/code/</code>) identisch angepasst, damit die Nummerierung projektweit konsistent bleibt. Datierete Änderungshistorien-Einträge in Anhang C sowie die Runden-Historie in <code>doku_etc/review_fortschritt.md</code> bleiben als Zeitpunkt-Aufzeichnungen bewusst bei der zum jeweiligen Zeitpunkt gültigen Nummerierung. <code>pytest</code> weiterhin 51/51 grün; Config-Ladetest bestätigt 43 statt 44 Checks und das Fehlen von <code>projektmetadaten.projektversion</code> in beiden YAML-Dateien.
0.49	24.07.2026	Verbesserung bei Prüfpunkt 25 (User-Feedback: “Funktioniert hervorragend. Eine Verbesserung hätte ich noch ... Hast du zu dem Zeitpunkt evtl. auch aus welchem Netzwerk heraus? So dass du schreiben könntest: 01Org / NW6?”): Die Meldung nannte bislang nur den zugreifenden Baustein, nicht die Netzwerk-/Codestelle darin. Die Information steckte bereits im selben <code>ReferenceLocation</code>-String (Format <code>@<Baustein> ► <Ort></code>), wurde bisher aber nach dem <code>►</code>-Zeichen verworfen. Live an 5 verschiedenen Bausteinen/187 Kreuzreferenz-Einträgen des Salzmaschine-Projekts die tatsächlich vorkommenden Formate gesammelt: <code>NW6</code> bzw. <code>NW6 (<Netzwerktitel>)</code> bei grafischen Netzwerken, <code>Ln: x C1: y</code> bei einem eigenständigen SCL-Baustein ohne Netzwerkgliederung (z. B. <code>LSNTP_Server</code>), sowie <code>NW 10 Ln: x C1: y</code> bei einem einzelnen SCL-Netzwerk innerhalb eines sonst grafischen Bausteins (z. B. <code>4805PrgMan /NW10</code>). Fix: <code>_LOCATION_PREFIX</code>-Regex erweitert, um zusätzlich zum Bausteinnamen auch den Text zwischen <code>►</code> und einer eventuellen Titel-Klammer zu erfassen (Mehrfach-Leerzeichen dabei

Version	Datum	Änderung
		auf eines reduziert); der Netzwerktitel selbst wird bewusst nicht mit ausgegeben. Live verifiziert: Meldung für 01TestOrgPrgDb / 1x_xx lautet jetzt exakt wie vom User gewünscht '010rg / NW6' (vorher nur '010rg'), zweiter Treffer 01PrgDb / 1x_30M1StopGap entsprechend '01Vis / NW5' . pytest weiterhin 51/51 grün. Beispiel und Besonderheiten bei Prüfpunkt 25 entsprechend ergänzt.
0.50	24.07.2026	Erweiterung bei Prüfpunkt 26 (User-Auftrag: "Funktioniert auch, aber ich brauch auch hier noch eine Erweiterung. Es gibt innerhalb von FBs und FCs sowohl Output-Tags als auch InputOutput-Tags. Bei denen müsste die gleiche Prüfung durchgeführt werden. Darf nur 1x beschrieben werden."): OutputMehrfachBeschriebenCheck prüfte bislang ausschließlich die Interface-Section "Output" . Fix: Zusätzlich wird die Section "InOut" (VAR_IN_OUT, bereits an anderer Stelle im Projekt als Sektionsname bestätigt, siehe comments.py / structure.py) gleichermaßen auf Mehrfach-Schreibzugriffe geprüft; die Meldung nennt die jeweilige Parameterart (Output-Parameter '...' bzw. InOut-Parameter '...'). Live gegen das Salzmaschine-Projekt vorher/nachher verglichen (git stash -Vergleich): 229 → 340 Befunde, die 111 neu hinzugekommenen sind durchgängig korrekt als InOut-Parameter beschriftet, die ursprünglichen 229 Output-Befunde blieben unverändert. pytest weiterhin 51/51 grün. Titel, "Was wird geprüft?" und Besonderheiten bei Prüfpunkt 26 entsprechend ergänzt; doppelte "Besonderheiten"-Überschrift (redaktionelles Überbleibsel aus einer früheren Version) im Zuge dessen zu einem Abschnitt zusammengeführt.
0.51	24.07.2026	Neuer Parameter bei Prüfpunkt 30 und Prüfpunkt 34 (User-Auftrag im Anschluss an eine Erklärung des Unterschieds beider Prüfpunkte: "Kannst du sowohl bei PP30 als auch bei PP34 die Strings, die du in den Kommentaren suchst (schreibschutz, ...), so wie bei PP21 mit in die YAML-Datei als Parameter aufnehmen. Dann ist der Benutzer flexibler."): Beide Checks (KnowHowSchutzCheck , SchreibschutzCheck , libraries.py) hatten die gesuchten Dokumentationshinweise bislang fest im Code verdrahtet ("know-how" / "knowhow" bzw. "schreibschutz" / "write-protect"). Neuer Parameter dokumentations_hinweise bei beiden (Standardwerte identisch zu den bisherigen fest verdrahteten Texten) — Liste literaler Teiltexen, case-insensitiv, kein Regex (bewusst analog zu ignorierte_meldungen bei Prüfpunkt 21, siehe Version 0.47: Klammern/Sonderzeichen in einer eigenen Formulierung hätten als Regex sonst denselben Fallstrick). Beide YAML-Dateien ergänzt. pytest weiterhin 51/51 grün (Config-Ladetest bestätigt korrekte Standardwerte in beiden Dateien). Parameter-Tabelle und Besonderheiten bei Prüfpunkt 30/34 entsprechend ergänzt.
0.52	25.07.2026	Prüfpunkt 27 ("Multi-Instanzen statt Einzel-Instanzen") komplett entfernt (User-Entscheidung nach dem in Version 0.-

Version	Datum	Änderung
		Historie/ review_fortschritt.md dokumentierten Zweifel aus Runde 52: “die Einzelinstanz-DB landet in den allermeisten Fällen im Systemordner und wird dann sowieso nicht geprüft”) und alle nachfolgenden Prüfpunkte 28–34 um 1 nach vorne auf 27–33 verschoben — analog zur Streichung von Prüfpunkt 22 in Version 0.48 betraf das durchgängig Code (MultiInstanzenCheck -Klasse und _MULTI_INSTANCE_CANDIDATES -Liste aus libraries.py entfernt, Registry-Eintrag entfernt, alle Docstrings/Kommentare 28–34 umnummeriert), beide YAML-Dateien (Konfigurationsblock entfernt, verbleibende Kommentare neu nummeriert, Kopfkomentar “34 Prüfpunkte” → “33”), README sowie die außerhalb des Code-Repos liegende Checkliste doku_etc/Pruefpunkte.md . Zusätzlich: Prüfpunkt 26 in der Oberfläche umbenannt zu “InOut und Output-Tag nur einmal beschrieben” (Titel/Anzeigenname, der Config-Schlüssel output_mehrfach_beschrieben blieb unverändert). PDF-Report um zwei neue Kapitel erweitert — Kapitel 7 dieses Handbuchs entsprechend überarbeitet (siehe dort für Details zu “Prüfpunkte-Übersicht” und “Anhang A”). GUI-Kosmetik (nicht gesondert dokumentiert, siehe README/Code): Log-Fenster verkleinert, Prüfpunkte-Übersicht auf der Eingabeseite von 3 auf 4 Kategorie-Spalten umgestellt. pytest weiterhin 51/51 grün.
0.53	25.07.2026	Umnummerierung auf User-Wunsch: Bei allen Prüfpunkten, die mindestens einen “b”-Unterpunkt haben, trägt der bisherige “nackte” Basispunkt jetzt zusätzlich ein “a” — Prüfpunkt 1/1b/1c → 1a/1b/1c, 11/11b → 11a/11b, 12/12b → 12a/12b, 18/18b/18c → 18a/18b/18c (Prüfpunkt 17/17b bewusst ausgenommen: 17b ist in doku_etc/Pruefpunkte.md nur als unimplementierter Planungs-Stub “(spätere Version)” vermerkt, existiert nirgends im tatsächlichen Programm — 17 bleibt daher unverändert “17”, kein “17a”). Betrifft registry.py (nummer -Felder sowie alle Beschreibungstexte), sämtliche Check-Module mit entsprechenden Docstring-Querverweisen (comments.py , structure.py , hardware.py , libraries.py , _tia_helpers.py), beide YAML-Dateien (Kommentare), tests/test_tia_helpers.py / test_models.py , README, dieses Handbuch (Überschriften, Fließtext-Querverweise und die zugehörigen Anker-Links — z. B. #prüfpunkt-1-... → #prüfpunkt-1a-...) sowie doku_etc/Pruefpunkte.md . Kapitelweite Bereichsangaben wie “Prüfpunkte 1-4” oder “Prüfpunkte 17-18c” bleiben bewusst unverändert (sie bezeichnen eine Spanne, keinen Einzelpunkt). Historische Zeitpunkt-Aufzeichnungen (Anhang-C-Einträge dieser Tabelle vor Version 0.53, doku_etc/review_fortschritt.md) bleiben wie bei früheren Umnummerierungen bewusst bei der zum jeweiligen Zeitpunkt gültigen Nummerierung. Die GUI übernimmt die neue Nummerierung automatisch (zeigt definition.nummer direkt aus der Registry, keine eigene Code-Änderung nötig). pytest weiterhin 51/51 grün.

Version	Datum	Änderung
0.54	25.07.2026	Performance-Optimierung, kein Verhaltensunterschied bei einem der 34 Prüfpunkte (gezielter Review auf User-Auftrag, vollständige Analyse siehe XML-Optimierung-Analysebericht.md im Vault-Root): Bestandsaufnahme ergab, dass <code>export_block_xml()</code> (Baustein-XML-Export für Netzwerk-/Interface-Analyse, <code>_tia_helpers.py</code>) von bis zu 10 unabhängigen Stellen in <code>comments.py</code> / <code>structure.py</code> / <code>libraries.py</code> ohne jeden Cache aufgerufen wurde — ein typischer FB wurde dadurch pro Lint-Lauf bis zu 8-mal exportiert (Prüfpunkte 1c, 3, 10, 11a, 14, 15, 16, 26), zusätzlich einmal je Instanz-DB über Prüfpunkt 25. Hochrechnung anhand der dokumentierten Bausteinzahlen des Salzmaschine-Projekts (288 Bausteine, davon 127 FB, restliche Aufteilung geschätzt): ca. 1.777 → 237 <code>Export()</code>-Aufrufe, Faktor ≈7,5 (~87 % weniger). Zweiter, unabhängig gefundener Fall: <code>ProjectTextComments.load()</code> (<code>project_texts.py</code>, projektweiter Excel-Export aller Projekttexte für DB-Variablen-Kommentare) wurde von den Prüfpunkten 1a/1b/1c jeweils pro PLC-Software erneut aufgerufen, statt das Ergebnis wiederzuverwenden. Fix: Beide Exporte cachen ihr Ergebnis jetzt lauf-gebunden (<code>export_block_xml()</code> /neue Funktion <code>reset_export_cache()</code>; <code>ProjectTextComments.load()</code> /neue Methode <code>.reset_cache()</code>). Der Cache-Key ist bewusst ein stabiler String (<code>plc_software.Name + block.Name</code>), keine Objektreferenz — <code>runner.py</code> verbindet die TIA-Portal-Session bei größeren Projekten periodisch neu (siehe Abschnitt 4.2), wonach alle vorherigen .NET-Objekte disposed sind. <code>runner.py</code> leert daher beide Caches explizit bei jedem (Re-)Connect (<code>_reset_run_caches()</code>); ein zwischenzeitlich fehlgeschlagener Export wird dadurch beim nächsten Zugriff erneut versucht statt dauerhaft als fehlgeschlagen zu gelten — fehlgeschlagene Exporte werden aus demselben Grund grundsätzlich nie gecacht. Alle 10 Aufrufstellen von <code>export_block_xml()</code> auf die neue Signatur <code>export_block_xml(block, plc_software)</code> umgestellt, Prüflogik und Rückgabewerte unverändert. Neuer Test <code>TestExportBlockXmlCache</code> (<code>tests/test_tia_helpers.py</code>, 5 Fälle: Cache-Hit bei gleichem Baustein, unabhängige Bausteine, gleicher Blockname auf zwei PLCs, Reset erzwingt Re-Export, fehlgeschlagener Export wird nicht gecacht) — reine Mock-Verifikation ohne TIA Portal/pythonnet, da auf Linux entwickelt. <code>pytest</code> 51/51 → 56/56 grün. Abschnitt 4.2 um einen Hinweis auf den Cache-Reset bei Reconnect ergänzt.